UNIVERSITI TEKNOLOGI MARA

SECURE CLOUD STORAGE WITH AI MALWARE ANALYZER AND AES ENCRYPTION

MUHAMMAD AQEEF BIN AZHAR

BACHELOR OF COMPUTER SCIENCE (Hons.)
NETCENTRIC COMPUTING

January 2026

UNIVERSITI TEKNOLOGI MARA

SECURE CLOUD STORAGE WITH AI MALWARE ANALYZER AND AES ENCRYPTION

MUHAMMAD AQEEF BIN AZHAR

Thesis submitted in fulfilment
of the requirements for the degree of
Bachelor of Computer Science (Hons).
Netcentric Computing

Faculty of Computer and Mathematical Sciences

January 2026

SUPERVISOR APPROVAL

Based on this supervision and evaluation of the project formulation thesis met on 7 January 2026 to conduct the final examination of Muhammad Aqeef bin Azhar on his for project formulation thesis entitled “Secure Cloud Storage With AI Malware Analyzer and AES Encryption” in accordance with Universiti Teknologi MARA Act 1976 (Akta 173). It is recommended that the student ne awarded the relevant degree. The Supervisor was as follows:

Approved by,



DR FAKARIAH HANI BINTI MOHD ALI

Supervisor, Lecturer

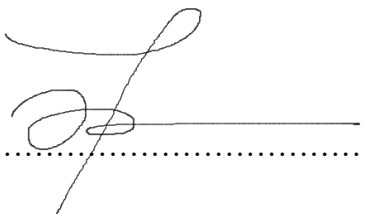
Faculty of Computer and Mathematical Sciences Universiti Teknologi MARA

Date: _____

AUTHOR'S DECLARATION

I declare that the work in this thesis was carried out in accordance with the regulations of Universiti Teknologi MARA. It is original and is the results of my own work, unless otherwise indicated or acknowledged as referenced work. This thesis has not been submitted to any other academic institution or non-academic institution for any degree or qualification.

I, hereby, acknowledge that I have been supplied with the Academic Rules and Regulations for Undergraduate, Universiti Teknologi MARA, regulating the conduct of my study and research.

Name of Student	:	MUHAMMAD AQEEF BIN AZHAR
Student ID. No.	:	2023414888
Programme	:	Bachelor of Computing Science (Hons) Netcentric Computing
Faculty	:	Faculty of Computer and Mathematical Sciences
Thesis Title	:	SECURE CLOUD STORAGE WITH AI MALWARE ANALYZER AND AES ENCRYPTION
Signature of Student	:	 ……..
Date	:	JULY 2025

ACKNOWLEDGEMENT

First and foremost, I am deeply grateful to God for granting me the opportunity to pursue my degree and for seeing me through this long and challenging journey to its successful completion. My heartfelt appreciation goes to my supervisor, Dr. Fakariah Hani Binti Mohd Ali, whose unwavering support, guidance, and encouragement have been invaluable throughout this project. Her dedication, expertise, and contributions have played a crucial role in its success.

I also extend my sincere gratitude to my CSP650 lecturer, Ts Dr Nurshahrily Idura Binti Haji Ramli s for her support and assistance. Her guidance and expertise in various areas have significantly contributed to the success of this project.

To my beloved parents, Azhar bin Che Omar and Suraya binti Mokhtar, as well as my family members and friends, I am profoundly thankful for your constant understanding, support, and encouragement. Your love and belief in me have been the foundation of my strength throughout this journey.

Lastly, I would like to acknowledge myself for the hard work, dedication, and perseverance I have invested in this project. The challenges along the way have been significant, but through determination and resilience, I was able to overcome them and achieve my goals. I am proud of what has been accomplished, and I am confident that the knowledge and skills gained during this journey will serve me well in the future.

To everyone who contributed to the success of this project, I offer my deepest gratitude. Your support and contributions will always be remembered and deeply appreciated.

ABSTRACT

The proposed project aims to develop a secure cloud storage system that integrates advanced AI malware analysis and sufficient AES encryption to safeguard user data. The system leverages the AI malware analyzer to analyze uploaded files for potential malware threats, ensuring clean files are stored. For enhanced security, all data is encrypted using the AES algorithm, providing a strong layer of protection against unauthorized access. This innovative combination of file analysis and encryption offers a comprehensive approach to secure cloud storage, catering to both individual and organizational needs. The malware analysis module utilizing the Random Forest model to checks each of the files that are uploaded to the system with identified malware signatures and then permits the file to get to the system. It will make sure that stored files are clean and verified only, significantly minimizing the risk associated with malware propagation in the storage environment. At the same time, the encryption variant makes use of the AES protocol that not only converts data to an encrypted form, but also preserves their integrity and authenticity by using inbuilt authentication measures. The system applies a simple user interface, which has supported mechanisms and capabilities like auto generation of the AES key, sharing of key in QR codes and policy acknowledgments to make it easy and transparent. The unique combination of effective use of malware identification solutions and powerful encryption algorithm ensures a high-level of security of cloud storage, which is why it can be used by individual people and organizations who need to protect valuable data. The result reveals that it is possible to merge the threat detection in real-time with cryptography to produce a safe, effective, and user-friendly cloud storage system.

TABLE OF CONTENT

SUPERVISOR APPROVAL 3
AUTHOR'S DECLARATION 4
ACKNOWLEDGEMENT 5
ABSTRACT 6
TABLE OF CONTENT 7
LIST OF FIGURES 11
LIST OF TABLE 11
CHAPTER 1: INTRODUCTION 12
1.0 Background Study 12
1.1 Problem Statement 13
1.2 Project Aim and Objectives 14
1.3 Project Scope 15
1.4 Significance of project 16
CHAPTER 2: LITERATURE REVIEW 18
2.0 Introduction 18
2.1 Cloud Storage System Architecture 18
2.1.1 Core Components of Cloud Storage Systems 18
2.1.2 Security Threats in Cloud Storage Environments 20
2.2 Types of Malware 21
2.2.1 Document-Based Malware 21
2.2.2 Executable and Script-Based Malware 22
2.2.3 Archive and Multimedia-Based Malware 23
2.3 Malware Detection Approaches 24
2.3.1 Static Analysis for Malware Detection 24
2.3.2 Dynamic Analysis for Malware Detection 25
2.3.3 Justification for Using Static Analysis 25
2.4 Limitations of Traditional Malware Detection in Cloud Storage 27
2.4.1 Signature-Based Detection Limitations 27
2.4.2 Zero-Day and Polymorphic Malware Challenges 28
2.4.3 False Positives and False Negatives 28
2.5 Machine Learning for Malware Detection 29
2.5.1 Common Machine Learning Algorithms 29
2.5.2 Feature Engineering for Multi-File Malware Detection 29
2.5.3 Recent Studies on AI-Based Malware Detection in Cloud Systems 30

2.6 Feature Extraction Techniques 31
2.6.1 Structural Features 31
2.6.2 Content and Script-Based Features 31
2.6.3 Metadata and Statistical Features 32
2.7 Datasets for Cloud Malware Detection Research 32
2.7.1 Challenges in Dataset Collection 32
2.7.2 Balanced and Imbalanced Dataset Considerations 33
2.8 Encryption Algorithm 34
2.8.1 Advance Encryption Standard (AES) 34
2.8.2 Data Encryption Standard 36
2.8.3 RSA (Rivest-Shamic-Adleman) 37
2.8.4 Eliptic Curve Cryptography (ECC) 37
2.8.5 Comparison of Encryption Algorithms and Focus on AES 38
2.9 Research Gap and Justification 39
2.9 Summary of Related Works 42
2.10 Summary 43
CHAPTER 3 : METHODOLOGY 45
3.1 Introduction 45
3.2 Project Framework 46
3.2.1 Requirement Analysis 49
3.2.2 Software 50
3.2.3 Hardware 53
3.2.4 Project Timeline 53
3.3 System Design 54
3.3.1 Random Forest Machine Learning Model 54
3.3.2 Dataset Used 55
3.3.3 Physical Design 56
3.3.4 Use Case Diagram 57
3.3.5 Flowchart 66
3.3.6 Entity-Relationship Diagram 74
3.3.6.1 USER Entity 75
3.3.6.2 AUTHENTICATION Entity 76
3.3.6.3 FILE Entity 77
3.3.6.4 KEY Entity 78
3.3.6.5 ACTION Entity 79
3.3.7 User Interface 80

3.3.7.1 Log In interface 80
3.3.7.2 Cloud Dashboard 80
3.3.7.3Upload File Interface 81
3.3.7.4 Analyzing Interface 81
3.3.7.5 Successful Analysing 82
3.3.7.6 Encryption Interface 82
3.3.7.7 Successful Encryption Interface 83
3.3.7.8 File Dashboard 83
3.3.7.9 File Decrypt Interface 85
3.3.7.10 Successful Decrypt Interface 86
3.4 Implementation 86
3.5 System Testing 88
3.5.1 Functional testing 88
3.5.2 Test Case Generation 88
3.5.3 Metrics to Measure System Performance 91
3.6 Documentation 94
3.7 Summary 94
CHAPTER 4 : RESULT AND FINDINGS 95
4.1 Introduction 95
4.2 Secure Cloud Storage with AI Malware Detection and AES Encryption 95
4.3 User View 96
4.3.1 Access to Landing Page 96
4.3.2 User Registration 97
4.3.3 Access to Login Page 97
4.3.4 Two-step verification page 98
4.3.5 Access to Dashboard Page 99
4.3.6 Profile Management Page 99
4.3.7 File Upload and Malware Analysis 100
4.3.7 Flagged File and Handling of Malicious Uploads 103
4.3.8 File Encryption 105
4.3.9 File Decryption 107
4.3.10 Download and Delete Files 108
4.4 Testing Results 110
4.4.1 Functional Testing 110
4.4.2 Machine Learning Model Evaluation 111

4.4.3 Held-out Validation of the Model 112
4.4.4 Encryption and Decryption Performance 113
4.4.5 Storage of Encrypted Files on the Server 113
4.4.6 Data Integrity Verification 114
4.4.7 Malware Detection and Corroboration 115
4.5 Discussion 116
4.6 Summary 117
CHAPTER 5 : CONCLUSION 118
5.0 Introduction 118
5.1 Purpose of the Project 118
5.2 Benefits 119
5.3 Limitations 119
5.4 Future Recommendation 120
5.5 Conclusion 120
REFERENCES 121
APPENDIX A: AI DETECTION REPORT 124
APPENDIX B: SIMILARITY INDEX RECEIPT 124

LIST OF FIGURES

Figure 2.1 Basic component for Cloud Storage 19
Figure 2.2 Phishing attack situation. 22
Figure 2.3 Payload Attack Situation. 23
Figure 2.4 AES encryption architecture design 35
Figure 2.5 AES Encryption and decryption process diagram 35
Table 2.2 Comparison of Cryptographic Method 39
Table 2.3 Related Works 42
Figure 3.1 Waterfall Model 46
Figure 3.2 Project Gannt Chart 53
Figure 3.3 Random Forest Model 55
Figure 3.4 Physical Design of System 56
Figure 3.5 Use case diagram 57
Figure 3.6 Flowchart of the system 66
Figure 3.7 Flowchart of the AI Malware Analyzer 71
Figure 3.8 Entity-Relation Diagram 74
Figure 3.9 Log in interface 80
Figure 3.10 Log in interface 81
Figure 3.11 Log in interface 81
Figure 3.12 Log in interface 82
Figure 3.13 Log in interface 82
Figure 3.14 Log in interface 83
Figure 3.15 Log in interface 83
Figure 3.16 Log in interface 85
Figure 3.17 Log in interface 85
Figure 3.18 Log in interface 86
Figure 4.20 Confusion matrix of the Random Forest classifier 112

LIST OF TABLE

Table 2.1 Comparison analysis method 26
Table 3.1 Waterfall model table 47
Table 3.2 Software Requirements 50
Table 3.3 Hardware Requirements 53
Table 3.3 Use Case Description Table 58
Table 3.4 User Entity 75
Table 3.5 Authentication Entity 76
Table 3.6 File Entity 77
Table 3.7 Key Entity 78
Table 3.8 Action Entity 79
Table 3.9 Test Case Generation 88
Table 3.7 Confusion Matrix Table 93
Table 4.1 Functional Testing Output 110
Table 4.2 Random Forest Performance on the Stratified Test Split 111
Table 4.3 Comparison of Validation Designs 112
Table 4.4 AES-256-CBC Encryption and Decryption Time 113
Table 4.5 Hash Comparison Before and After Encryption 115
Table 4.6 Results of Malware Detection and Corroboration 115

CHAPTER 1: INTRODUCTION

1.0 Background Study

The trend toward using cloud storage for confidential data has created an increased need for high-level data security protocols for both organizations and individuals. Cyberattacks are constantly changing, and as a result, many traditional signature-based scanners identifying malware are unable to identify new types of malware or malware that has been altered to evade detection, nor can they identify previously unknown malware hiding in files uploaded to the cloud. The implications of this are significant for the cloud where organizations face the risk of ransomware attacks, data loss due to large amounts of data being stolen and the potential for large numbers of malicious files being shared. As a result, there is an urgent need for Artificial Intelligence AI based, intelligent, adaptive and automated malware scanning systems to scan files for malware and notify users before they upload them to the cloud.

AI based malware detection is a new concept that relies on machine learning to analyze the characteristics of files to identify emerging patterns of malicious behavior that traditional methods of malware scanning may fail to detect. In contrast to existing methods of threat detection used by cloud service providers, AI-based malware detection provides a better opportunity for organizations to proactively detect threats faster and with a lower likelihood of missing any. Additionally, Organizations can further strengthen their security by using Advanced Encryption Standard (AES) to encrypt legitimate user data so that it remains confidential, thus preventing unauthorized access to that data in the event of a breach.

This initiative addresses the increasing demand for secure cloud storage by combining AI powered Malware detection and AES encryption into an all in one Security platform. By combining intelligent automated Threat Analysis with strict Data Protection methods, the goal is to create a robust, effective and reliable Cloud Storage Solution to help overcome today's security challenges.

1.1 Problem Statement

The fast world transition to digital operations through clouds has resulted in the integration of platforms such as cloud storage as a necessity, but this has also brought in dire security issues, and the absence of security has been seen as the greatest obstacle in the extensive use of cloud computing. Another aspect of the threat environment that has been especially consistent is document-based attacks, in which there has been a rise in the popularity and persistence of malicious files as an attack vector (Liu et al., 2025). Consequently, conventional cloud-based storage often does not identify malware that lies within uploaded files, leaving both users and the system itself vulnerable to severe cyber-attacks (Liu et al., 2025).

Signature-based scanners, including services such as VirusTotal, remain a widely used and effective first line of defence, but they depend on databases of previously identified malware signatures and can therefore struggle to identify entirely new or heavily obfuscated threats (Kim et al., 2025). This limitation is especially pronounced for advanced attacks such as zero-day malware, which is too recent to be captured by existing signature databases (Deldar and Abadi, 2024), and polymorphic or JavaScript-based malware that uses metamorphic techniques to continually alter its appearance and evade detection. Relying on a signature-based scanner in isolation therefore leaves a structural gap: a file can pass VirusTotal's checks yet still contain malicious intent encoded in its internal structure rather than matched against a known signature.

Thus, the main weakness of the existing ecosystem is a critical gap in the development of an integrated solution where signature-based detection, AI-based structural analysis, and cryptographic storage work together to manage secure documents holistically. Existing systems treat these components as either standalone tools or a simple pass/fail gate, rather than combining them into a layered decision process. This lack of integration means users do not have an automatic and intelligent system that can weigh multiple, independent indicators of risk to determine the security of an uploaded file prior to encryption and cloud storage, and requires a strong, proactive, layered solution as a scalable means of primary defence (Sinha & Sai, 2023).

1.2 Project Aim and Objectives

1. To extract structural features from files for machine-learning-based malware analysis.

2. To design verdict engine that combines VirusTotal signature-based scanning with the AI classifier's output to produce a three-way file disposition clean, suspicious, or malicious.

3. To develop a Python Flask API that integrates the AI detection module with the cloud storage PHP system.

4. To evaluate a Random Forest classifier to distinguish benign and malicious PDF files based on their structural features.

1.3 Project Scope

The focus of this project is the creation of a secure cloud storage platform advanced with a layered malware detection process combining signature-based scanning and an AI-based structural analyser together with AES-256-CBC encryption. The target users are individuals and organisations that frequently handle sensitive documents, such as personal records, financial statements, contracts, and confidential reports, in PDF format.

The system incorporates a verdict engine that evaluates each uploaded PDF file using two independent signals: VirusTotal's signature-based scanning as the primary detector, and a Random Forest machine learning model trained on structural PDF features as a supporting AI signal. Based on the combined result, a file is classified as clean, suspicious, or malicious; malicious files are rejected outright, suspicious files are stored under a flagged status for review, and clean files proceed directly to encryption. This two-signal approach allows the system to catch threats that either detection method alone may miss, extending beyond the limits of conventional single-method malware scanning.

In complement, the system uses AES-256-CBC encryption to protect the confidentiality of files at rest, with encryption keys generated per file and managed separately from the encrypted data itself. The system provides users with their AES key both as plain text and as a scannable QR code, and delivers the key by email upon successful upload as an additional, non-blocking convenience channel.

The backend is developed using Python and Flask to serve the AI malware detection model as an API, while the web platform is developed using PHP to handle user interaction, file uploads, encryption routines, and authentication. Encrypted file blobs are stored in a cloud storage bucket, while user records, file metadata, verdicts, and encryption keys are stored in a managed SQL database, reflecting the stateless nature of the cloud deployment environment. Both the Flask API and PHP web application are deployed as containerised services on Google Cloud Platform.

The project also involves designing a simple and accessible user interface through which users can upload files, view scan results, generate and retrieve AES keys via QR code, and manage their stored documents Overall, the project delivers a cloud storage service that pairs signature-based and AI-driven malware detection with AES-256-CBC encryption, supported by a convenient and transparent user experience for secure document management.

1.4 Significance of project

The secure and reliable cloud storage platform developed in this project, combining signature-based scanning, AI-driven structural malware analysis, and AES-256-CBC encryption, offers a practical and affordable approach to safeguarding confidential electronic files. Many existing security solutions rely solely on either signature-based detection, which is fast but limited to known threats, or costly standalone AI systems that are rarely integrated directly into the file storage workflow. This project combines both approaches into a single verdict engine, allowing the system to catch threats that a signature-based scanner alone would miss, without discarding the speed and reliability that signature-based detection already provides. This layered approach reduces reliance on any single point of failure in the detection process and offers stronger protection than either method used in isolation, while remaining accessible to small businesses, organisations, and individual users who may not have the resources for enterprise-grade commercial security solutions. By identifying and flagging suspicious or malicious uploads before they are stored, the system reduces the likely financial losses, operational disruption, and recovery costs associated with cyber-threats.

This project also contributes to the creation of safer digital infrastructures and greater confidence in cloud-based services. Combining structural AI analysis with an established signature-based detector reflects a practical, defensible approach to malware detection that acknowledges the strengths and limitations of each method, rather than treating either as a complete solution on its own. On a community and societal level, the system encourages safer file-handling habits by making layered threat detection a transparent and convenient part of everyday cloud storage use, helping to reduce the spread of malicious files through shared networks and online platforms. The system's use of freely available cloud infrastructure and open-source tools keeps costs low, allowing even resource-constrained users and organisations to benefit from a multi-layered security approach.

Overall, this project addresses a significant gap in current cloud security practice: the disconnect between AI-based malware research and its practical, combined use alongside existing signature-based tools in a deployable, end-to-end system. By demonstrating that structural AI analysis and signature-based scanning can be integrated into a single automated verdict process, alongside AES-256-CBC encryption for data at rest, this project improves digital

safety, strengthens cybersecurity awareness, and provides a reliable, reproducible model for securing document-based cloud storage against both known and previously unseen threats.

CHAPTER 2: LITERATURE REVIEW

2.0 Introduction

Due to all of the benefits that come with a cloud storage system, including accessibility, scalability, and economical advantages, it has become one of the most commonly used platforms for storing and managing large amounts of data. With its increased popularity comes an increased amount of susceptibility to threats associated with cloud storage. One of the largest threats associated with using cloud storage is the potential for uploading malware into your cloud-hosted data. The risk of malware being uploaded by users includes documents, executable files, image files, and archived/compressed folders, all of which can be leveraged by hackers to infiltrate the infrastructure of the cloud provider as well as users' desktops. An alternative solution to reduce the risk of introducing malware via cloud-based storage into a users desktop would be to use an AI-based Malware Analysis Tool, which can be integrated into every cloud hosted storage service. The remainder of the chapter will only review published papers associated with the threat of malware to multiple types of files associated with cloud storage security, as well as the use of traditional malware detection methods and the use of machine learning for the intelligent detection of malware for all types of files associated with malware.

2.1 Cloud Storage System Architecture

A cloud storage system is a distributed system of servers that store, control and avail user data on the internet. Though this architecture is highly available and scalable, it also provides a centralized point of attack by malware distribution. The cloud storage architecture will be also of significance to determine the possible security vulnerabilities and the malware scanning mechanisms in the cloud storage architecture.

2.1.1 Core Components of Cloud Storage Systems

Cloud storage infrastructures are developed with the reference to the official structure and security rules that are offered by the recognized standardization agencies, including the International Organization of Standardization (ISO), the National Institute of Standards and Technology (NIST), and the Cloud Security Alliance (CSA) These groups present a structured set of recommendations in order to make sure that cloud computing will be secure, dependable, and

able to safeguard the data of the users against numerous threats. (Savitha, 2023)

According to the previous studies, there are several key attributes of secure cloud storage system such as data reliability and retrievability. Nevertheless, confidentiality, integrity and availability (CIA) are the main security principles that are mainly discussed in the literature. These three features are generally perceived to be the key components of any safe cloud-based system (Savitha, 2023). Confidentiality is used to make certain that the data stored can only be accessed by authorized individuals, integrity is used to make sure that the data will not be modified or distorted in any way during storage and transmission and availability is used in order to make sure that the data and services are always available when needed.

The concepts of confidentiality and integrity are essential in avoiding unauthorized vistas as well as malicious modification of data stored. Lack of security The cloud storage systems are susceptible to data breach, tampering, and malware-based attacks without appropriate security mechanisms, which can destroy sensitive information and system trust (Savitha, 2023). In the effort of enhancing the security of the system, there are other services that are added to the cloud storage infrastructure to help improve security, access control policies, and malware detection mechanisms. Encryption will ensure data protection at rest and in transit, access control will allow file operations of authorized user only and malware scanning will allow to detect and prevent storage of malicious files. Collectively, these elements guarantee the general integrity, confidentiality and dependability of cloud storage facilities.

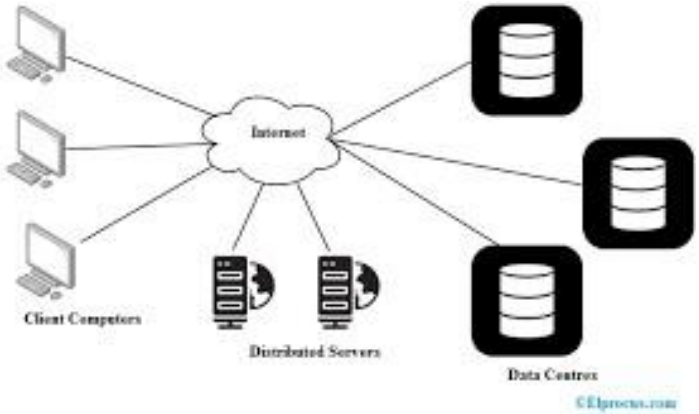


Figure 2.1 Basic component for Cloud Storage

Figure 2.1 demonstrates the minimal constituent to operate a cloud storage. It should have clients, Internet, server and Data bases where the data can be stored on the cloud.

2.1.2 Security Threats in Cloud Storage Environments

Storing a great number of sensitive documents and files leads to cloud storage systems becoming very tempting targets of malware attacks. As Shaikh F. (2021) indicates, an effective cloud storage solution should have good privacy and security assurance in order to secure confidential data that is stored on cloud environments. Nonetheless, irrespective of these security measures, cybercriminals are still using cloud-based services to spread and offer malicious files.

Software attacking cloud storage systems can be integrated into a broad assortment of data file formats, such as document, picture files, and executable files, compressed files, and multimedia files. These rogue files usually serve as vectors to concealed malicious code which occurs when the user downloads, opens, or engages with the compromised information. After activation, the payload can carry out unwelcome activities like installing new malware, backdoor creation, or remote access to the affected system. In contemporary malware attacks, the malware is now more advanced through the use of advanced evasion strategies like encoding, obfuscation, and benign behaviour mimicry to cover a malicious purpose in files (Kim et al., 2025). Harmful code is often disguised with the help of encoding and obfuscation that make it hard to detect the threat, which is identified correctly by the traditional signature-based detection systems. Also, certain malware programs are made to resemble the normal file behaviour which means that they can get by security checks and be in the computer without being detected over a long period.

Such malicious acts may have such dire consequences as stealing sensitive information, obtaining account access illegally, and taking advantage of threats to the cloud infrastructure. The reason is that one infect file in the cloud storage environment can endanger not only individual users but the whole system since the resources are shared, and services are interdependent. Consequently, the different kinds of malwares and their mode of attacks must be known to effectively come up with detection and prevention systems. This, consequently, makes the incorporation of advanced security solutions like AI-powered malware analysers into the cloud storage systems a requirement. Such solutions have the capacity of analysing file features and pattern of behaviour with the view of detecting concealed threats and, thus, promoting the overall privacy, security and credibility of cloud storage systems. (Kim et al., 2025)

2.2 Types of Malware

The malwares are of various types, each having a different malicious function to maliciously operate. Viruses bind themselves to the legal files and proliferate upon the execution of the infected file. The worms are automatically spread throughout networks without human intervention. Trojans will masquerade as authoritative software to trick the user into running the Trojan. Ransomware is an encrypted user data that requires money in order to unlock the data, and spyware is the third software that silently gathers user passwords and other personal data. These varieties of malware indicate how diverse the threat environment of the contemporary computing setting can be.

2.2.1 Document-Based Malware

Document based malware is the malware software that is presented in the form of files and Microsoft office documents like word, Excel, and PowerPoint. The reason why these types of files are often exploited is the fact that they are highly trusted and are commonly transferred both in personal and organizational settings. Consequently, such files have more chances to be opened without any suspicion by the users, and the malware delivery is more likely to be successful.

A document-based attack is when the threat actors use the documents as vectors of shipping and implementing malicious payloads. It is done with the help of taking advantage of the particularities of document formats or the vulnerabilities of a document reader application that allows running the harmful code by opening the file (Liu et al., 2025). The scripts, macros or exploit code embedded in malicious documents will automatically execute when the documents are opened and enable attackers to obtain unauthorized access, or install other malware or steal sensitive information.

The phishing campaigns are particularly vulnerable to document-based malware because it is widely used to deceive users. Attackers usually masquerade as reputable parties or known bodies like banks or government to convince the victims to open the malicious file. By doing so, victims can share their secrets, such as their bank account or credit and debit cards, passwords, and other personal identity details, without realizing it (Yahva et al., 2021). Social engineering and technical exploitation combined further complicate the situation with document-based malware as a constant and a major issue in the contemporary cybersecurity environments.

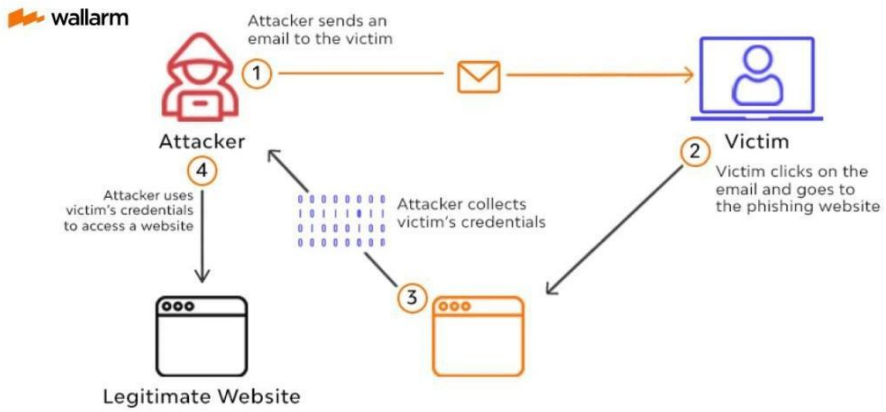


Figure 2.2 Phishing attack situation.

2.2.2 Executable and Script-Based Malware

Executable and script malware Executable and script malware can also be defined as malicious programs presented in these formats: executable (.exe) and script files (.bat, .sh), and scripts written in high-level programming languages such as Python, JavaScript. These files are not the documents based malware, but clearly crafted files, which are set to maliciously operate on the target system once executed. This can be unauthorized access to the system, data leakage, or introduction of other malicious codes. (Cosovan et al., 2021)

The attackers usually take advantage of the popularity and functionality of popular software to share executable and script-based viruses. To illustrate, from the moment the first vulnerability of Adobe Reader was detected in 2008, files have been actively misused as vectors of attack because of a high number of users and the ability to use embedded JavaScript to execute malicious code in the form of scripts within the documents (Cosovan et al., 2021). This has enabled the attackers to integrate executable behaviour with script-based methods to make their attacks more effective.

Executable and script based malware is a major security challenge in cloud storage set ups. The threat actors can also intentionally masquerade as malicious executables as harmless or valid files, e.g. software patches or utility software, to avoid detection systems. After these files have been downloaded and run by the end users, they are able to breach local systems and even connected cloud environments and this points out the necessity of sophisticated methods of detecting malware, especially those that use artificial intelligence. (Cosovan et al., 2021)

2.2.3 Archive and Multimedia-Based Malware

A common attack method is the use of polyglot files containing compressed archive files like ZIP, RAR, and other compressed files as a container to introduce malicious content into a single file system (Koch et al., 2025). These files are designed to be considered as a variety of valid file types, so that malware can be concealed in a way that is not detected by more traditional security services. Similarly, multimedia content can also be weaponized like images, videos and used to carry on the hidden malware via steganography or malformed file structures.

Covert and malicious communication through information hiding and steganographic channels has become a more and more common feature of real-world cyberattacks (Vielhauer et al., 2025). With the implementation of malicious code inside seemingly innocuous media files, an attacker is able to create a covert channel of communications or drop payloads without arousing any concerns. Such advanced hiding systems allow enemies to circumvent traditional detection tools, which are mainly based on file extensions or the analysis of the file type, which is a great challenge to the current security systems.

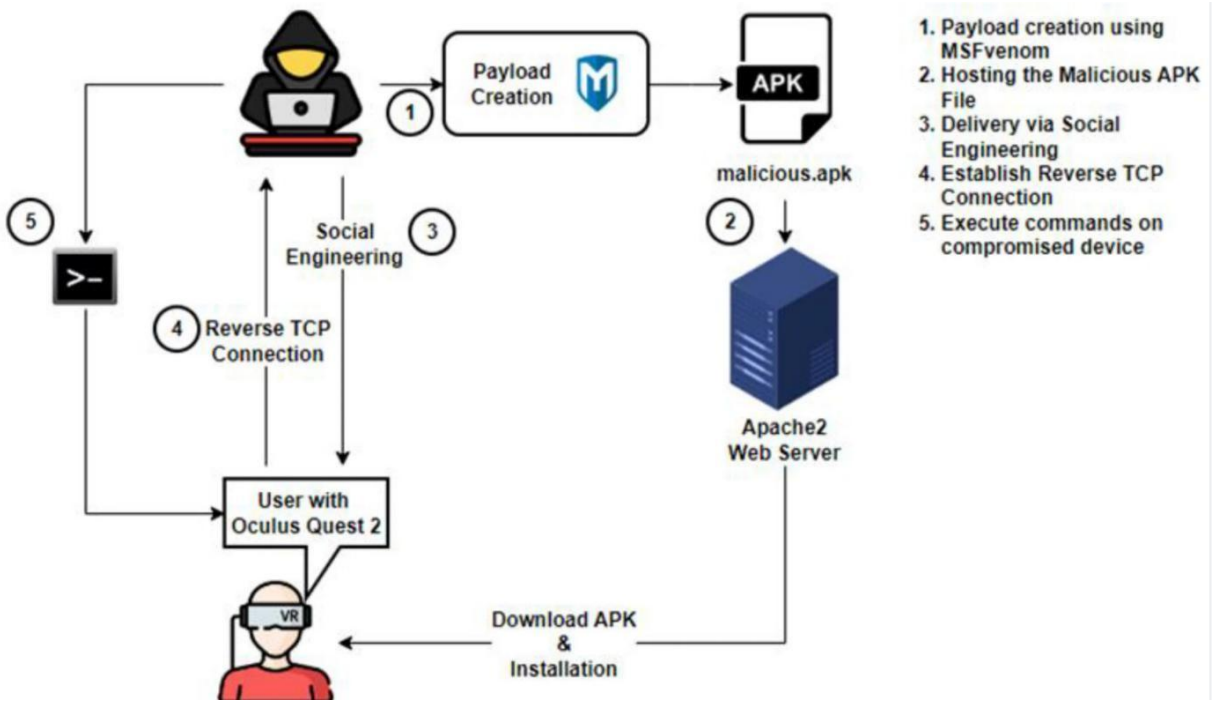


Figure 2.3 Payload Attack Situation.

2.3 Malware Detection Approaches

Malware detection: The cloud-based storage systems constitute a crucial security need because many and varied files are uploaded by the users. Cloud systems usually use dynamic analysis and static analysis as a complementary method of detection to protect as well as function effectively. Although dynamic analysis offers a better understanding of behavior, static analysis would be better since it is the most appropriate detection mechanism in cloud settings because it is fast, scalable, and safe. As such, this project uses the method of malware detection by utilizing the use of a static analysis as the primary choice.

2.3.1 Static Analysis for Malware Detection

The process of examining the code, metadata, and internal structure of a file but not executing the file is called the static analysis (Sinha and Sai, 2023). It is especially efficient in the cloud where it can be dangerous to execute untrusted files. In spite of the fact that the use of the concept of a static analysis can be carried out on its own, very frequently it is accompanied by the dynamic analysis to create a complete malware detection approach (Sinha & Sai, 2023).

Recent studies have gone further to develop the concept of static analysis beyond conventional signature-based detection into an approach where document formats like PDF files are viewed as binary executables. This allows a more in-depth structural and semantic analysis of files, which can identify malicious intent even where the code is highly obfuscated (Liu et al., 2025).

The process of modern static analysis usually involves the following stages of work. The first step involves creating an Intermediate Representation (IR) which in this case is the PDF Object Intermediate Representation (PDFObj IR). This IR can transform PDF objects into an assembly-style, low-level representation that can be analyzed in detail to examine internal code structures (Liu et al., 2025).

This is followed by the extraction of semantic features, based on pre-trained language models, of the IR to allow the system to obtain the underlying intent or purpose of embedded code as opposed to simply relying on the surface-level syntax (Liu et al., 2025).

Lastly, structural features are determined by building a Graph of Object References, which is a model that describes object relationships to each other in the file. This graph has the ability to detect anomalies that are hard to pick up by other tools (v.e. irregular object reference or covert

executable) which are powerful indicators of malicious activity (Liu et al., 2025). All these steps allow the passive files of malware to be converted into rich feature sets which can be used to classify malware using artificial intelligence.

2.3.2 Dynamic Analysis for Malware Detection

Dynamic analysis tracks a file's behaviour when it runs on a CPU to identify malware by looking for file-processing activity on the Operating System such as memory allocation, creating and deleting files and communicating with other computers via a network (Katkar et al., 2021). This technique is very effective for identifying obfuscated or polymorphic malware which can escape detection using static methods.

In dynamic analysis, it is necessary to run the malware in a safe and secure way; this is done by executing the malware in a separate virtualised environment, or "sandbox" (Sinha & Sai, 2023). Many sandbox technologies are now available to safely run malware to collect extensive behavioural data on the actions of the malware; for example, Docker enables the protection of the host system while allowing detailed analysis of its activities (Sinha & Sai, 2023).

2.3.3 Justification for Using Static Analysis

The use of dynamic analyses to identify malicious programmes relies on the observation of how a programme behaves when it is executed in a computer system. During an execution, dynamic analysis focuses on how a programme interacts with the computer system, such as by manipulating files stored on the computer, using the computer's memory or communicating with the network (Katkar et al., 2021). Dynamic analysis is particularly useful when identifying obfuscated or polymorphic versions of malware that would normally be ignored by a static analysis.

Dynamic analyses, conducted using sandbox software (Sinha & Sai, 2023), are executed in a secure, isolated virtual environment called a sandbox. Technologies such as Docker do not allow any form of malware to escape from the sandbox and inflict damage on the host computer while also providing detailed information about how the malware executes (Sinha & Sai, 2023). Based on the execution trace provided during a sandbox analysis, researchers can develop a "Malware Infection Tree" that graphically depicts how and when malicious actions were taken during the malware's execution (Ghosh K, 2015).

However, dynamic analysis is very resource-intensive compared to static analysis, meaning that it takes a lot of computer processing power to conduct dynamic analysis, making it less scalable than static analysis. Any malware that detects a sandbox will also often change its behaviour or stop execution to avoid being detected by static analysis techniques (Sinha & Sai, 2023). Therefore, dynamic analysis cannot be relied upon as the primary means to identify malware within a large volume of data in a cloud environment.

Table 2.1 Comparison analysis method

Criteria	Static Analysis	Dynamic Analysis	Signature Based-detection
Analysis Method	Examines files without executing them by extracting features such as opcode sequences, permissions, metadata, byte patterns, and structure	Executes malware in a sandbox or virtual environment and observes runtime behavior	Matches files against a database of known malware signatures
Security Risk	Safe	Risky	Safe
Scalability	High	Poor	High
Detection Speed	Fast	Slow	Very Fast
Suitability for AI Integration	Excellent for ideal for ML classification models	Limited	Poor
Cloud Storage Compatibility	Highly suitable because pre-upload scanning	Not suitable for real-time uploads	Suitable but weak security
Overall Effectiveness	High	Low	Low

Several key reasons exist for adopting Static analysis as the preferred method for this project, as shown in Table 2.1. The primary reason for utilizing Static analysis is its high level of safety. By not executing malicious files, Static analysis removes any chance of a system being compromised. Additionally, Static analysis can be run with much less computational power than traditional methods. As discussed throughout this thesis, Static analysis is well suited to leveraging modern machine learning technologies. By combining Static analysis with Modern Machine Learning technologies, unknown and zero-day threats are also identified. This combination is vital for protecting against malware in today's cloud storage.

Another benefit of Static analysis is that it can be applied to large numbers of files concurrently. As mentioned previously, having the ability to scan large amounts of files using this methodology will greatly improve the performance of a secure cloud storage solution. The AI malware analyzer developed for this project is based on this methodology.

2.4 Limitations of Traditional Malware Detection in Cloud Storage

Traditional malware detectors cannot keep pace with today’s malware, due to the amount of malware that exists and the complexity of the many new forms of malware being created every day (Ghosh et al., 2021), especially in an environment characterized by such a high volume of data such as Cloud Storage.

2.4.1 Signature-Based Detection Limitations

Signature-based Detection Methods, used in conventional Intrusion Detection Systems (IDS), have several inherent limitations that diminish their ability to provide an effective defensive measure against modern threats. The primary limitation is that signature-based methods are incapable of detecting new threats. This limitation is since signature-based methods rely on a signature database composed of previously detected malware signature entries . In addition, these types of systems also produce a high False Positive Rate (FPR) and generate many unnecessary alert notifications and often over-utilize the resources of the detection system . Therefore, traditional malware detection methods cannot be relied upon to provide a reliable identification of unknown malware (Patel & Ghosh, 2024).

2.4.2 Zero-Day and Polymorphic Malware Challenges

New Issues Related To Malware That Nobody Knows About: The term New Issues Related To Malware That Nobody Knows About refers to malware that's newly revealed and/or no one has previously encountered; consequently, there are no existing anti-malware products to deal with them as of yet (Deldar & Abadi, 2024). Because of both the newness of these threats and the complete lack of methodologies for mitigating these kinds of threats, it is extremely difficult for cyber security professionals to identify and defend against these types of malwares .

Malware That Changes Its Structure: This type of malware includes malware that will continually change its code while maintaining its function as malware. Malware embedded within web-based software (e.g., Javascript based) often utilizes metamorphic techniques that continuously alter their appearance (Cosovan et al., 2021). Therefore, the polymorphic characteristic of such malware would be difficult for signature-based detection systems to identify because a single piece of malware may be recognized at the level of tens of thousands based on its changes to its appearance until its finally identified. High rates of false positive and false negative responses typify the general inability of conventional security systems to deal with unknown and polymorphic malware variants (Patel & Ghosh, 2024).

2.4.3 False Positives and False Negatives

The large amount of both false positives (FPs) and false negative (FNs) produced through conventional malware detection methods is a significant aspect of the failure of these methods in protecting computer systems against malicious attacks (Patel & Ghosh, 2024). Specifically, the presence of a high number of FPs creates security fatigue where users regard some legitimate files as malicious and also fail to use up the valuable time and resources that they would use to investigate possible instances of malware in connection with the legitimate file. The high number of FNs associated with traditional malware detection methods means that computer systems continue to be exposed to threats that go undetected until they compromise the security of the computer system (Patel & Ghosh, 2024). For example, optimized and payload-aware IDS frameworks such as those developed with the Xavier-CMAE model are designed to minimize both the amount of time and the amount of effort put into investigations related to detecting FPs; as a

result, elite models provide exceptionally low FP Rates, with some reported FP Rates as low as 0.0182% Click or tap here to enter text. clearly demonstrating what modern systems based on AI are striving to achieve.

2.5 Machine Learning for Malware Detection

The use of Machine Learning (ML) to detect malware is quickly becoming the most popular research area associated with malware detection today (Kim et al., 2025). AI-based products are much more flexible and can generalize information for various types of systems than traditional (i.e. signature-based) approaches (Kim et al., 2025). As a result, ML-driven PDF malware classifiers have become popular among researchers because they provide very accurate results when it comes to identifying PDF malware (Liu & Matuszek, 2025). In addition, the use of Large Language Models (LLM) is changing both how threats are detected and how they are created on the fly (Ahi & Valizadeh, 2025).

2.5.1 Common Machine Learning Algorithms

Various models, both in statistical and machine learning, have been developed to assist with the detection of Malware. For the base detection of JavaScript Malware, a recommendation is to implement Hidden Markov Models based on Probability as well as Linear Classifiers. The AMD-XAI-ML framework provided an extensive test of nine different Machine Learning models using a benchmark dataset to highlight the multiple types of algorithms used for this purpose in the Android Malware Detection Field (Liu et al., 2025).

2.5.2 Feature Engineering for Multi-File Malware Detection

High-quality feature engineering is a key determinant of both the accuracy and dependability of any Machine-learning detection system. However, the persistent problem of feature instability remains an obstacle to producing effective detection models, regardless of the sophistication of modern machine-learning methodologies, as evidenced by the continuing reliance upon historical feature engineering strategies when utilizing 'feature extraction' techniques for the classification of different file formats (Liu et al., 2025), therefore, as a solution to this issue, contemporary methodologies are designed to produce highly discriminative features that can better represent an extensive variety of different types of documents.

While examining file types, for example, structural features utilize Object Reference

Graphs to evaluate the file's structure and integrity. On the other hand, semantic features utilize techniques such as PDFObjIR to characterize the relationship between a document's components, but emphasise on a more semantic level through extracting meaning from each component (Liu et al., 2025).

The payload-centric method of identifying malicious activity continues to rise in popularity within current-day Intrusion Detection Systems. This technique identifies distinct non-statistical characteristics associated with the malicious Payloads. The acceleration of this technique comes from the use of the innovative Tokenization techniques, Hex2Int, and the application of Pre-trained Large Language Model Tokenizers. These tokenisation methods allow Intrusion Detection Systems to conduct comprehensive and contextually informed analysis of these sophisticated payload-based attacks (Kim et al., 2025).

2.5.3 Recent Studies on AI-Based Malware Detection in Cloud Systems

Recent research has produced substantial advancements in creating very specialized and effective Artificial Intelligence (AI) models to improve security within both a networked environment and file-based environment. One such advancement is called Payload-Aware Intrusion Detection Systems (IDS), which implements a dual-path design of two different AI Models: Xavier-CMAE, which features real-time scalability with an accuracy percentage of 99.9718% and a False Positive Rate of 0.0182%; and the LLM-CMAE, which uses pre-trained Large Language Model tokenizers so that the alerts are contextually informed, while providing 99.9696% accuracy and an FPR of 0.0194% (Kim et al., 2025).

Another notable innovation is Small Sample Detection, which uses distance-based analytic techniques to allow for effective detection of malware present in files using only a small training dataset—specifically, only twenty (20) benign documents. Research suggests that this technique, has the potential of detecting upwards of 90.50% of malware samples and thus being a valid approach for organisations that may not have the benefit of sufficient training datasets (Liu & Matuszek, 2025).

The incorporation of Explainable AI (XAI) into Malware Detection Frameworks represents a significant advancement in the development of cyber security tools that are interpretable. For example, the AMD-XAI-ML framework will be one of the first systems created with XAI specifically for detecting Android malware (Patel & Ghosh, 2024). By utilising XAI methodology,

security analysts can better understand the reasons for the assigned classifications, which will help to establish greater confidence, accountability and responsiveness in mitigating threats.

2.6 Feature Extraction Techniques

According to Deldar and Abadi (2023), the process of feature extraction is essential and the most important first step of any complex deep learning based malware detection system. Early versions of AI detection systems (i.e. pre-Deep Learning), however, limited the number and types of features used, resulting in very basic feature representations (flow or only statistical information) and therefore, very constrained interpretation of more complex attacks that include payloads, such as (Kim et al., 2025).

2.6.1 Structural Features

The way a file is arranged gives clues to the user about the intent behind it, and that includes documents like PDF files. Structural Features provide detailed insight into the internal structure of the file using an Object Reference Graph that models how these files interact with other files and how they relate to each other; this is especially helpful for identifying malicious documents. Analyzing Structural Features is essential in detecting malicious files.

2.6.2 Content and Script-Based Features

The modern technique of analyzing content through contemporary methods is far superior to prior to 2020 as it provides the analyst with a more diverse and adjustable set of tools to perform their analysis on content. Current algorithms allow the extraction of a much greater range of methods from digital items than simply searching for specific keywords. Prior to 2020, the only technique available for conducting a content analysis was to search for key words.

The development of semantic features is considered a significant step forward as classifiers for semantic features use the semantic representation of a file's attributes so that the classifier can better evaluate the meaning of that file. In addition, classifiers use large language models that have already been trained to analyse the semantic properties of each file (R. Liu et al., 2023)

Another key development is the use of payload characteristics in intrusion detection systems (IDS). IDSs currently use advanced tokenization techniques to analyze malicious payloads more efficiently than was possible before. Some of the methods that IDSs are currently using include Hex2Int and tokenized LLMs. Therefore, IDS can identify low complexity file and

network threats at a more sophisticated level than ever before. This capability is made possible by incorporating method-based analysis with the traditional signature-based approach to intrusion detection (Kim et al, 2025).

2.6.3 Metadata and Statistical Features

Smutz and Stavrou (2021) found that various attributes within malicious files can reveal their intent using metadata attributes to identify the creator/designer of the file, time/date it was created, and other header-type information that can be extracted via different detection techniques. Yet, Smutz and Stavrou (2021) also showed that performing analysis on flow-level or statistical data alone will not provide as great of an analytical capability as when utilizing flow-level and statistical data in conjunction with other types of attacks (Kim et al., 2025).

2.7 Datasets for Cloud Malware Detection Research

The datasets utilized in developing, training, and evaluating AI-based malware-detection systems are key, meaning that the overall performance of the machine-learning model created is heavily dependent on the dataset used. Datasets used to train and test the AI-based malware detection system should also cover as many different file types, attack vectors, and evasion methods as is reasonably possible to ensure the trained AI-based malware detection system's capabilities align with those in real-world environments.

2.7.1 Challenges in Dataset Collection

Creating useful and representative malware datasets is one of the main focuses of document security research since how well a malware detection function operates depends heavily on the quality, diversity and realism of the datasets they’re trained and evaluated on (Nicholas, 2025). Developers of comprehensive datasets should strive to include a variety of malware behaviours, file formats and obfuscation methods so that Detection Models can generalise effectively to real threats. Incomplete datasets will negatively impact the detection rate, lead to increased false positivity rates and, consequently, reduce the level of robustness when faced with unseen malware variations.

There is a need for urgent evaluation concerning Rapid Advancements in and Increased Acceptance of Generative Artificial Intelligence (GenAI) because GenAI permits the automated production of high-scale and evasive types of malware. The capability of GenAI to generate

polymorphic malware, adversarial files and misleading payloads makes it impossible to rely on previous curation and validation of datasets; therefore, the process of ensuring the integrity, consistency and risk-conformity of datasets continues to be more and more difficult.

To solve these problems, researchers have created a Quality Assurance (QA) framework that will assist in developing high-quality multimodal assessment datasets that can identify the many varied risk factors introduced by the use of GenAI (Han et al., 2025). This framework will provide for systematic validation of risk annotation and cross-modal consistency checks for text-based, image-based, execution-based, and script-based layers. The use of a structured QA process will allow for improving the reliability of datasets, increasing the effectiveness of AI-based models for malware detection, and protecting the entire document security system from the new threats created by GenAI.

2.7.2 Balanced and Imbalanced Dataset Considerations

One of the main problems associated with the use of imbalanced datasets for Classification in Machine Learning deals with building classifiers capable of identifying Malicious and Benign files. In most cases, the number of Benign files that are available for use in the training data is significantly more than that of Malicious files within the dataset. As a result, Classifiers built using imbalanced training data sets typically have an inherent Bias toward the majority class (Benign) which causes them to have limited capability to accurately classify Malicious files (especially) and results in poor Generalization performance in real-world situations (Kumar et al., 2025). When trained on skewed training datasets, Models can present a false sense of High Overall Accuracy, even though the Model has Failed to capture/identify the presence of significant threats; this phenomenon (Accuracy Paradox) is well-documented within the literature. To address this issue, researchers have developed numerous techniques for balancing the class distribution in the dataset, prior to building a Machine Learning Model.

Oversampling, undersampling, and hybrid techniques such as SMOTE are data level techniques to balance class imbalances in datasets (Kumar et al. 2025). Ensemble methods combine sampling methods with algorithmic changes to address classifier imbalance issues. These techniques must be used appropriately to train classifiers on a representative sample of data to increase metrics such as precision, recall, and F1 score for minority classes. In file malware analysis, the availability of balanced datasets (i.e., the use of established datasets such as Contagio)

is important to fairly evaluate and validate detection models, which in turn allows the development of reliable and fair AI-based security systems (Kumar et al. 2025).

2.8 Encryption Algorithm

Encryption algorithms protect data in digital forms while it is being stored (Data at Rest), sent across a network (Data in Transit) and processed (Data in use). Encryption algorithms are the basis of all modern cryptography, which provides confidentiality, integrity and authentication of the information it protects. Encryption methods can be grouped into two categories; Symmetric Encryption and Asymmetric Encryption, and both types have specific uses because they operate differently and provide varying levels of security (Shilin et al., 2023; Mallouli et al., 2020).

2.8.1 Advance Encryption Standard (AES)

Widely recognized as a critical element in the contemporary field of information security, the Advanced Encryption Standard (AES) is an established symmetric encryption algorithm. AES applies a fixed block size of 128 bits and supports a broad range of customisable key lengths including 128, 192 and 256 bits. This gives AES a high degree of customisability and allows many different configurations to be derived from it. The AES' substitution-permutation design is robust against differential and linear cryptographic attacks; in fact, AES provides excellent security for many forms of sensitive data including storage and telecommunications applications (Shilin et al., 2023). Additionally, AES's efficient operations will allow it to be used in situations where both speed and security are important. Recent research on enhanced AES includes securing dynamic key generation and providing even higher levels of security while preserving encryption efficiency (Awasthi & Kohli, 2025).

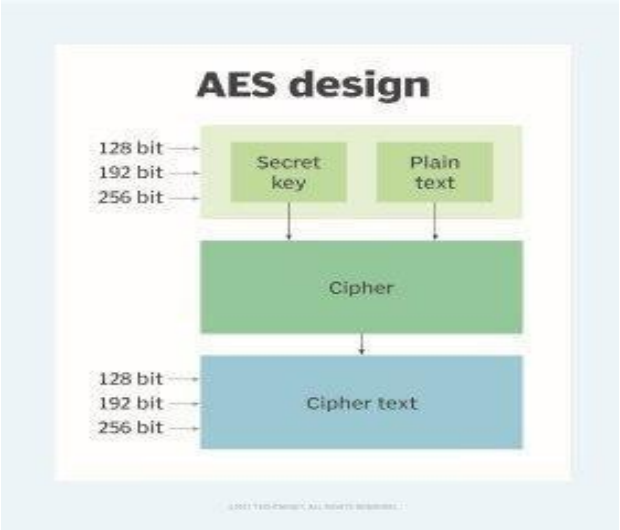


Figure 2.4 AES encryption architecture design

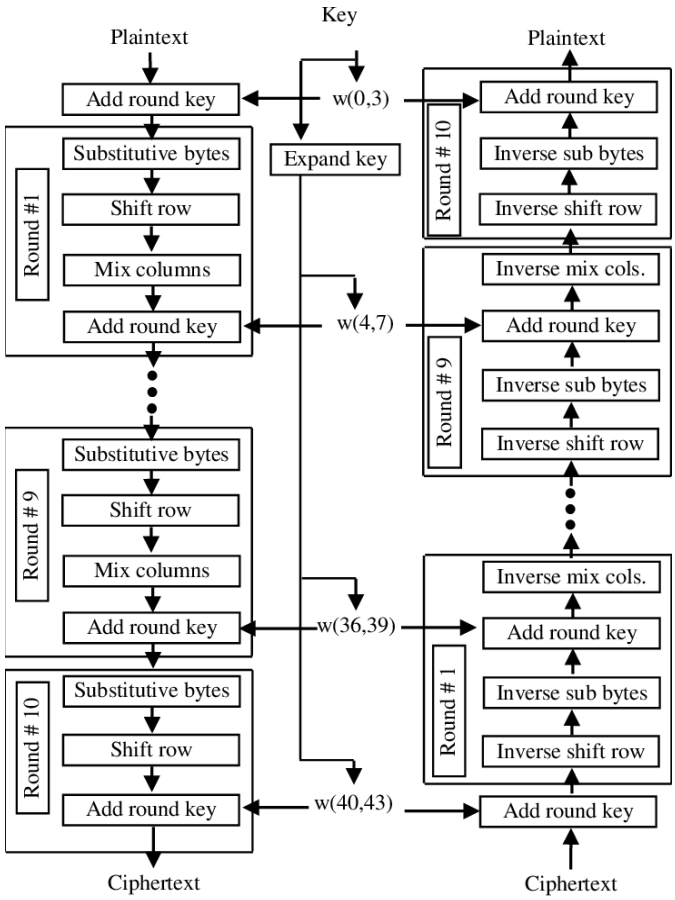


Figure 2.5 AES Encryption and decryption process diagram

Figure 2.4 and 2.5 Illustrate overview on how AES encryption works in storing data and files, including encryption and decryption process to maintain a secure encrypted data in system environment .

2.8.2 Data Encryption Standard

Although the Data Encryption Standard (DES) operated on 64-bit blocks and used a 56-bit key, its functionality has become obsolete due to the significant increase in computing power and therefore vulnerability to brute-force attack. However, DES provided the foundation for the creation of structured methods of securely sharing data and is an important stepping stone toward the development of new encryption technologies (Awasthi & Kohli, 2023). DES developed the building blocks for the creation of more advanced and secure encryption technologies, such as Triple DES (3DES) and AES, which provide longer key lengths and improved ability to withstand cryptanalysis.

2.8.3 RSA (Rivest-Shamic-Adleman)

The RSA algorithm is an asymmetric cryptography method developed by Rivest, Shamir, and Adleman, and is considered a fundamental method of asymmetric cryptography. RSA is based on the production of two secret keys: a public key used to encrypt data, and a private key used to decrypt the data. What makes RSA secure is that current computational techniques (as of February 2023) make it very difficult for attackers to reverse-engineer RSA encryption. RSA is considered the standard for secure data transmission and digital signature validation (Mallouli et al., 2020). Although RSA offers an extremely high level of security, it is computationally expensive, demanding a much larger number of processing resources to use RSA to encrypt data as key sizes grow larger and to then decrypt that data. This has resulted in the use of alternative more efficient algorithms (such as Elliptic Curve Cryptography (ECC))) as a replacement for RSA in some instances (Awasthi & Kohli, 2023; Mallouli et al., 2020).

2.8.4 Eliptic Curve Cryptography (ECC)

Asymmetric Encryption has advanced significantly through the introduction of Elliptic Curve Cryptography (ECC). In contrast to RSA, which has larger key sizes needed to provide the same level of security as ECC (example: 256-bit ECC provides the same level of security as a 3072-bit RSA key (Mallouli et al., 2020)), ECC's small key sizes offer a much more efficient option for users with limited resources, including mobile devices, Internet of Things (IoT) systems, and blockchain technologies. The mathematical basis for ECC is its algebraic structure in the Equation of an Elliptical Curve intersecting a Finite Field. This allows for ECC to provide secure key exchanges, digital signatures, and Encryption. Because of the computational efficiency and scalability of ECC, it is increasingly being adopted in modern Security Protocols (Shilin et al., 2023; Mallouli et al., 2020).

2.8.5 Comparison of Encryption Algorithms and Focus on AES

Encryption algorithms are essential to protecting data sent to and from cloud storage solutions, as well as in "rest mode", meaning it is not actively being accessed or printed. For example, symmetric encryption algorithms (e.g., AES and DES) provide fast encryption speeds and efficient operations. On the other hand, asymmetric encryption algorithms (e.g., RSA and ECC) provide secure methods of managing digital certificates and authenticating users to those certificates. The algorithm selected will depend on the specific security needs of the cloud storage system and the user.

Because the algorithm that has become most commonly used is AES, there is a high demand for the algorithm's ease of use and high level of security with a low amount of computational overhead. AES outperforms DES by providing significantly higher levels of security with greater computational efficiency through its ability to support key lengths of 128, 192, or 256 bits versus DES, which provides only a 56-bit key that is easily broken through brute-force attacks. RSA and ECC provide essential tools for creating secure key exchanges and digital signatures, but the computational overhead produced by these asymmetric algorithms makes them less suitable for encrypting large datasets that stored in cloud servers. For example, a minimum of 3072-bits of RSA key length would be needed to provide equivalent protection to a single 256-bit AES key (Mallouli et al., 2020).

Cloud storage applications are well-suited for AES due to their large amounts of data. AES encrypts/decrypts files with minimum latency, making it easy to integrate into cloud workflows. Furthermore, new implementations of AES, such as using hardware acceleration or parallel processing, have improved overall performance within cloud environments. This improvement allows AES to encrypt data while being transmitted over encrypted communication channels and protect it when stored on servers (Shilin et al., 2023; Awasthi & Kohli, 2023).

AES is also a good fit for secure multi-tenant environments because it can encrypt each user's data independently. This provides isolation and confidentiality for each user. This is particularly useful for public or hybrid cloud infrastructures, which are shared by many different users/organizations who store their data in the same environment. As cloud storage continues to grow and evolve, AES will continue to

play a critical role in solving current and future data security issues. Table 2.2 illustrates various cryptographic methods and how they compare and show how AES is an excellent fit for the project.

Table 2.2 Comparison of Cryptographic Method

	RSA	AES	ECC	DES
ALGORTIHM TYPE	ASYMMETRIC	SYMMETRIC	ASYMMETRIC	SYMMETRIC
KEY LENGTH	2048 TO 4096 BITS	128,192 OR 256 BITS	VARIES	56 BITS
SECURITY	HIGH	HIGH	MODERATE	MODERATE
SPEED	MODERATE TO SLOW	FAST	FAST	FAST
USE CASES	SECURE DATA TRANSMISSION, DIGITAL SIGNATURES	ENCRYPTING ELECTRONIC DATA	ENCRYPTION, SIGNATURES, KEY EXCHANGE	ENCRYTPTION ELECTRONIC DATA

2.9 Research Gap and Justification

Overall, while the literature indicates significant advancements in the use of AI for the identification of malware and the security of data stored in cloud environments, there is still an apparent disconnect between implementing these technologies into a fully-integrated end-to-end secure framework for cloud storage. While there are numerous studies focusing on specific components within the field such as payload-aware Intrusion Detection Systems (IDS), adversarial-robust (AR) file analysis, and cryptography (GDPR-compliant) for safeguarding sensitive information, very few have attempted to create a system that effectively combines the functions of AI-based malware scanning with automated Encryption & Data Storage.while some products have attempted to combine the two functions into one, the majority of current products

treat the functions of malware detection and data encryption as separate processes. As a result, there is often a delay in operational effectiveness from when malware is detected until when the encrypted file is saved and this creates multiple points for security failure.

Additionally, a large portion of current research into AI-based malware is limited to narrow and particularised fields such as Android apps (Patel & Ghosh 2024) and network threats but does not take into account the potential for developing cloud-native detection solutions for Document malware types such as PDF files. Though PDF files are widely

accepted and referenced in literature as an attack vector (Liu et al. 2025) there are few well developed and comprehensive frameworks that would integrate the Static Analysis component of an AI powered model directly into a cloud storage file up-load channel prior to transferring the file to an end-user. Lastly, although complex models can produce high correct decision rates when tested in controlled laboratory conditions it is difficult to find sufficient published material outlining how such models can effectively be packaged and provided as simple and low-cost API services that are compatible with popular web programming languages such as PHP and NodeJS, creating a significant disconnect between current theoretical work on AI-based malware protection and how the tools can be practically deployed to produce results on a scalable basis.

The direct response of this project to these gaps represents the justification for this study's inception. This Secure Cloud Storage System with AI-based Malware Analysis that utilizes AES Encryption will create a singular work flow toward attaining holistic security for both organizations and individual users. This Secure Cloud Storage System assures that the security provided by AI Malware Analysis is a prerequisite for both encryption and storage thus reinforcing Confidentiality, Integrity, and Availability (CIA) security principles (Carvalho et al., 2017). This project has a primary focus on Static File Analysis that addresses a common vector for attack through cloud-style environments on a scalable basis (Sinha & Sai, 2023). Building on the concepts of the Python Flask API for the delivery of AI Malware Analysis and the web-based user interface to perform Secure Cloud Storage, this study will provide a

working and capable architecture that fills the gap between past academic research in cybersecurity and actual implementation of cybersecurity technologies. This study will provide an automated, proactive defense against future threats of Zero-Day Malware and polymorphic malware by fulfilling a critical demand in the security model for Cloud-style Storage.

2.9 Summary of Related Works

Table 2.3 Related Works

Title	Key Features	Strengths	Limitations	Proposed Solution
Secure File Storage on Cloud Using Hybrid Cryptography (Sharma et al., 2020)	AES, ElGamal encryption, SHA-256, and 2FA	Strong encryption with 2FA authentication	Lacks malware detection capabilities	Adds malware analyzer to prevent malicious file uploads
Improved Cloud Storage Security Using Three Layers Cryptography Algorithms (Kumar et al., 2020)	AES, ECC, and RSA-based multi-layer encryption	Robust key management and encryption	High computational cost, no malware detection	Simplifies encryption using AES and integrates malware analyzer
Securing Cloud Storage Data Using Hybrid AES-ECC Cryptographic Approach (Chhabra & Gupta, 2021)	Combination of AES and ECC	Optimized for resource-constrained environments	Lacks malware detection mechanisms	Incorporates malware detection before encryption

Title	Key Features	Strengths	Limitations	Proposed Solution
Development of a Secure Web-Based Cloud Storage System Using 3D-AES Block Cipher Cryptography Algorithm (Ali & Hassan, 2019)	Enhanced AES block cipher	Advanced encryption techniques	Complex implementation, no malware detection	Balances security and efficiency with AES and malware analyzer
Enhanced AES Implementation (Feistel & S-Box Extensions 2022)	AES enhanced with Feistel structure and SBox algorithm to increase encryption robustness.	Provides high security for data transmission and storage.	Slightly increases encryption and decryption time.	Focus on algorithmic optimizations to reduce overhead while maintaining encryption strength.

2.10 Summary

In summary, the literature reviewed in this article shows tremendous progress in the security of cloud storage. There have been advances in the areas of encryption, data integrity and artificial intelligence-based malware detection. Past work has focused largely on the use of AES, RSA, ECC, or Hybrid Cryptographic algorithms for providing security to data within the cloud. Encryption alone is an effective means to protect data both at rest and while it is being transferred. However, the vast majority of existing solutions concentrate on encryption and access control, often ignoring malware detection at the time when an end-user uploads a file to a Cloud Service Provider (CSP). Therefore, encrypted cloud storage solutions run the risk of containing malicious files as well posing risks to both end-users and the cloud platform. AI-based malware detection technology has received much attention through studies

conducted on various methods for detecting sophisticated threats like zero-day and polymorphic malware with the use of machine learning models and static analysis techniques. These studies show that these technologies can achieve very high levels of accuracy when detecting malware, while at the same time producing very low numbers of false positives,

however most of the time when conducting these studies, it is done in controlled lab environments or testing environments where these technologies have been implemented and used in real-world cloud storage infrastructure. Also, many AI-based malware detection systems focus on specific application domains such as Android applications or network traffic, as opposed to supporting multinomial file types like documents, executables, archives and multimedia files that are processed through cloud storage solutions.

There is a lack of integrated secure cloud storage systems that provide both an AI-based malware analysis process and an efficient encryption mechanism. Malware detection and encryption are treated as separate processes by most current solutions which creates vulnerabilities and reduced operational efficiencies for cloud environments. To address the identified limitation, the proposed system uses static-analysis based AI malware analysis as a prerequisite to secure uploading of files into the cloud environment, followed by AES encryption to securely store the data in the cloud. This integration allows for increased confidentiality, integrity, and availability of the stored data while allowing for continued scalability and suitability for large volume(s) of cloud solutions. The final outcome of this project will be to develop an entirely integrated approach that bridges the gap between theoretical and practical implementations and ultimately provides a cohesive and deployable solution to improve the security of cloud storage from modern and continuously changing types of malware threats.

CHAPTER 3 : METHODOLOGY

3.1 Introduction

This section explains how the research was conducted and which approach was taken to complete this project. The approach used by this researcher is the methodology that was used as a structure for the completion of the project on time. Furthermore, this chapter provided a detailed step by step process for the Secure Cloud Storage with AI Malware Analysis and AES Encryption; how this was planned, designed, developed, and tested as part of the project's overall functionality and success.

3.2 Project Framework

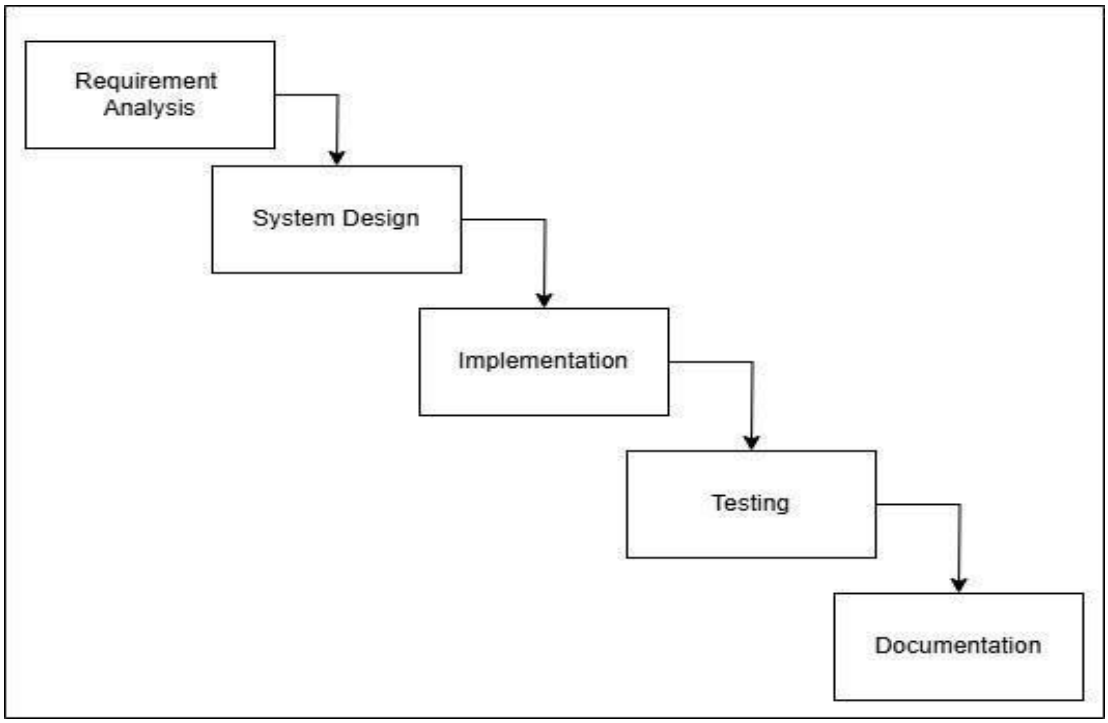


Figure 3.1 Waterfall Model

Figure 3.1 illustrated the Waterfall model used to develop the "Secure Cloud Storage with AI Malware Analyzer and AES Encryption" project, aiming to achieve the objectives outlined in Chapter 1. This research methodology consists of six major phases, each with its own

processes: Requirement Gathering and Analysis, Design, Implementation, Testing and Documentation

Table 3.1 Waterfall model table

STAGES	**TASK**	**DELIVERABLES**
Requirement & Analysis	Preliminary research and literature review on the following topic: 1) Performed initial research and review existing literature on Cloud Computing, Malware Analysis, Data Breaches, Encryption Algorithms. 2) Outlined the background of the study, including the problem statement, objectives, scope, and significance of the research. 3) Compared various mechanisms and techniques, identifying their strengths and weaknesses in relation to the proposed project	1) A comprehensive review of literature on Cloud Computing, Data Breaches, Artificial Intelligence, Encryption Algorithms, and Malware 2) Clear definition of the background, problem statement, objectives, scope, and significance of the study 3) A comparative analysis of different mechanisms and techniques, highlighting their strengths and weaknesses in the context of the Encryption Algorithm and Malware analysis.

STAGES	TASK	DELIVERABLES
System Design	Designed the system architecture, including the Artificial Intelligence, encryption, algorithm, flowchart, use case diagram, entity-relationship diagram, and user interface mock- up.	Detailed designs of the algorithm, flowchart, use case diagram, entity-relationship diagram, and user interface mock-up, providing a blueprint for the implementation phase. 1. Implement artificial intelligence malware analysis in the project for file analyzing 2. Implement the AES encryption and decryption functions. 3. Develop the web application using appropriate technologies such as PHP, HTML, CSS, Phyton
Implementation	• Implemented AI malware analysis phyton flask in the project for file analyzing • Implemented the AES encryption and decryption functions. • Developed the web application using appropriate technologies	A functional web-based secure cloud storage system with AI Malware Analyzer and AES encryption to protect data.

STAGES	TASK	DELIVERABLES
Testing	• Performed functional testing to ensure all components work as expected. • Generated test cases to validate the system's functionality, performance, and security.	Test results verifying that the system meets the specified requirements and functions correctly.
Documentation	Gathered and arranged all the project related information.	1) Final report 2) System of a proposed project

3.2.1 Requirement Analysis

In the first phase of the project, an initial study was conducted and various sources of literature were reviewed for the subject areas of Cloud Computing, Data Breaches, Encryption Algorithms, and Malware analysis. This is an important first step of the study to create a solid foundation and provide theoretical basis for the design and implementation of the project. This phase includes outlining the background of the project, including a problem statement, which outlines the gaps in currently available secure File Storage Systems and supports the identification of system requirements for secure File Storage in a cloud environment; objectives, which establish what the system is intended to achieve; and lastly, the scope of the project defines the boundaries of the project.

The literature review is conducted to compare techniques and technologies. For instance, various encryption algorithms are compared and contrasted based on several criteria, including security, efficiency, and performance. Other methods of Malware detection and analyzer are also analyzed to determine whether the methods can be used for the proposed project; therefore, the comparison will enable the analysis of strengths and weaknesses and allow the design of the proposed system to use tested solutions and improve the existing solutions.

3.2.2 Software

Software requirements are those requirements that outline the exact tools, programs, and systems required for a system to work correctly. As part of the system's requirements, the requirements will include the development tool sets needed (operating system, programming languages), as well as any required software (Development Frameworks) to provide support for implementing/executing features of the system (application software). For example, Phyton, PHP, MySQL, and a Web Server are all tools and systems that need to be in place to successfully implement/deploy this project.

Table 3.2 Software Requirements

Software Component	Operating System	Processor	Memory	Description
HTML	-	-	-	Hypertext Markup Language is used to design the structure and content of online pages.
CSS	-	-	-	Cascading Style Sheets employed for styling and formatting HTML elements, enhancing the visual presentation of web pages.
JavaScript	-	-	-	Programming language used to create dynamic and interactive effects within web browsers.
Phyton	-	-	-	Main programming language for implementing the AI-based malware analyzer. It enables the development of machine learning models for static malware detection.
MySQL	Windows, Linux,	1.6 GHz dual-core	4 GB RAM	Relational database management system (RDBMS) used for storing and

Software Component	Operating System	Processor	Memory	Description
	macOS	processor or equivalent		managing project data.
PHP	Windows, Linux, macOS	2.0 GHz dual-core processor or equivalent	2 GB RAM	PHP serves as the core programming language for the backend, managing file encryption (AES), malware scanning integration using Phyton flask for AI malware analyzing and server-side logic. It ensures seamless processing of user requests and secure file handling.
Virus Total` API	-	-	-	An API service used to support analyze uploaded files for potential malware. Provides integration for secure file scanning before storing in the cloud.
Google Cloud Platform (GCP)	Windows, Linux, macOS	2.0 GHz dual-core processor or equivalent	2 GB RAM	GCP is cloud-based platform designed to simplify the deployment of web applications, APIs, and static websites. It is particularly popular for hosting applications due to its developer-friendly approach, seamless integration with Git, and support serverless architecture and pay-per-use feature.

Visual Studio Code	Windows, Linux, macOS	1.6 GHz or faster processor	1 GB RAM	Microsoft released a free source-code editor for Windows, Linux, and macOS. Supported features include debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and Git integration.

3.2.3 Hardware

The main hardware requirements for a given system are the hardware resources or devices that must be available, including servers, storage systems, Ethernet (networking equipment), and all user-available I/O peripherals needed to run the system. The selection of the hardware required for running any system is based on the performance needs of the system, the scalability of the system as it grows over time, and how well the selected hardware works with the software being used by the system.

Table 3.3 Hardware Requirements

Hardware Component	Requirement Specifications
Laptop	11th Gen Intel ® Core ™
Chipset	I5-1135g7
Ram	16.0 GB
Operating System	Windows 11

3.2.4 Project Timeline

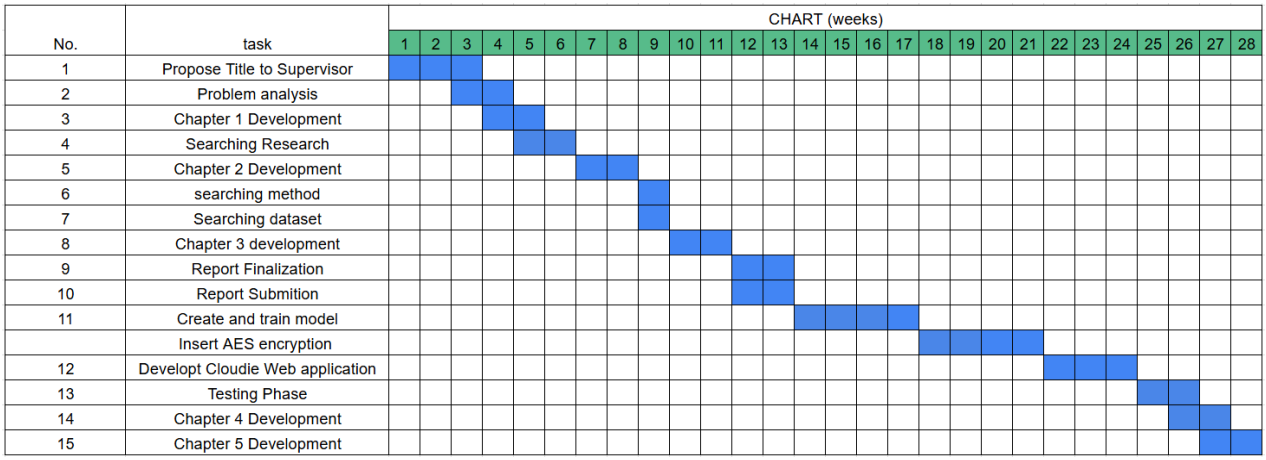


Figure 3.2 Project Gannt Chart

Table 3.4 Shows the timeline for this project for the whole development

3.3 System Design

In the "Secure Cloud Storage with AI Malware Analyzer and AES Encryption" project, the system design phase focused on a comprehensive architecture for integrating AI malware analysis combined with encryption technology to create a more secure environment for data. During this phase, the logical and physical elements of the system were defined and the structure and processes necessary to provide secure and easy-to-use cloud storage were established. The design phase is a process to design the system, and within this process, use case, flow chart, and interface designs were compiled to show how the proposed system would function.

3.3.1 Random Forest Machine Learning Model

In this work, a Random Forest is chosen as the primary machine learning tool for malware detection because it is effective, very accurate in classifying, and very effective at modelling high dimensional space or complicated sets of features. The Random Forest is built using the approach of an ensemble (the combination of many trees) which allows it to build multiple decision trees and produces its prediction from the majority vote of those trees, thus reducing overfitting and improving the accuracy of the Random Forest model. In this case, the Random Forest is trained on the statically derived feature set of different file types in a cloud storage system, using the metadata of the files, the structure of the files, and their statistical analysis. The Random Forest has the ability to model non-linear relationships and to withstand the effects of noise and redundant features, so it is very well-suited to static malware investigation. Additionally, the Random Forest gives feature importance scores to the trained model which can help to understand the main indicators for determining whether a file is malicious or not, thereby improving both malware detection and transparency of the system.

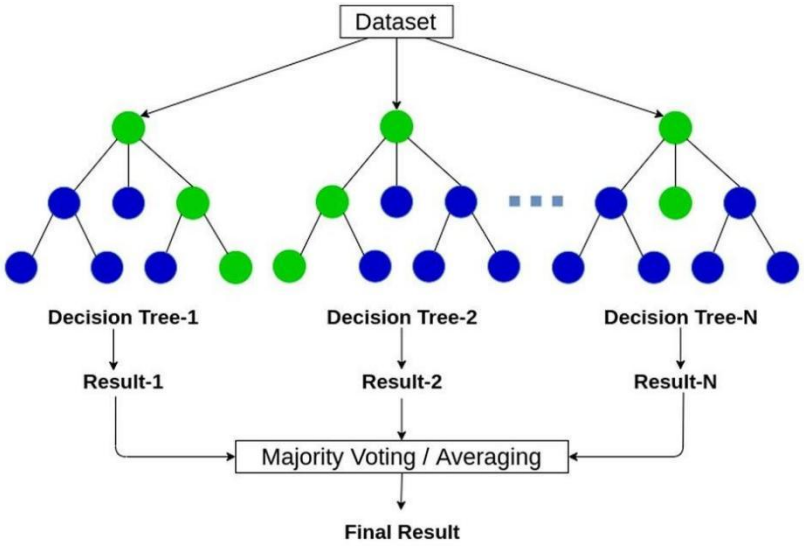


Figure 3.3 Random Forest Model

3.3.2 Dataset Used

The researchers have used a publicly available malware PDF dataset with 18,126 files indicating whether they are benign or malicious. This particular dataset had been selected for its broad range of structural and behavioral characteristics able to be exploited in malware PDF attacks. Each file from this collection was pre-analysed to extract relevant distinguishing traits for identifying malware and has the class label to identify if a file is benign or malicious, which will allow for supervised machine learning classification.

The dataset consisted of 32 feature extractions (excluding the filename) detailing the internal structure, content behavior, and metadata properties of PDF files. They are aimed to identify an individual's patterns of benign usage and the malicious behaviors of many PDF files, including embedded scripts, self-execution actions, and more suspicious object structure. The large dataset and the numerous features make this dataset ideal for developing and testing AI-based malware detection models.

The chosen dataset fits the goals and objectives of the proposed AI-based malware analysis tool for Cloud Storage systems. Because of how often a Cloud Storage Platform manages PDF files uploaded by users, it is imperative that the identification of malicious PDF files provides confidence in the integrity and privacy of user's data. Since the dataset is focused on specification of low-level PDF structures and embedded code execution behaviors, training models on this dataset can enable the

identification of both "known" and "unknown" variants of malware.

Also, features including JavaScript execution, Embedded File Types and Automated Actions also are beneficial in identifying advanced and obfuscated malware exploitation techniques. This makes the dataset well suited to benchmark the performance of machine learning classification methods in identifying malicious files located in Cloud Storage environments.

3.3.3 Physical Design

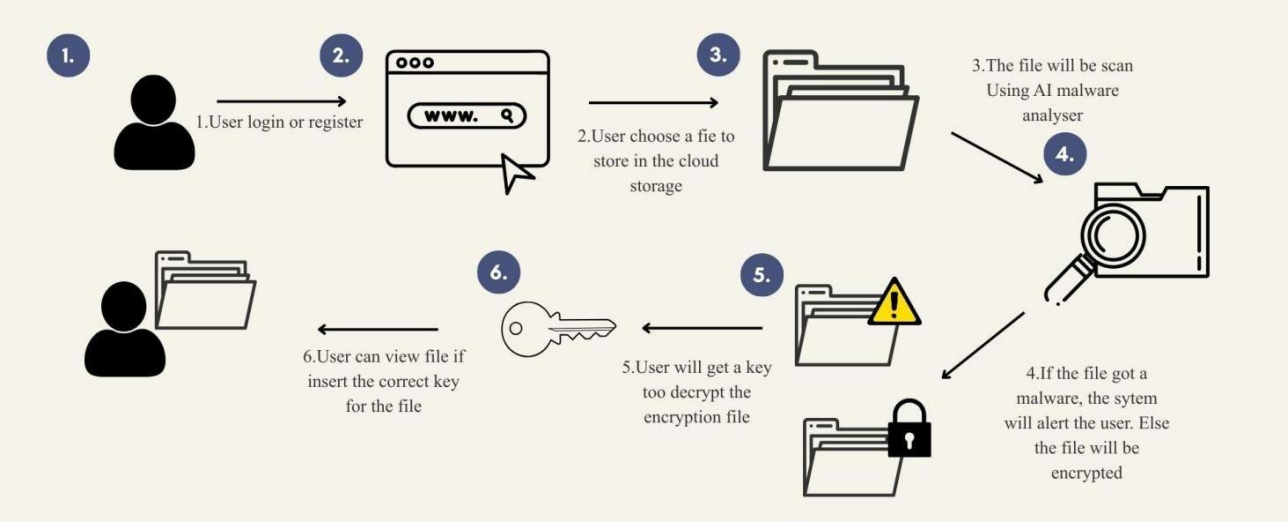


Figure 3.4 Physical Design of System

"Figure 3.4 depicts a physical diagram of the Cloudie system. The process starts when the user logs in or registers. After a successful login, the user selects a file to store. The system submits the file to a verdict engine that runs two checks in parallel: a VirusTotal signature-based scan and an AI-based structural analysis using the trained Random Forest model. If VirusTotal flags the file as malicious, it is rejected regardless of the AI result. If VirusTotal reports the file as clean but the AI model flags it as suspicious, the file is still encrypted and stored, but tagged with a 'suspicious' status for the user's awareness. If both signals agree the file is clean, it is encrypted using AES-256-CBC and stored normally. The encrypted file itself is stored in a cloud storage bucket, while file metadata, the verdict, and the encryption key are recorded in a separate managed SQL database. The user is then provided with the AES key as both plain text and a QR code for future decryption."

3.3.4 Use Case Diagram

\

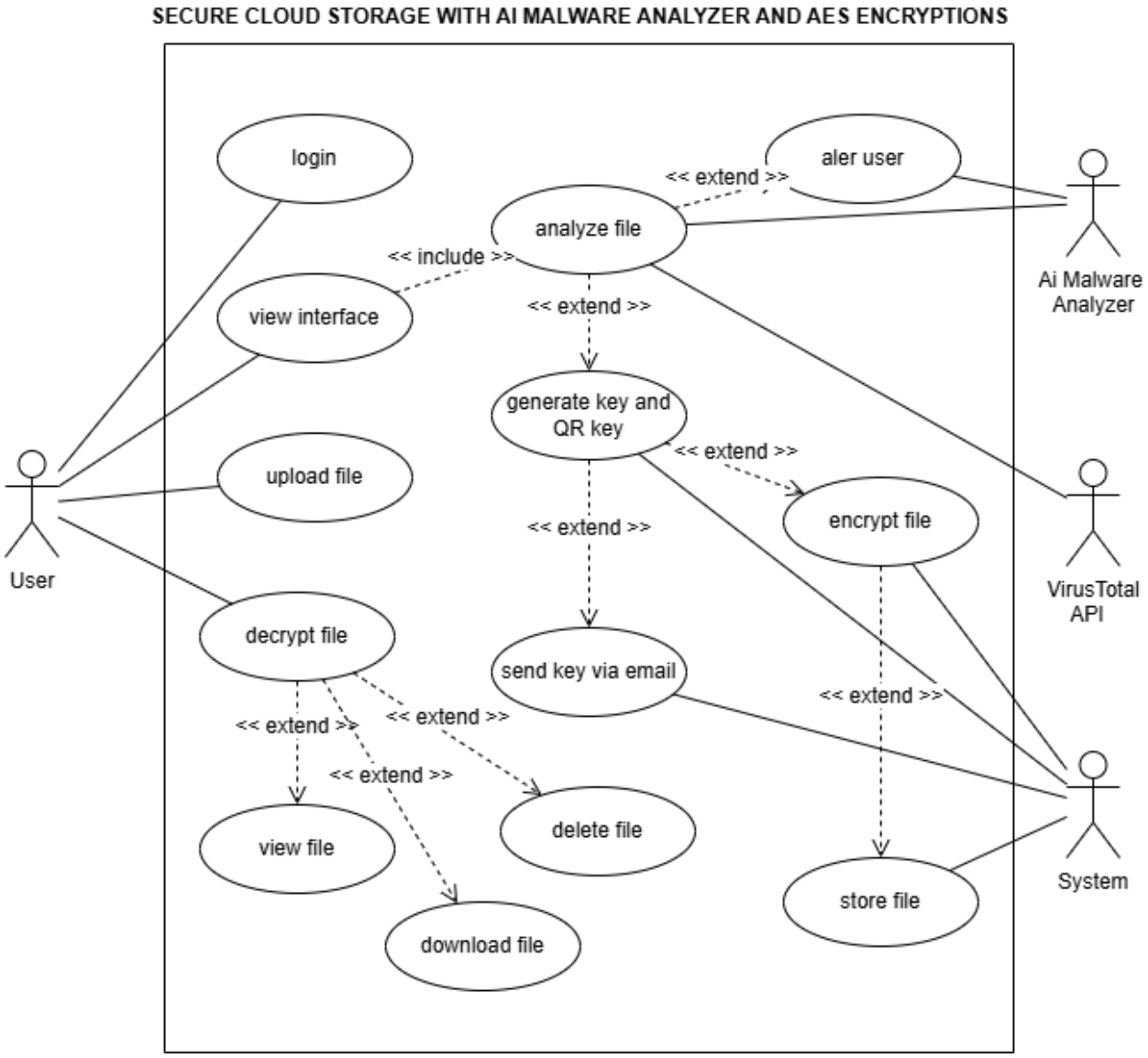


Figure 3.5 Use case diagram

Figure 3.2 provides illustrations of the interactions between the user, the system, and the AI malware analyzer. It highlighted key functionalities such as file upload, malware analysis, encryption, secure storage, and file management, ensuring a streamlined and secure process for handling sensitive data in the cloud. The details of proposed use case will be explained at table 3.5 below.

Table 3.3 Use Case Description Table

Use Case Diagram	Login
Actors	User
Description	The user provides login credentials to gain access to their account in the system. **Basic Path:** 1. User enters valid credentials and is authenticated by the system. **Alternate Path:** Invalid credentials trigger an error and retry prompt.

Use Case Diagram	View Interface
Actors	User
Description	After logging in, the user accesses the main interface, which displays options for upload file and view dashboard **Basic Path:** 1. The interface loads successfully. 2. User choose the action on the interface **Alternate Path:** System fails to load the interface and shows an error message.

Use Case Diagram	Upload File
Actors	User
Description	The user uploads a file to the system, **Basic Path:** 1. The user chooses the file to be upload. 2. The fie is successfully uploaded. **Alternate Path:** Upload fails due to file format issues, size, or network errors.

Use Case Diagram	Analyze File
Actors	AI Malware Analyzer, VirusTotal API
Description	The system submits the uploaded file to VirusTotal for signature-based scanning and to the AI model for structural analysis. The verdict engine combines both results into a clean, suspicious, or malicious classification. **Basic Path:** 1. File is scanned by AI model and Virus total in parallel 2. Results will be noticed to the user **Alternate Path:** The analyzer encounters errors and notifies the user to retry.

Use Case Diagram	Alert User
Actors	AI Malware Analyzer, User
Description	The system alerts the user about the malware analysis results, recommending further actions if threats are found. **Basic Path:** 1. The user receives an alert regarding threats. **Alternate Path:** Alerts fail, and results are displayed manually.

Use Case Diagram	Generate Key and QR Key
Actors	System
Description	The system generates a unique AES key to encrypt the uploaded file, and QR from AES Key generated to ease the user to save the key while ensuring its security. **Basic Path:** 1. Key and QR Key is generated successfully. **Alternate Path:** Key generation fails, and the user is prompted to retry.

Use Case Diagram	Encrypt File
Actors	System, User
Description	The uploaded file is encrypted using the generated AES key, ensuring its confidentiality. **Basic Path:** 1. User input the AES key generated 2. File is successfully encrypted. **Alternate Path:** Encryption fails due to resource or compatibility issues, prompting the user to retry.

Use Case Diagram	Store File
Actors	System
Description	The encrypted file is securely stored in the system's cloud storage for future access. **Basic Path:** 1. File is stored securely after encrypted. **Alternate Path:** Storage fails, and the user is notified of insufficient space or system errors.

Use Case Diagram	Send Key
Actors	System
Description	The system securely sends the encryption key to the user or user can save the key during encryption for later use during decryption. **Basic Path:** 1. Key is securely sent or stored by the user **Alternate Path:** Key transfer fails, and the user is notified of the error.

Use Case Diagram	Decrypt File
Actors	User, System
Description	The user requests the decryption of a file, and the system retrieves the key to decrypt it. **Basic Path:** 1. User input the AES key saved from the system 2. File is decrypted successfully. **Alternate Path:** Decryption fails due to mismatched keys or system errors, prompting an error message.

Use Case Diagram	Delete File
Actors	User, System
Description	The user deletes a file from the cloud storage, permanently removing it from the system. **Basic Path:** 1. User decrypt the file using matched AES key 2. User performs action to delete the file 3. The file is deleted successfully. **Alternate Path:** Deletion fails due to system issues, and the user is notified to retry.

Use Case Diagram	View File
Actors	User, System
Description	The user views the contents of a decrypted file through the system interface. **Basic Path:** 1. User decrypt the file using matched AES key 2. User performs action to view the file 3. File content is displayed successfully. **Alternate Path:** File fails to load, and an error message is displayed.

Use Case Diagram	Download File
Actors	User, System
Description	The user downloads a decrypted file from the system to their local device. **Basic Path:** 1. User decrypt the file using matched AES key 2. User performs action to download the file 3. File is downloaded successfully **Alternate Path:** Download fails due to network issues, prompting the user to retry.

Use Case Diagram	Send Key via Email
Actors	System
Description	Upon successful upload and encryption, the system sends the AES key to the user's registered email address **Basic Path:** 1. File is successfully encrypted. 2. System sends the key by email (non-blocking). **Alternate Path:** Email delivery fails silently; the upload process is not interrupted, and the user can still retrieve the key from the on-screen display or QR code.

Table 3.5 summarized the key use cases for the Secure Cloud System Using AI

Malware Analyzer and AES Encryption, outlined the interactions between the user, system, and AI malware analyzer. These use cases include logging in, uploading files, analyzing files for malware, alerting the user, encrypting files using AES, securely storing and retrieving files, and managing file access and deletion.

3.3.5 Flowchart

The system flowchart is shown during the design process to illustrate the system's flows and is represented by symbols. It shows the sequence of steps needed to complete the process to secure a file in the Secure Cloud System using AI malware analyzer, VirusTotal and AES encryption.

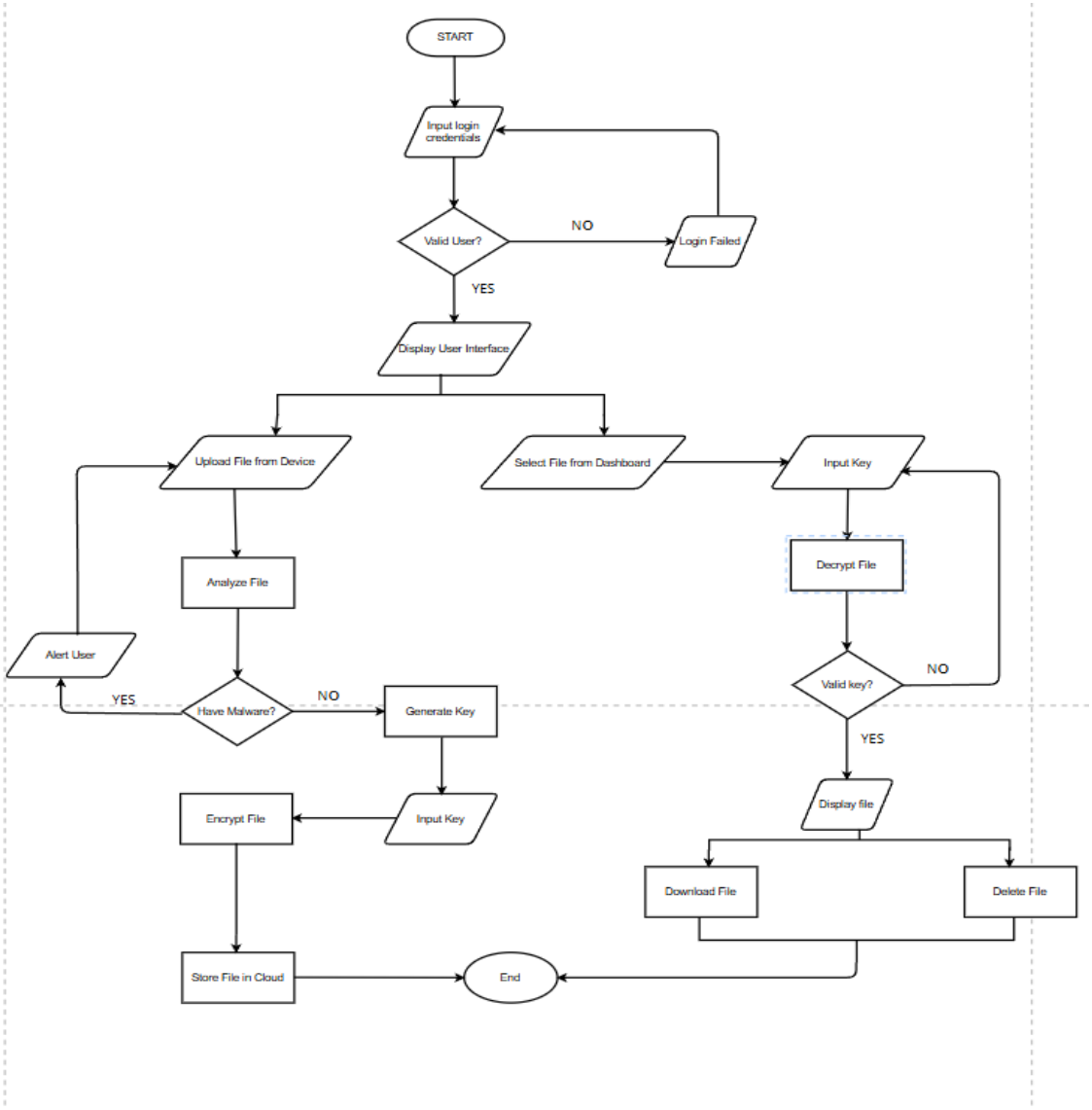


Figure 3.6 Flowchart of the system

Figure 3.3 provides a detailed process of a Secure Cloud System Using AI Malware Analyzer and AES Encryption, showing the whole flow process of a proposed system.

Below is a comprehensive explanation based on the flowchart:

1. **Start and Login Process**

Input Login Credentials: The process begins with the user entering their credentials (username and password).

Validation: The system checks whether the credentials are valid.

1) If the credentials are invalid, the system displays a "Login Failed" message, and the process terminates.
2) If valid, the system allows the user to proceed by displaying the user interface.

2. **User Interface Options**

From the user interface, the user has two primary options:

1) Upload File from Device
2) Select File from Dashboard

3. **File Upload and Malware Analysis**

If the user uploads a file, the system submits it in parallel to VirusTotal for signature-based scanning and to the AI model for structural analysis. The verdict engine combines both results into one of three outcomes:

1) Malicious (flagged by VirusTotal, regardless of the AI result) — the system alerts the user and rejects the file.

2) Suspicious (clean by VirusTotal but flagged by the AI model) — the file proceeds to encryption but is tagged for the user's awareness.

3) Clean (both signals agree) — the file proceeds to encryption normally.

4. **File Encryption and Storage**

If no malware is detected:

1) The system generate the user to input a key.
2) The file is encrypted using the provided key.
3) The encrypted file is stored in the cloud storage.

5. **File Decryption and Management**

If the user selects a file from the dashboard:

1) The system prompts the user to input the decryption key.
2) Decision Point - Key Validation:
    a) If the decryption key is invalid, the decryption fails, and the user is prompted to re-enter the key.
    b) If the key is valid, the system successfully decrypts the file and displays its content.

6. **File Actions**

After decryption, the user has two options:

1) Download File: The user can save the decrypted file locally.
2) Delete File: The user can delete the file from the cloud storage.

7. **End**

The process concludes after the file is either downloaded or deleted or the new file stored in cloud storage

The attached flowchart depicts a complete and highly secured process for storing files in the cloud. With the assistance of artificial intelligence (AI) to detect malware

threats, files are protected before they are stored, and files are encrypted using Advanced Encryption Standard (AES) when they are stored. User authentication, malware alerts (email or text notifications), encryption and decryption of files, and file management all work together to create an ideal and secure experience for the user.

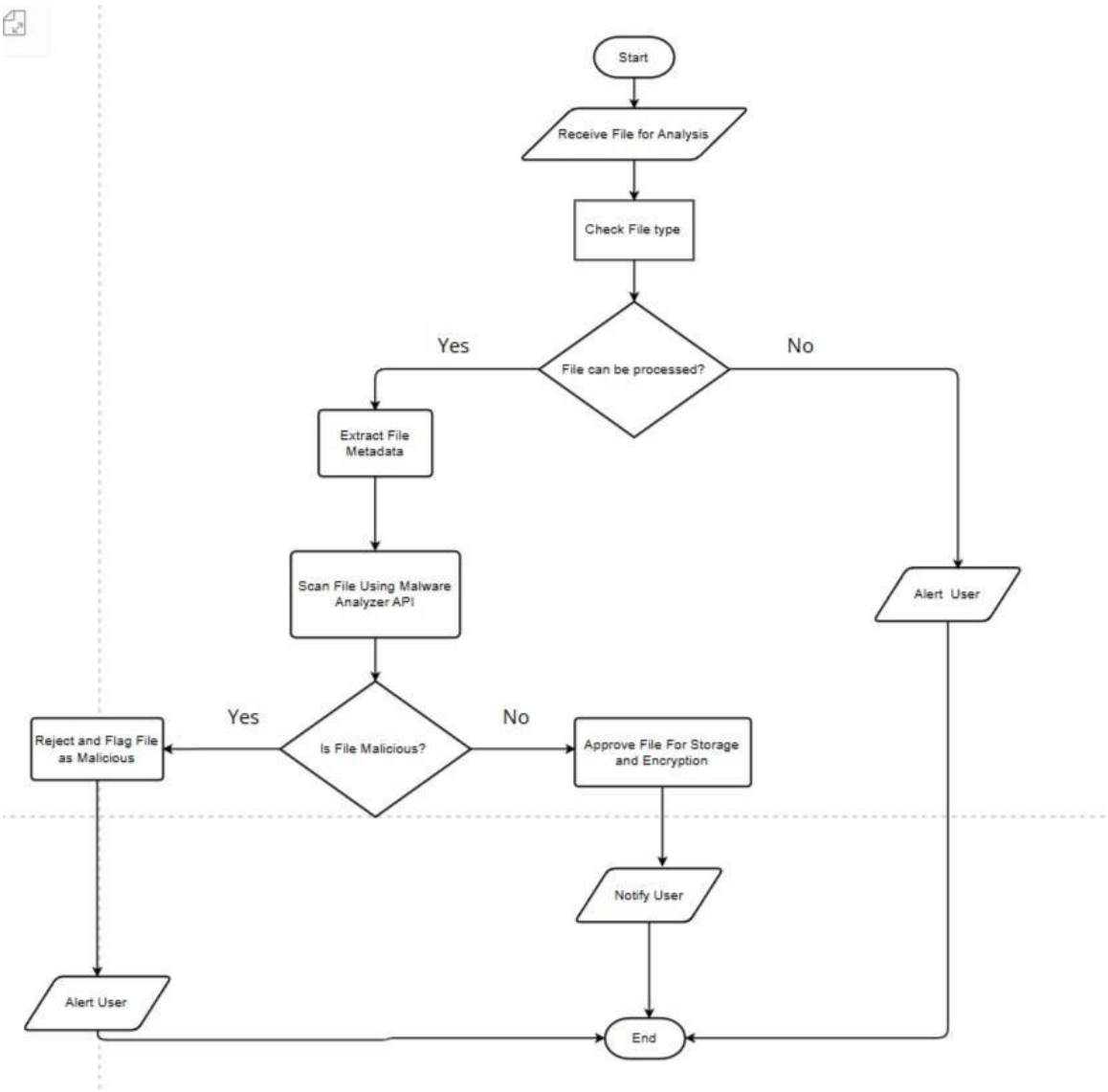


Figure 3.7 Flowchart of the AI Malware Analyzer.

As shown in figure 3.7, a complete outline or flow chart of how the AI Malware analyzer will perform its functions and analysis when analyzing incoming files which will ultimately be stored on the computer after completing analysis of that file. Below is a detailed outline explaining step by step each part of the flow chart provided above.

1. **Start**

   The process start when a user uploads a file.

2. **Receive File for Analysis**

   The system get the file and prepares it for further analysis.

3. **Check File Type**

   The file type is checked to determine if it is supported by the system.

   a) If the file cannot be processed: The system alerts the user and ends the process.

   b) If the file is supported: The process moves to the next step.

4. **Extract File Metadata**

   Essential file details such as size, type, and format are extracted to aid in the analysis.

5. **Scan File Using AI Malware Analyzer**

   The file is scanned for potential malware threats using AI malware analyzer.

   Is File Malicious?

   The results of the scan are analyzed:

   c) If the file is malicious: The system rejects the file, flags it as a threat, and alerts the user.

   d) If the file is safe: The file is approved for storage and encryption.

6. **Approve File for Storage and Encryption**

   Safe files will be proceed to another process which encryption and storing file in the cloud.

7. **Notify User**

   The user is notified about the status of their file (whether it was safely

stored or rejected due to malicious content).

8. End

The process concludes after the necessary actions have been taken

3.3.6 Entity-Relationship Diagram

Figure 3.8 Entity-Relationship Diagram

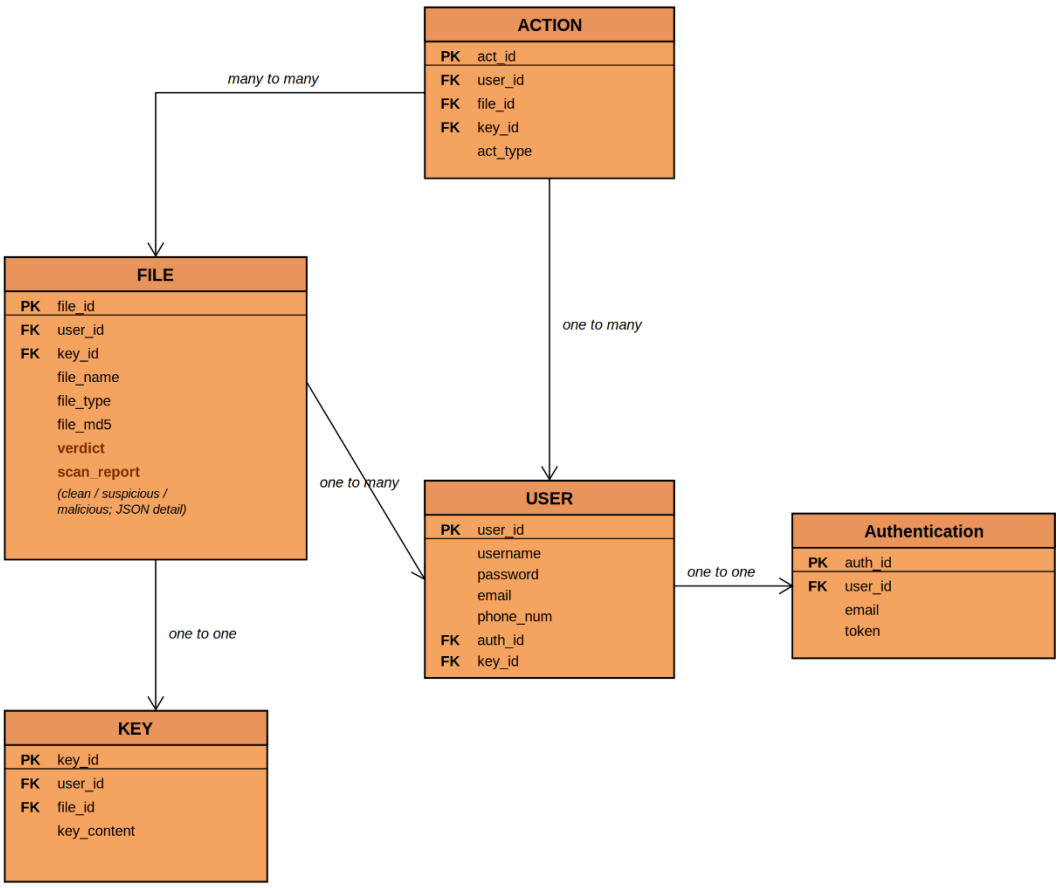


Figure 3.8 Entity-Relation Diagram

Figure 3.5 shows ERD for the project Secure Cloud System using AI Malware Analyzer and AES encryption. This diagram provides a clear view for each entity and attributes for the system.

3.3.6.1 USER Entity

Table 3.4 User Entity

Attribute	Description	Key	Data Type
user_id	Unique identifier for the user	Primary Key	Integer
username	Username of the user		Varchar/String
password	Password of the user		Varchar/String
email	Email address of the user		Varchar/String
phone_num	Phone number of the user		Varchar/String

Table 3.6 shows User Entity to represent the users

of the system. Relationships:

1) One-to-One with AUTHENTICATION (for managing secure user login and tokens).
2) One-to-Many with ACTION (a user can perform multiple actions).
3) One-to-Many with FILE (a user can upload multiple files).

3.3.6.2 AUTHENTICATION Entity

Table 3.5 Authentication Entity

Attribute	Description	Key	Data Type
auth_id	Unique identifier for authentication	Primary Key	Integer
user_id	References user_id in USER	Foreign Key	Integer
email	Email used for authentication		Varchar
token	Authentication token		String

Table 3.7 shows Authentication Entity to stores user authentication details.

Relationship:

4) One-to-One with USER (linked to user login).

3.3.6.3 FILE Entity

Table 3.6 File Entity

Attribute	Description	Key	Data Type
file_id	Unique identifier for the file	Primary Key	Integer
user_id	References user_id in USER	Foreign Key	Integer
key_id	References key_id in KEY	Foreign Key	Integer
file_name	Name of the file		Varchar/String
file_type	Type of the file (e.g., .txt)		Varchar/String
file_md5	MD5 hash for file integrity		Varchar/String
verdict	Combined VirusTotal + AI classification: clean, suspicious, or malicious		Varchar (16)
Scan_report	Stores the detailed VirusTotal and AI scan output for later review		JSON

Table 3.8 shows File Entity to represents files stored in the cloud Relationships:

2) One-to-Many with USER (multiple files can belong to one user).
3) One-to-One with KEY (each file has a unique encryption key).
4) Many-to-Many with ACTION (files are associated with various actions).

3.3.6.4 KEY Entity

Table 3.7 Key Entity

Attribute	Description	Key	Data Type
key_id	Unique identifier for the key	Primary Key	Integer
user_id	References user_id in USER	Foreign Key	Integer
file_id	References file_id in FILE	Foreign Key	Integer
key_content	Content of the encryption key		Varchar/String

Table 3.9 represents Key Entity to manages encryption keys for secure file storage. Relationship:

5) One-to-One with FILE (each file is assigned one unique key).

3.3.6.5 ACTION Entity

Table 3.8 Action Entity

Attribute	Description	Key	Data Type
act_id	Unique identifier for the action	Primary Key	Integer
user_id	References user_id in USER	Foreign Key	Integer
file_id	References file_id in FILE	Foreign Key	Integer
key_id	References key_id in KEY	Foreign Key	Integer
act_type	Type of action (e.g., upload)		Varchar/String

Table 3.10 represents Action Entity to tracks actions performed in the system, such as uploading, downloading, or scanning files.

Relationships:

6) Many-to-Many with FILE.
7) One-to-Many with USER.

This ERD ensures secure cloud storage by integrating AI malware analysis and AES encryption, with clear relationships between users, files, actions, keys, and authentication details

3.3.7 User Interface

The user interface mock-up is presented during the design process to make a design prototype before proceeding to develop the front end.

3.3.7.1 Log In interface

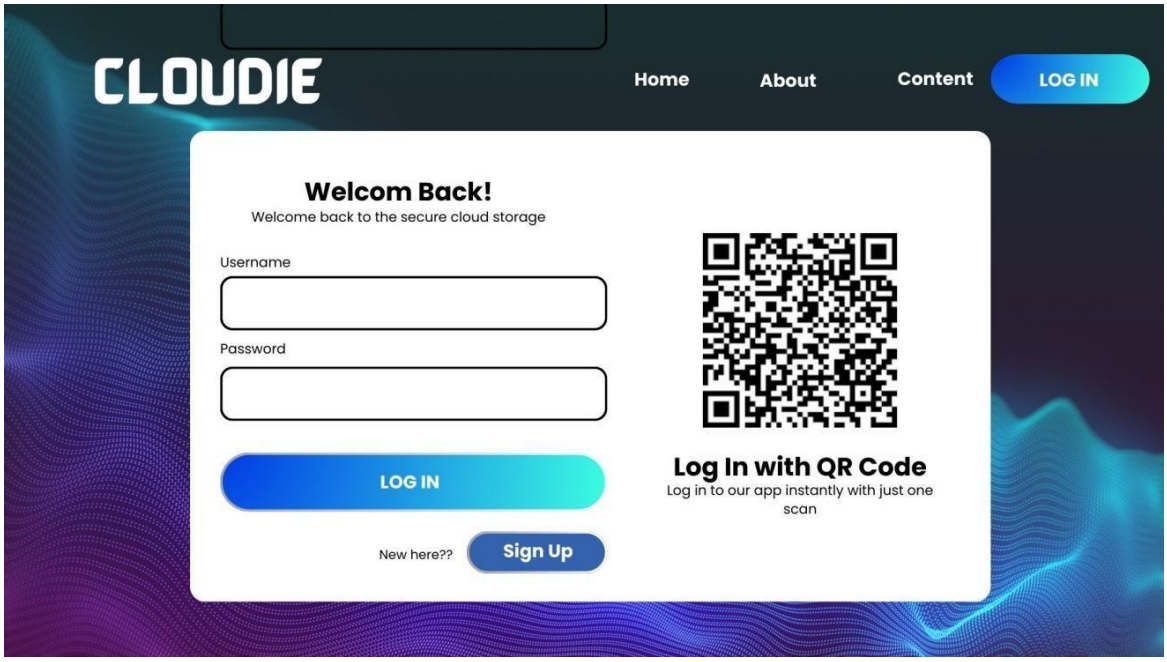


Figure 3.9 Log in interface

Figure 3.6 shows the login interface in the cloud system for the user

3.3.7.2 Cloud Dashboard

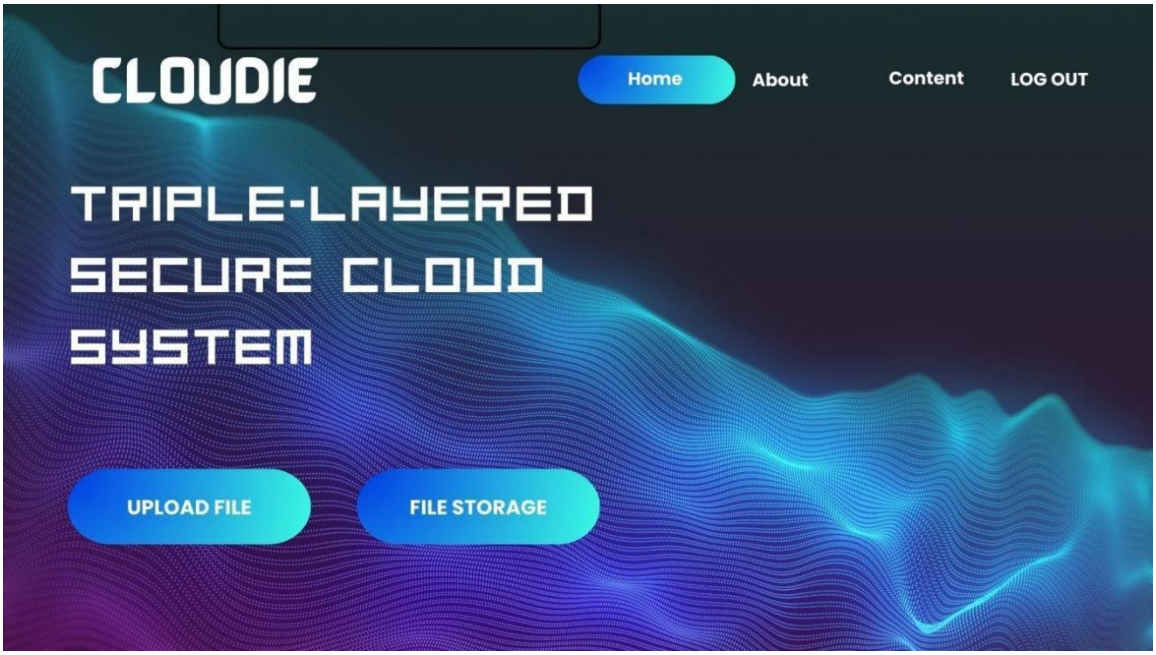


Figure 3.10 Log in interface

Figure 3.7 shows the homepage after the user successful log in.

3.3.7.3Upload File Interface

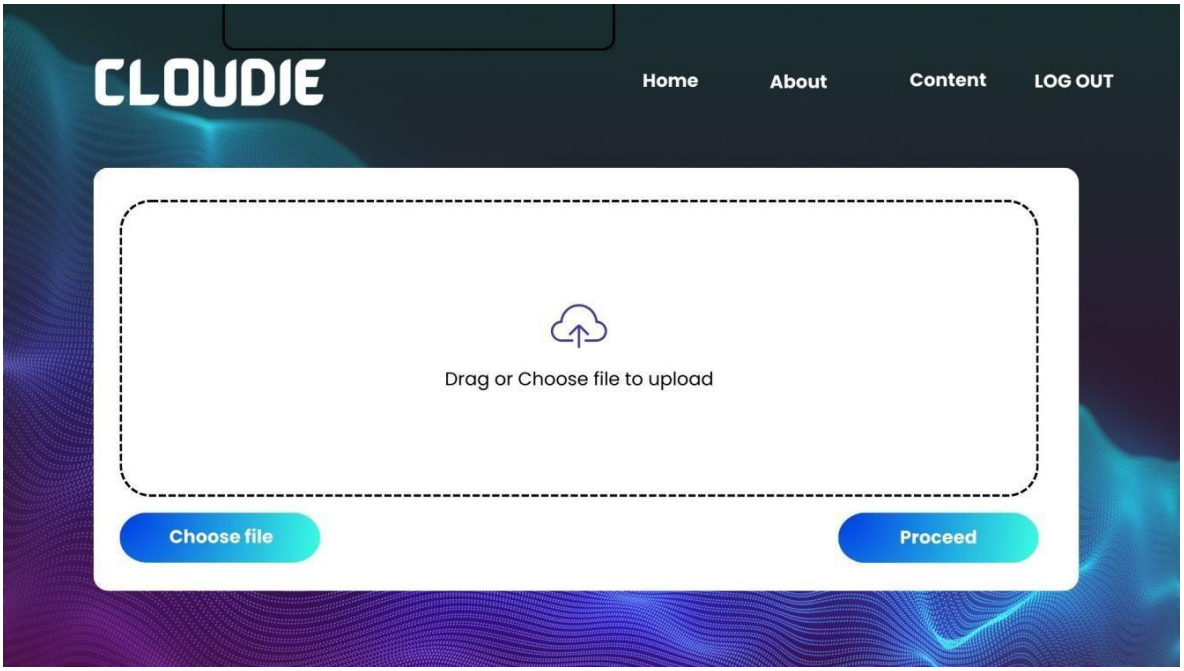


Figure 3.11 Log in interface

Figure 3.8 shows the Upload Interface if the user choose to upload a file.

3.3.7.4 Analyzing Interface

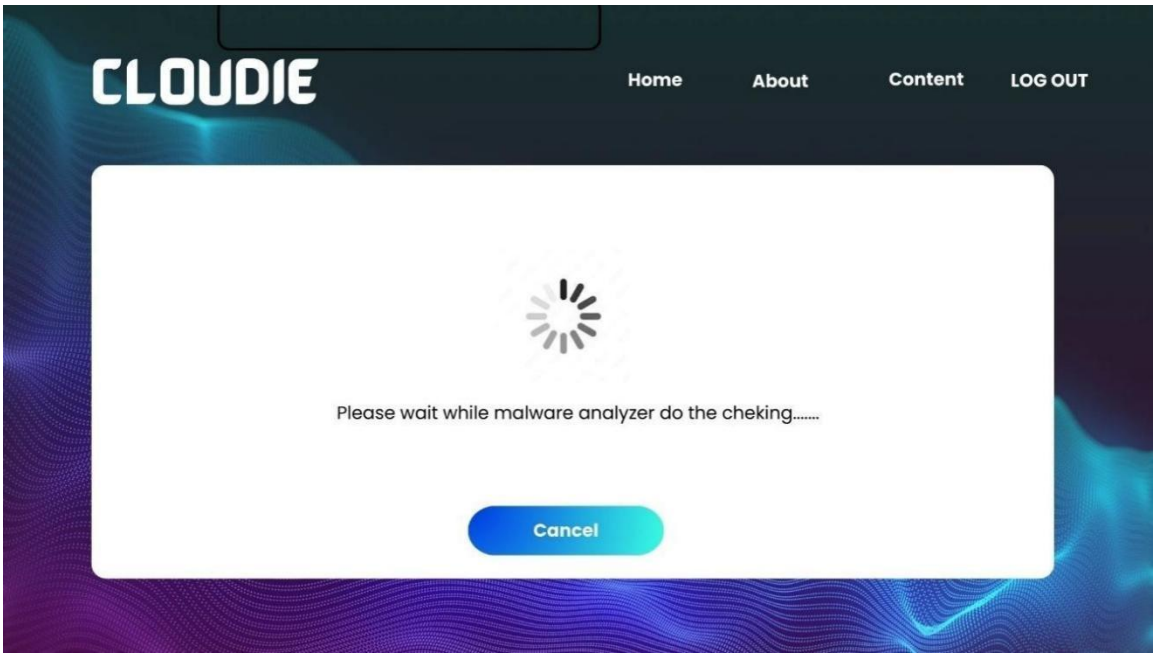


Figure 3.12 Log in interface

Figure 3.9 shows the AI malware analysing the file before reporting the findings to the user.

3.3.7.5 Successful Analysing

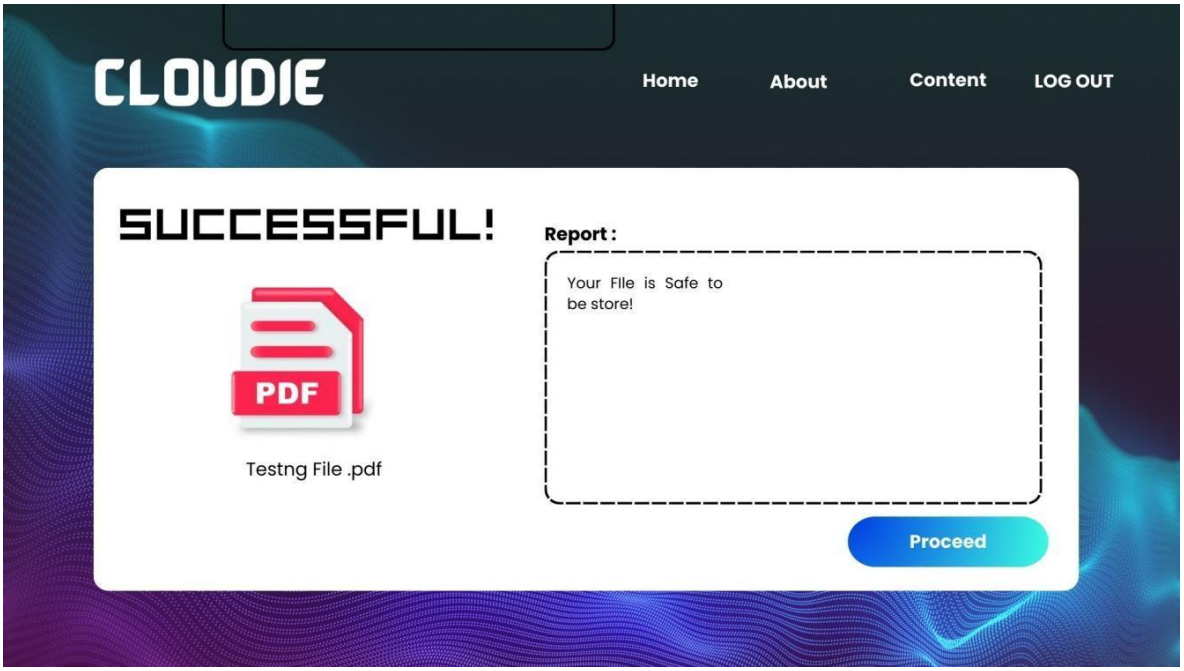


Figure 3.13 Log in interface

Figure 3.10 shows the successful if the file is clean from malware and generating the report for the user to know the details for the malware detection.

3.3.7.6 Encryption Interface

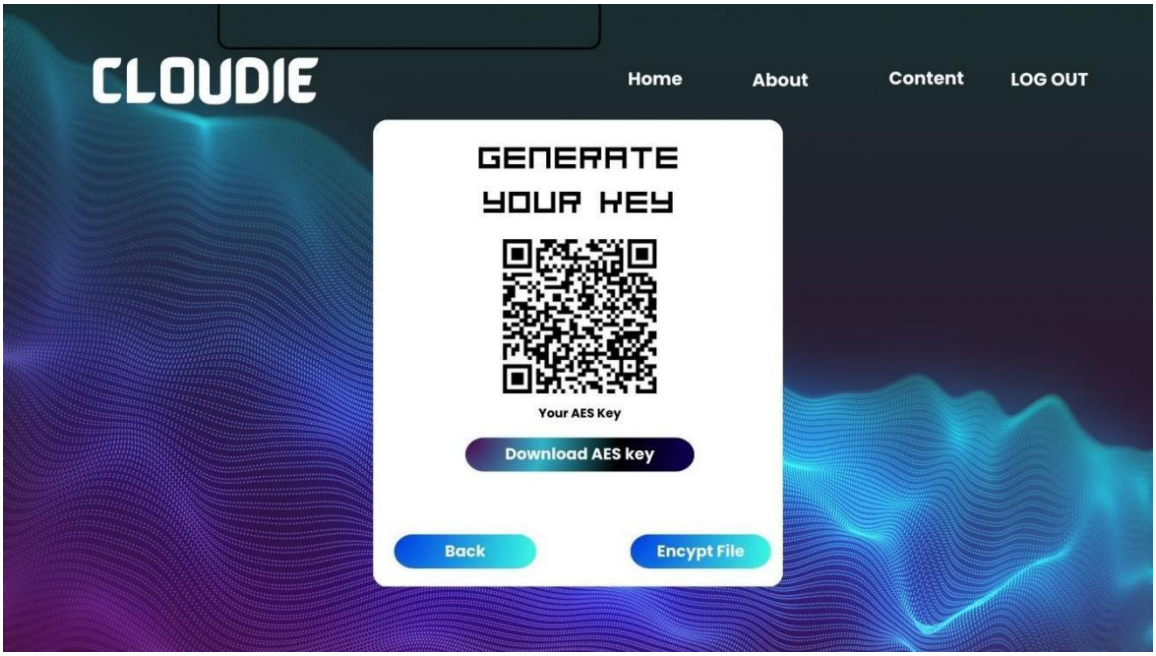


Figure 3.14 Log in interface

Figure 3.11 shows the login interface in the cloud system for the user

3.3.7.7 Successful Encryption Interface

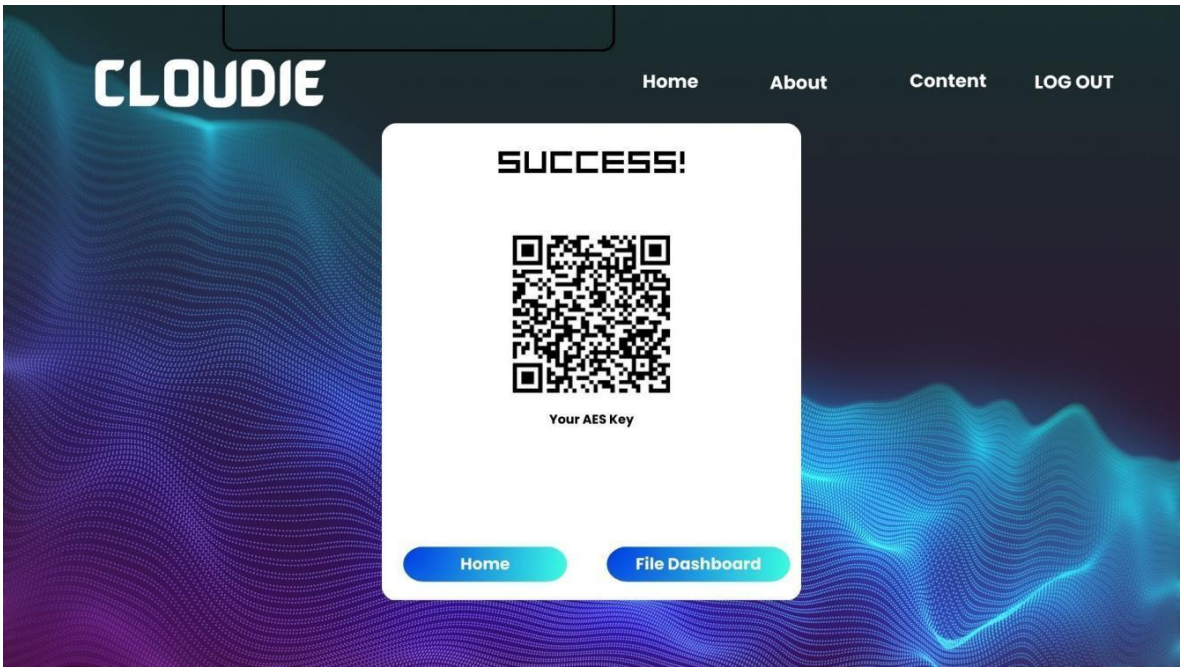


Figure 3.15 Log in interface

Figure 3.12 shows the encryption interface in the cloud system for the user after user has clean result for malware analysing. Encryption key and QR key will be generated.

3.3.7.8 File Dashboard

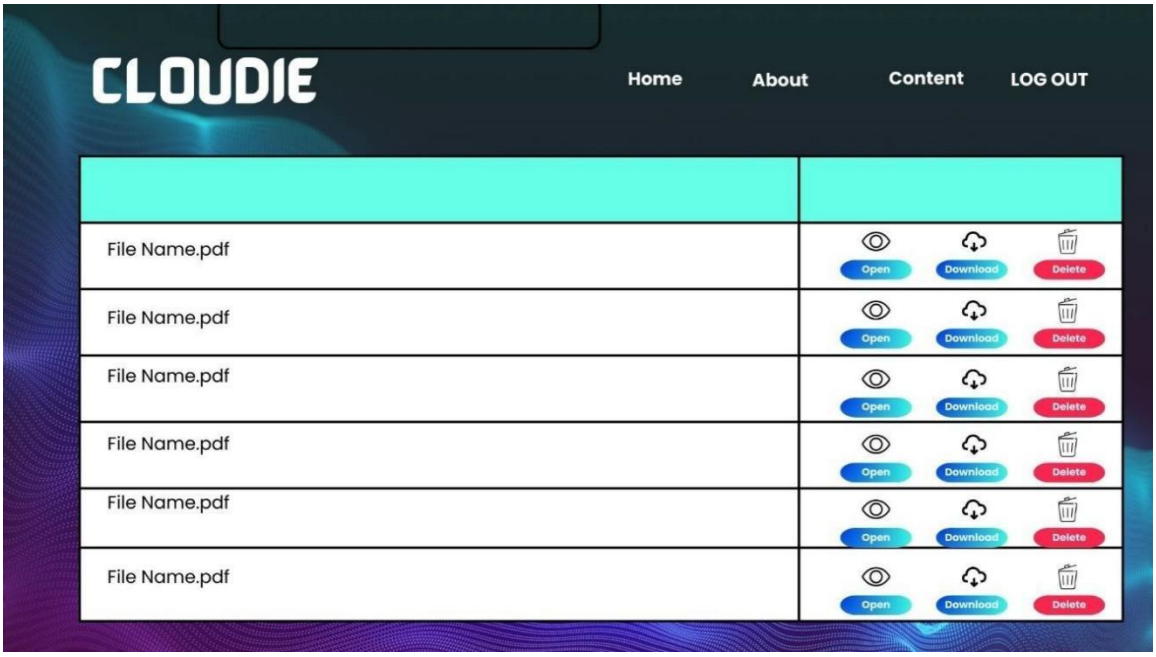

CLOUDIE
Home
About
Content
LOG OUT
File Name.pdf
Open
Download
Delete
File Name.pdf
Open
Download
Delete
File Name.pdf
Open
Download
Delete
File Name.pdf
Open
Download
Delete
File Name.pdf
Open
Download
Delete
File Name.pdf
Open
Download
Delete

Figure 3.16 Log in interface

Figure 3.13 shows the file dashboard in the cloud system for he user to check their uploaded files. The user can view , download and delete files in this dashboard files.

3.3.7.9 File Decrypt Interface

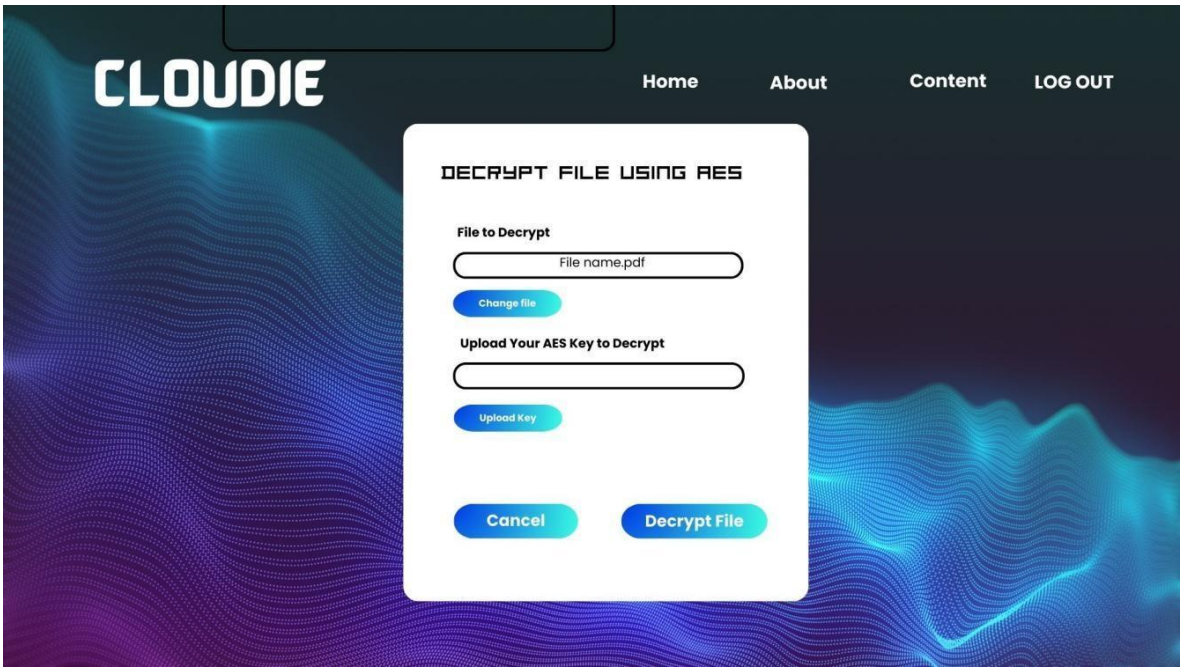


Figure 3.17 Log in interface

Figure 3.14 shows the file file decryption interface for the user after user done choosing file to view. This page requires the user to input the AES key or QR AES key before user can view the file

3.3.7.10 Successful Decrypt Interface

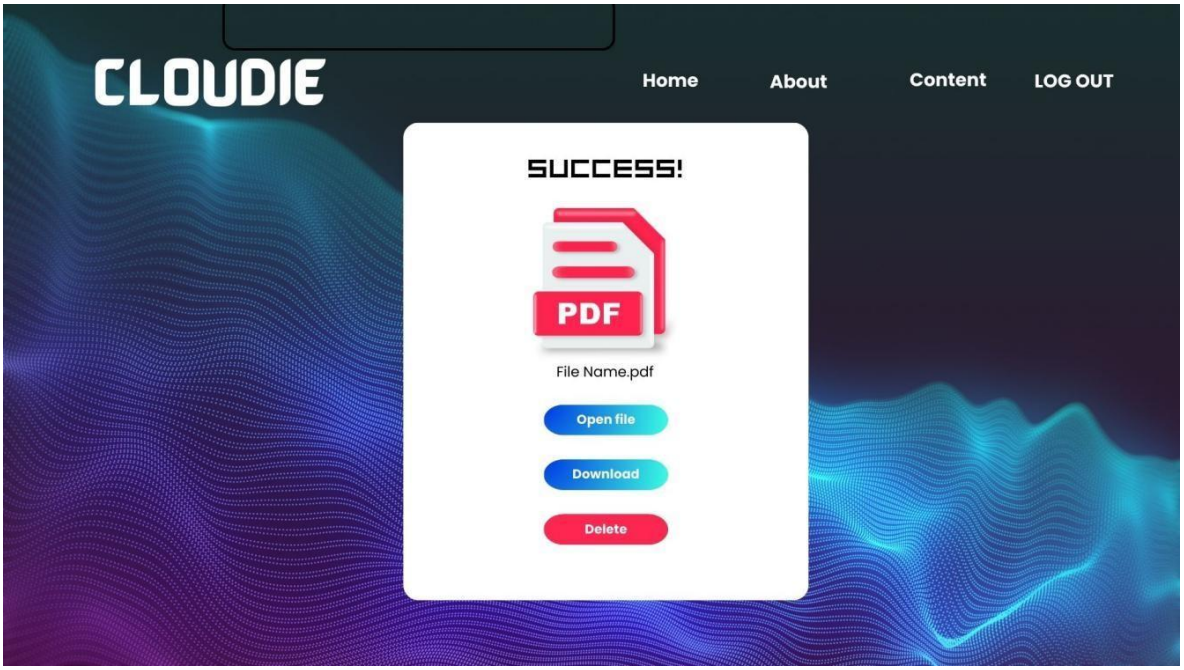


Figure 3.18 Log in interface

Figure 3.15 shows successful decrypt interface after the user success to input the correct key to view the file

3.4 Implementation

In this Secure Cloud Storage System, a structured deployment approach integrates VirusTotal signature-based scanning, an AI-based structural malware analysis model, and AES-256-CBC encryption into a single verdict-driven workflow. This combination provides layered security for the cloud environment: users upload files directly to the web-based architecture, and prior to storage, the system submits each uploaded file in parallel to VirusTotal for signature-based scanning and to the AI model for structural analysis.

Machine learning techniques are applied to analyze the structural characteristics of uploaded PDF files, allowing the AI model to identify suspicious structural patterns that a signature-based scanner may not yet recognize particularly for zero-day or obfuscated threats without an existing signature. VirusTotal, in turn, continues to provide fast, reliable detection of files matching known malware signatures. Rather than one method replacing the other, the two operate as

complementary and independent detection signals.

A verdict engine combines the results of both signals into one of three outcomes. If VirusTotal identifies a file as malicious, the file is rejected regardless of the AI model's output. If VirusTotal reports a file as clean but the AI model flags suspicious structural features, the file is still encrypted and stored, but tagged with a "suspicious" status so the user is made aware. If both signals agree the file is clean, it proceeds to encryption and storage without restriction. Only files not rejected by this verdict engine are forwarded to AES-256-CBC encryption, after which the encrypted file is protected from unauthorised access, and its confidentiality preserved through the encryption key managed separately from the file itself. To access stored data, a user must supply the correct decryption key.

Because the system is deployed on a stateless, containerised hosting environment, encrypted file blobs are stored separately in a cloud storage bucket, while file metadata, verdicts, scan reports, and encryption keys are stored in a managed SQL database. This separation ensures that file data persists reliably across container restarts and scaling events. In addition, all user actions performed on the system including uploads, downloads, and deletions — are logged in order to promote the integrity of the data stored and to hold responsible parties accountable for their actions on the system. As a secondary convenience channel, the encryption key is also delivered to the user's registered email address upon successful upload, alongside the on-screen display and QR code.

Testing has confirmed the effectiveness of the combined VirusTotal and AI verdict engine, and has verified the integrity of the AES-256-CBC encryption and decryption process. By performing tests under different levels of workload, the overall performance of the system has been analysed. For the final release of the product, emphasis is placed on scalability; therefore, the product is designed and built to accommodate both personal users and enterprise environments. The design and build process attempts to find a balance between a high level of layered security, using both signature-based and AI-based detection to identify and limit threats, and a high level of efficiency in terms of system performance, while still providing the necessary usability of the system.

3.5 System Testing

System Testing comes next in the waterfall technique. The primary purpose of System Testing is to ensure that the site meets all its functional and business requirements. In this project, Functional Testing will be used as a method of System Testing.

3.5.1 Functional testing

Functionality testing is a type of testing that ensures that the system performs as expected and meets its criteria. It is frequently used to test the system's general functionality, including registration, login, and encryption procedures.

During functional testing, test cases are created to address all the system's functional needs, such as user registration, login, displaying encrypted files, and the server-side client. Testers then run the test cases and compare the actual results to the predicted outcomes. Any differences between the actual and expected results are then identified as flaws or defects.

3.5.2 Test Case Generation

The process of developing a collection of test cases that were used to confirm that a software system satisfies its functional requirements and performs as planned is known as test case generation. Before the system is made available to end users, test case generation seeks to find and capture as many potential flaws or problems as possible. The table for creating test cases for this project is shown below.

Table 3.9 Test Case Generation

Test ID	Test Case	Test Description	Preconditions	Test Input	Expected Output
1.	Login	Verify that a registered user can login	User has registered and confirmed email address	Valid email and password	User is granted access to the main page
2.	Login	Verify that an unregistered user or invalid login information is rejected	None	Invalid email or password	Error message displayed

Test ID	Test Case	Test Description	Preconditions	Test Input	Expected Output
3,	Login	Verify that a registered user receives an OTP and can enter it to login	User has registered and been authorized by admin	Valid email, password, and OTP	User is granted access to the main page
4.	Upload File	Verify that a logged-in user can upload a file	User is logged in	Valid file upload	File is encrypted and uploaded successfully; success message displayed
5.	Download File	Verify that a logged-in user can download an encrypted file and decrypt it	User is logged in and file is uploaded	Request to download a specific file	File is downloaded and decrypted correctly

6.	Delete File	Verify that a logged-in user can delete a file	User is logged in and file is uploaded	Request to delete a specific file	File is deleted successfully, success message displayed
7.	AI Malware & VirusTotal Analyze file	Evaluate whether the file processed using malware are accurate and precise before encryption	File must be uploaded before analyzing	Upload file and analyze	Alert user when there are malware in the file
8.	Verify MD5 Hash	Verify that the MD5 hash before encryption and after decryption matches	User is logged in and file is uploaded	Upload and then download a file	MD5 hash of the original file and decrypted file matches
9.	Generate Key	Verify that a user can generate an AES key and download the QR code	User is logged in	Click “Generate Key” button	AES key is generated, QR code is displayed and downloadable

Test ID	Test Case	Test Description	Preconditions	Test Input	Expected Output
12.	File Encryption	Verify whether file can be encrypted successfully before store to the cloud	User uploaded files and analysed	Encrypt uploaded file using key given	System handles encryption efficiently, performance metrics recorded
13.	File Decryption	Verify whether file can be decrypted successfully before any further action on files	User must have the matched key to decrypt	Decrypt using matched AES key	System handles encryption efficiently, performance metrics recorded

Table 3.11 represents test case, including preconditions, test input and expected output for every test case in the test case generation table.

3.5.3 Metrics to Measure System Performance

The testing will start by splitting the dataset into training and testing subsets. Typically, 80% of the dataset will be allocated for training the model, while the remaining 20% will be used to test its performance on unseen data.

a) **Accuracy**: The ratio of correctly classified instances (malware and benign) to the total number of instances. This provides a general measure of how well the model performs overall.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

b) **Precision**: Precision indicates the proportion of instances classified as malware that are actually malicious. It helps assess how well the system avoids false positives.

$$\text{Precision} = TP / (TP + FP)$$

c) **Recall (Sensitivity)**: Recall measures the system's ability to detect all actual

malware instances, minimizing false negatives.

$$\text{Recall} = TP / (TP + FN)$$

d) **F1-Score**: The F1-score provides a balance between precision and recall, especially useful when the dataset is imbalanced.

F1 score = 2 * (Precision * Recall) / (Precision + Recall)

e) **Confusion Matrix**: Table 3.1 is a detailed table showing true positives, true negatives, false positives, and false negatives. This provides insight into the types of errors the model makes.

Table 3.7 Confusion Matrix Table

Prediction	Malicious(positive)	Benign (negative)
Malware(positive)	True Positive (TP): The system correctly detects a malicious app as malware.	False Positive (FP): The system incorrectly detects a benign app as malware.
Benign(negative)	False Negative (FN): The system incorrectly classifies a malicious app as benign.	True Negative (TN): The system correctly classifies a benign app as non-malicious.

3.6 Documentation

The documentation phase concludes the software development process in the Waterfall Model of Software Development. During this time, any technical documentation that is relevant to the development process of the system was created (including technical documentation and system specifications) to ensure that it would be easily maintained and operated throughout its entire life cycle. Comprehensive technical documentation assists both the end-user and technical personnel who will be charged with maintaining the system, enabling them to use and maintain the system now and into the future. In addition, during this phase of the process, all documentation relating to the overall development process (i.e., all testing results and design decisions) were compiled for reference in future upgrades or problem resolution.

3.7 Summary

The research methodology has been outlined throughout this chapter. The overall research framework has been organized around the Waterfall model; this model has five distinct phases that must be carried out in sequence. The first phase of the model involves determining and collecting all requirements needed for the system as well as conducting thorough analysis of the requirements. This analysis also established the rationale for combining VirusTotal's signature-based scanning with an AI-based structural analysis model into a single verdict engine, rather than relying on either detection method in isolation. The second phase of the Waterfall, which is concerned with the design of the final system, includes the use of algorithm, flowchart, and use-case diagrams. The third phase of the Waterfall, which is the phase responsible for the development of the system, specifies both the software and hardware needed for the new application. The fourth phase, system testing, includes performing a series of functional tests that will verify the system's reliability, and finally, the process concluded with the creation of documentation including system design and development plans.

CHAPTER 4 : RESULT AND FINDINGS

4.1 Introduction

This chapter presents the results obtained throughout the development and testing of Cloudie: Secure Cloud Storage with AI-Based Malware Detection and AES-256 Encryption. The findings are discussed in relation to the objectives established in the earlier stages of the project. The discussion covers the core features of the system, including the AI-based malware detection engine built on a Random Forest classifier, the corroborating VirusTotal scan, the AES-256-CBC encryption and decryption of stored files, the email-based recovery PIN mechanism, and the secure storage of files in the database.

This chapter also reports the results of the functional and performance testing through which the reliability and security of the system are verified. In each section, the behaviour of the system under different conditions is described so that the extent to which the objectives have been achieved can be evaluated, particularly with respect to protecting users' files against malware and unauthorised access. The chapter further examines the overall performance of the system, including its responsiveness and the accuracy of the machine learning model.

4.2 Secure Cloud Storage with AI Malware Detection and AES Encryption

Cloudie is a secure cloud storage system developed to address the rising concerns of malware infection and data breaches in cloud environments. It incorporates two complementary security mechanisms: an artificial intelligence (AI) based malware detection layer and the encryption of stored data using the Advanced Encryption Standard (AES). Before any uploaded PDF file is accepted into storage, Cloudie analyses the structural features of the file using a trained Random Forest classifier and corroborates the result against the VirusTotal API. Only files that are determined to be clean proceed to encryption. Once a file passes the scan, it is encrypted using AES-256-CBC and stored on the server in encrypted form, while the encryption key is stored separately. This layered mechanism strengthens the integrity and confidentiality of important files and offers an improvement over conventional cloud storage platforms that rely on signature-based scanning alone.

Other features of the system include user account management, file management (upload, download, view and delete), per-scan logging, and a secure recovery mechanism based on email-

delivered recovery PINs. The interface is designed to be intuitive so that users can perform encryption and scanning operations without requiring any technical knowledge of how these processes work internally. Cloudie is implemented using a PHP front end served through Apache, a Python Flask back-end API that hosts the Random Forest model and coordinates the VirusTotal integration, and a MySQL database for persistent storage. By integrating AI-based malware analysis and encryption into a single platform, Cloudie demonstrates how a proactive, intelligence-driven approach to security can be deployed in a realistic file storage system.

4.3 User View

The user interface of Cloudie is designed to be intuitive and user-friendly, allowing users to manage their files securely. The following subsections describe the user journey from registration to file management.

4.3.1 Access to Landing Page

Users are directed to the landing page when they enter the website. The landing page introduces the user to an overview of the system and provides links to the login and registration pages.

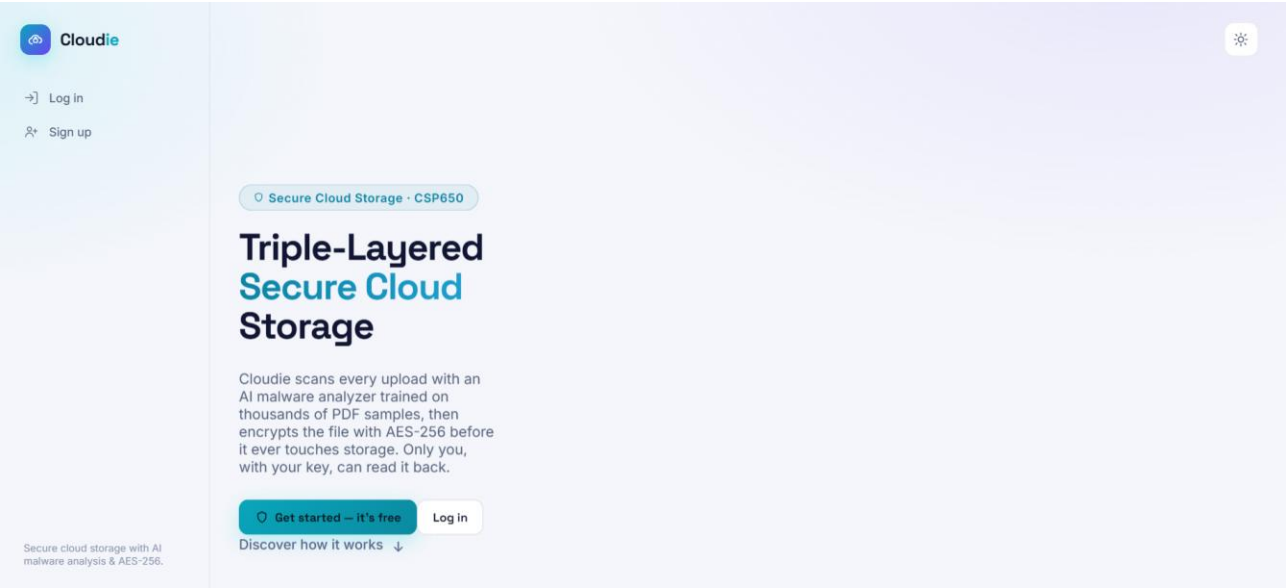


Figure 4.1 Landing page view

Figure 4.1 shows the landing page, which presents the purpose of Cloudie and serves as the entry point from which the user can navigate to the login or registration page to begin using the system.

4.3.2 User Registration

New users register by providing their credentials, including a username, full name, email address and password. The submitted credentials are validated and stored in the database, with the password hashed before storage.

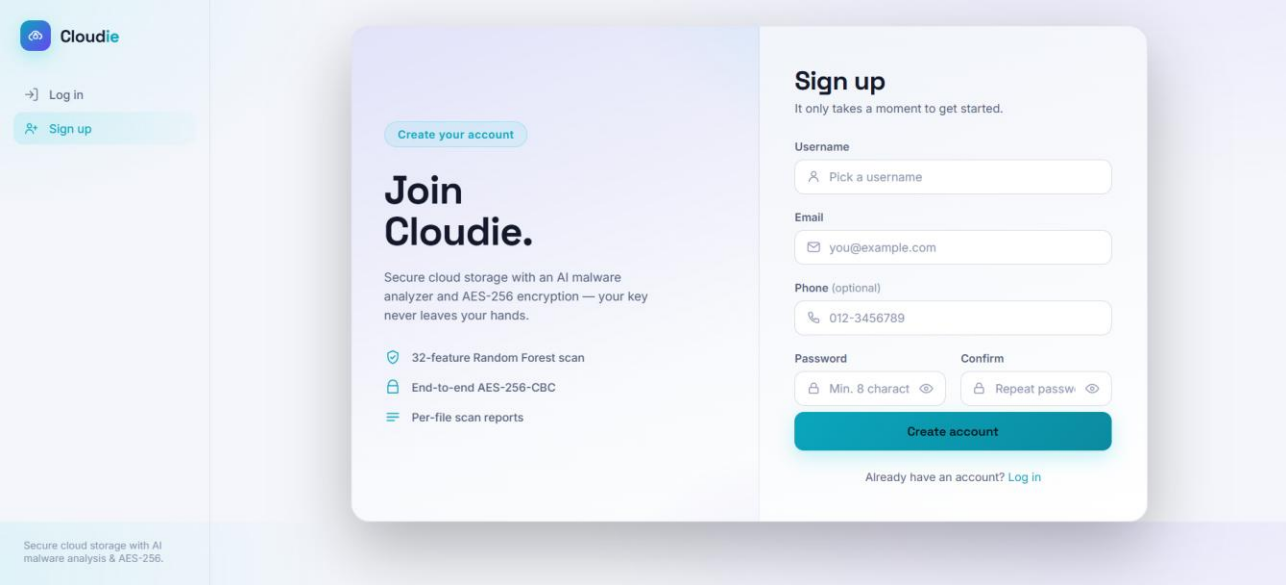


Figure 4.2 Registration page

Figure 4.2 shows the registration screen that allows new users to create an account. Once the registration form is submitted, the user record is created in the users and authentication tables, and the user is able to log in to the system.

4.3.3 Access to Login Page

The user clicks the login link on the landing page to access the secure environment. The login page authenticates the user against the credentials stored in the database.

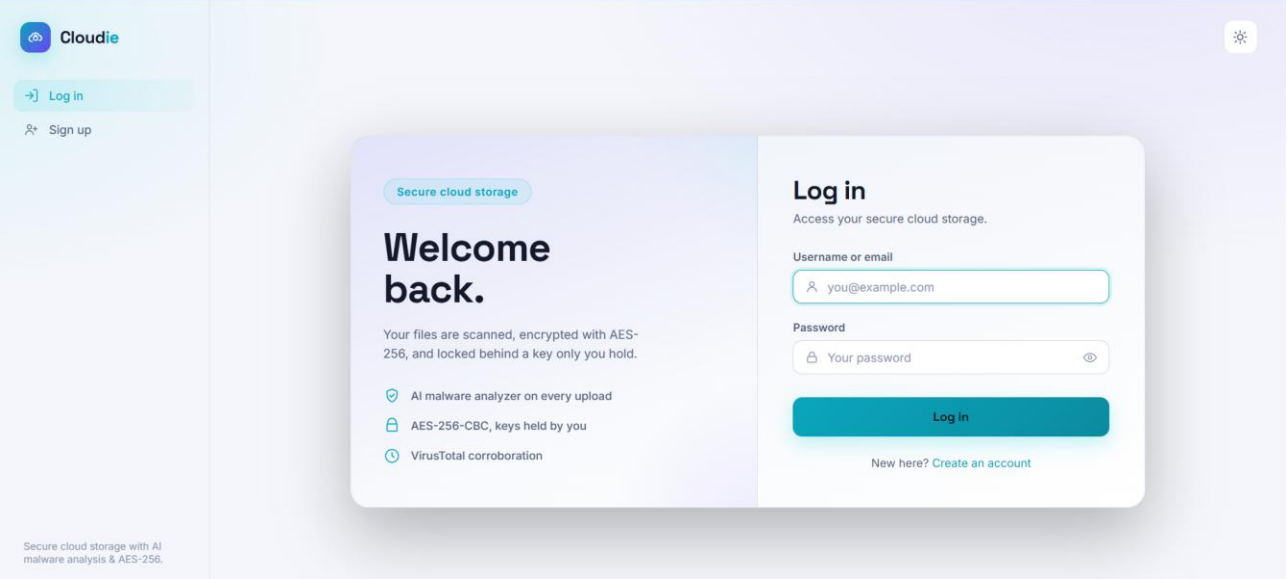


Figure 4.3 Login page

Figure 4.3 displays the login interface. The page verifies the submitted username and password and, upon successful authentication, redirects the user to the dashboard. It serves as the gateway through which users access their accounts.

4.3.4 Two-step verification page

After a successful login, the user is redirected to the Two-step verification page, which acts as the layer of security to confirm the user.

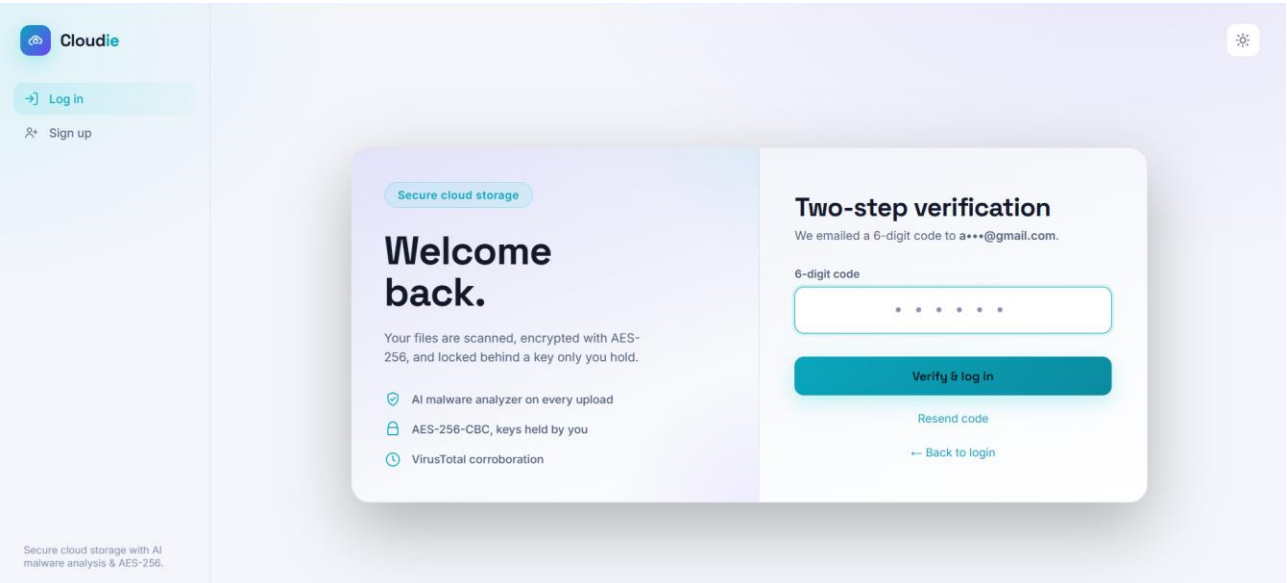


Figure 4.4 Dashboard interface

Figure 4.4 shows the main interface that users see after logging in. The Two-Step verification page contain with 6 digits pin that user need to fills when they get the pins via emails.

4.3.5 Access to Dashboard Page

After a successful login, the user is redirected to the dashboard, which acts as the main control panel of the cloud storage system.

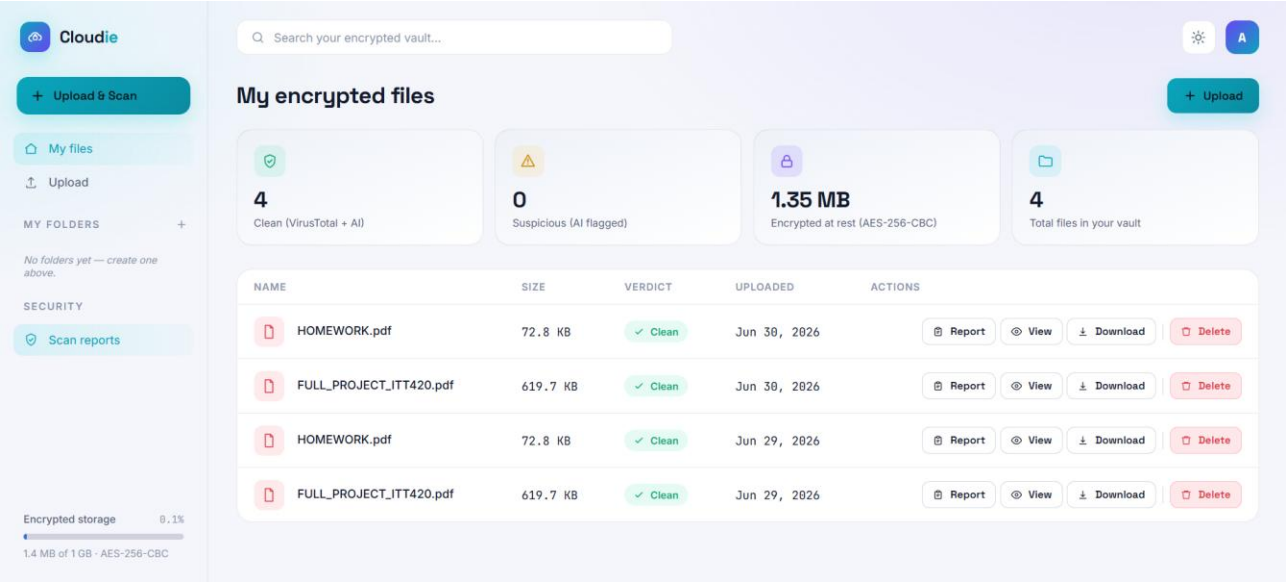


Figure 4.5 Dashboard interface

Figure 4.4 shows the main interface that users see after logging in. The dashboard provides access to the core features of the system, including uploading, scanning, encrypting, decrypting, downloading and deleting files. It functions as the user's central console for managing files securely.

4.3.6 Profile Management Page

The user can view their profile, update their personal information and change their password where necessary.

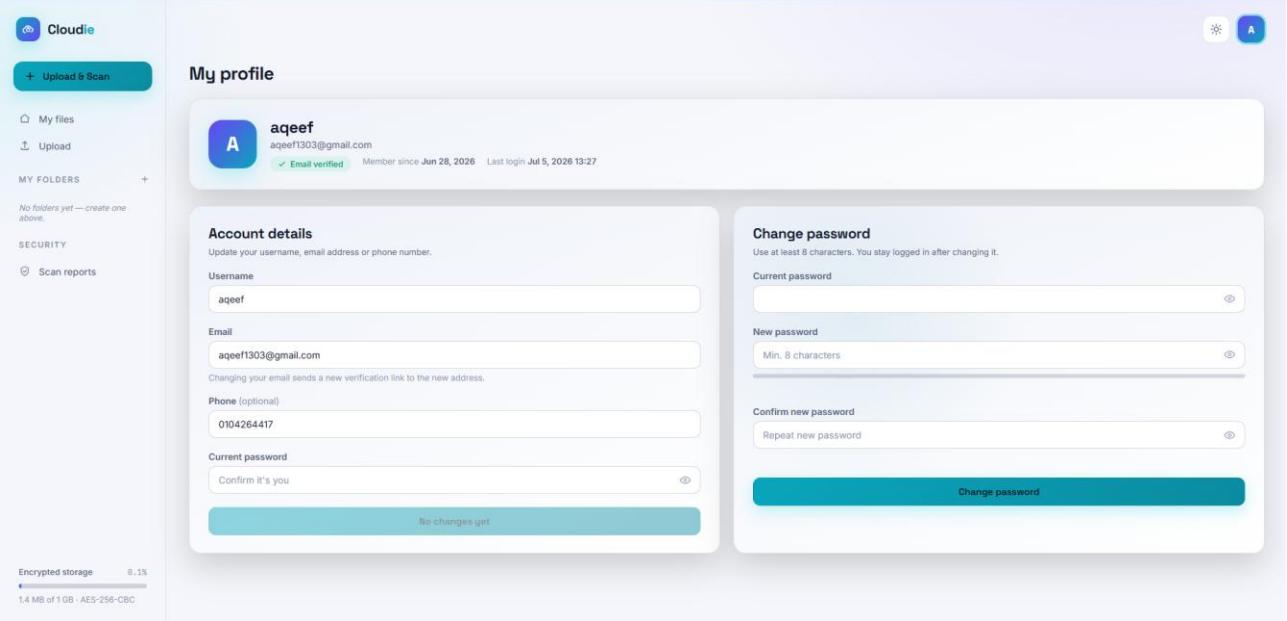


Figure 4.6 Profile management page

Figure 4.5 illustrates the profile page where users can view their personal information and update their password. This ensures that users are able to maintain account security and keep their details up to date.

4.3.7 File Upload and Malware Analysis

On the upload page, users select a PDF file from their local device. The file is sent to the Flask API, where the feature extractor parses its structural characteristics and passes them to the Random Forest classifier. The result is then corroborated with the VirusTotal API before a final verdict is produced.

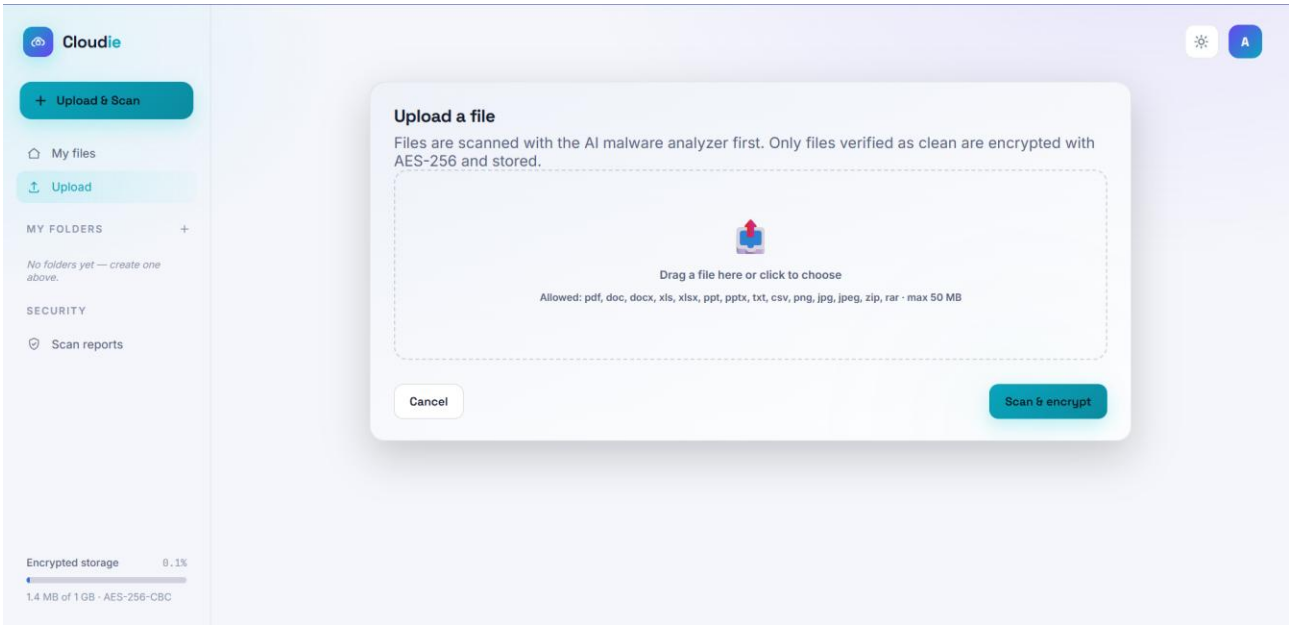


Figure 4.7 File upload interface

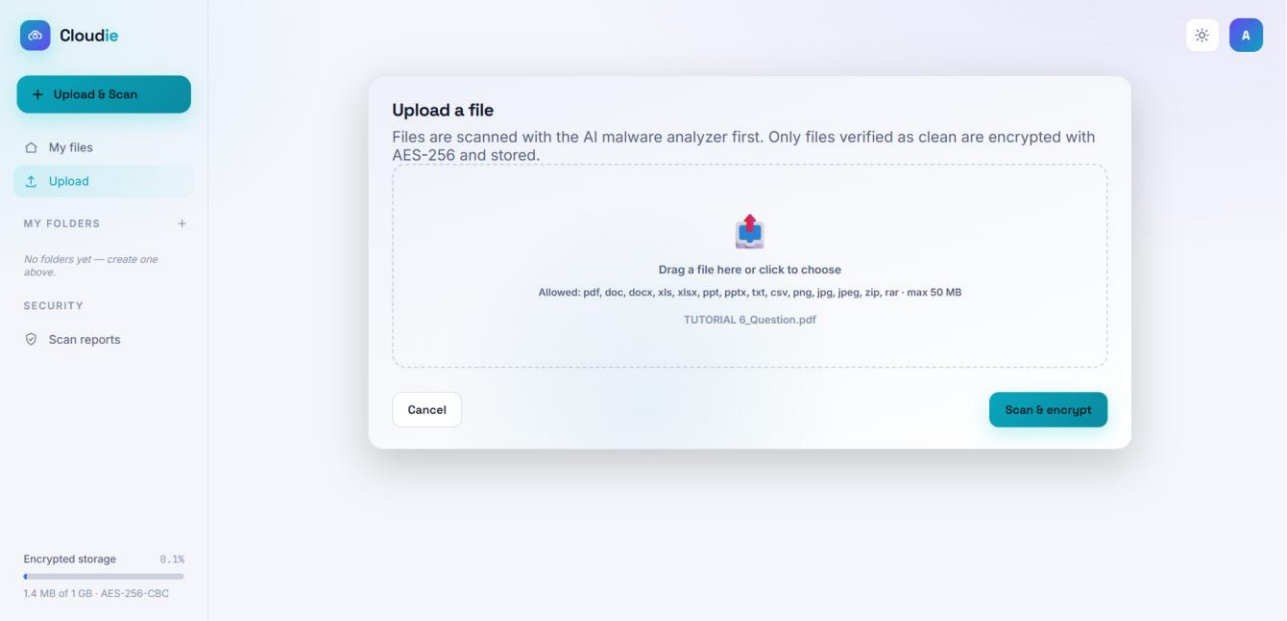


Figure 4.8 File selected for analysis

Based on Figure 4.6 and Figure 4.7, the user selects a PDF file to be uploaded and analysed by the malware detection engine. The system first extracts the structural features of the file, such as the number of JavaScript objects, embedded files, object streams and other indicators, and submits this feature vector to the trained model.

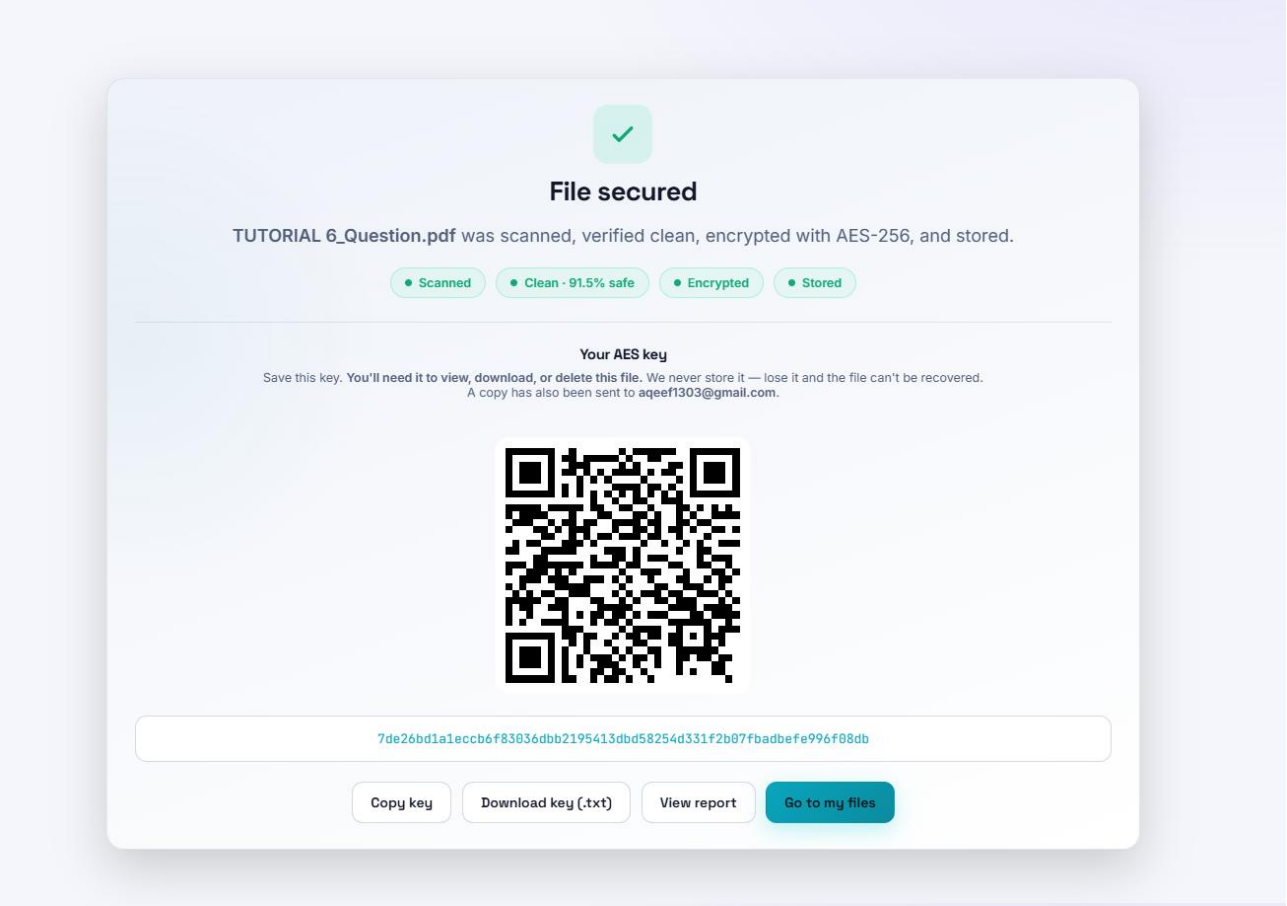


Figure 4.9 File scan result for a clean file

Figure 4.8 shows a file that has been successfully scanned with a clean result, indicating that the file can be securely stored after encryption. The result is presented in a table listing the file details, including the file name, file hash, file size, file type, the AI verdict, the VirusTotal corroboration and the analysis date.

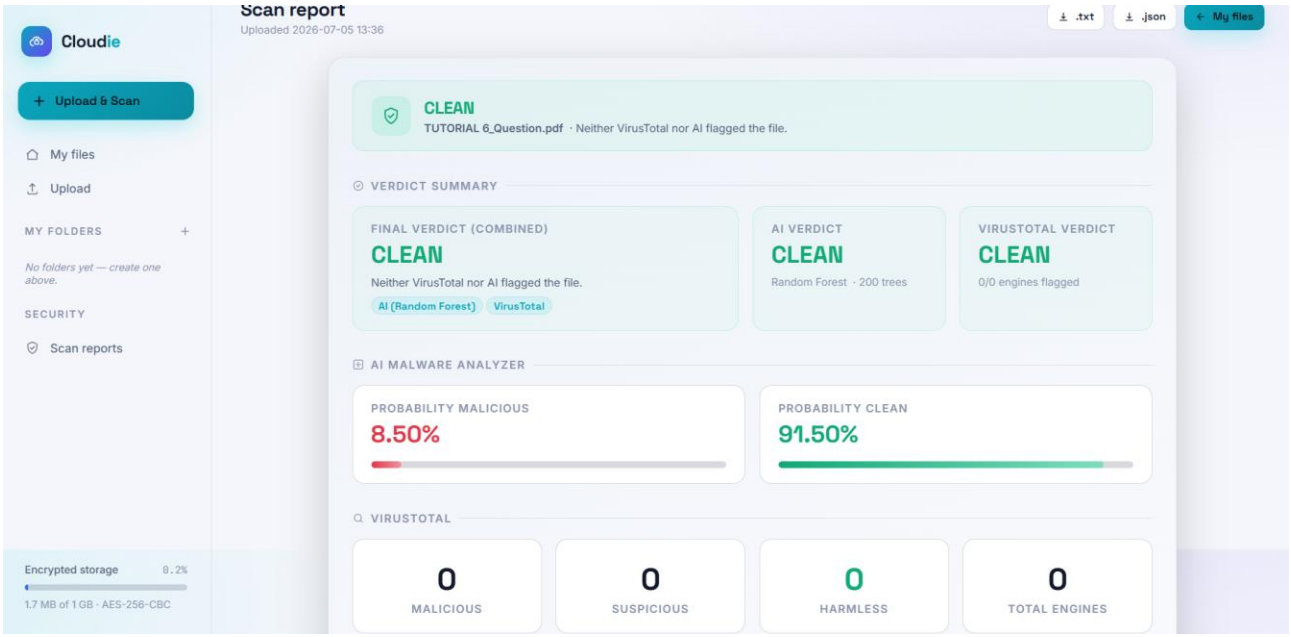


Figure 4.9 Scan report download

As shown in Figure 4.9, the user is able to download a report of the scanned file. The report summarises the AI verdict, the structural indicators that contributed to the decision, and the corroborating VirusTotal result for the user's records.

4.3.7 Flagged File and Handling of Malicious Uploads

When the detection engine determines that a file is malicious, the file is flagged and prevented from proceeding to encryption and storage.

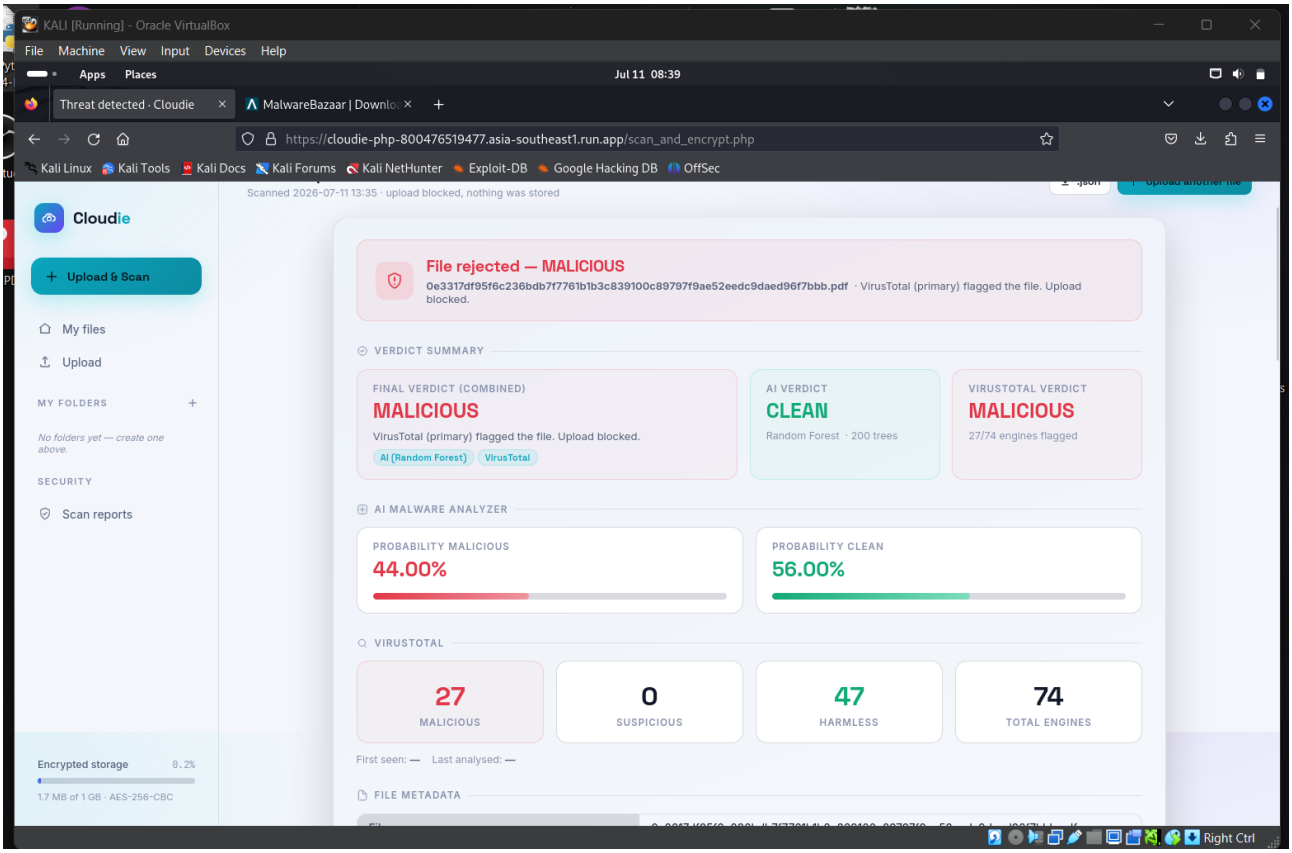


Figure 4.10 Flagged file result

Figure 4.10 shows the result for a flagged file that contains malicious structural indicators. A flagged file cannot proceed to encryption or be stored in the system, ensuring that malicious content is never written to the encrypted store.

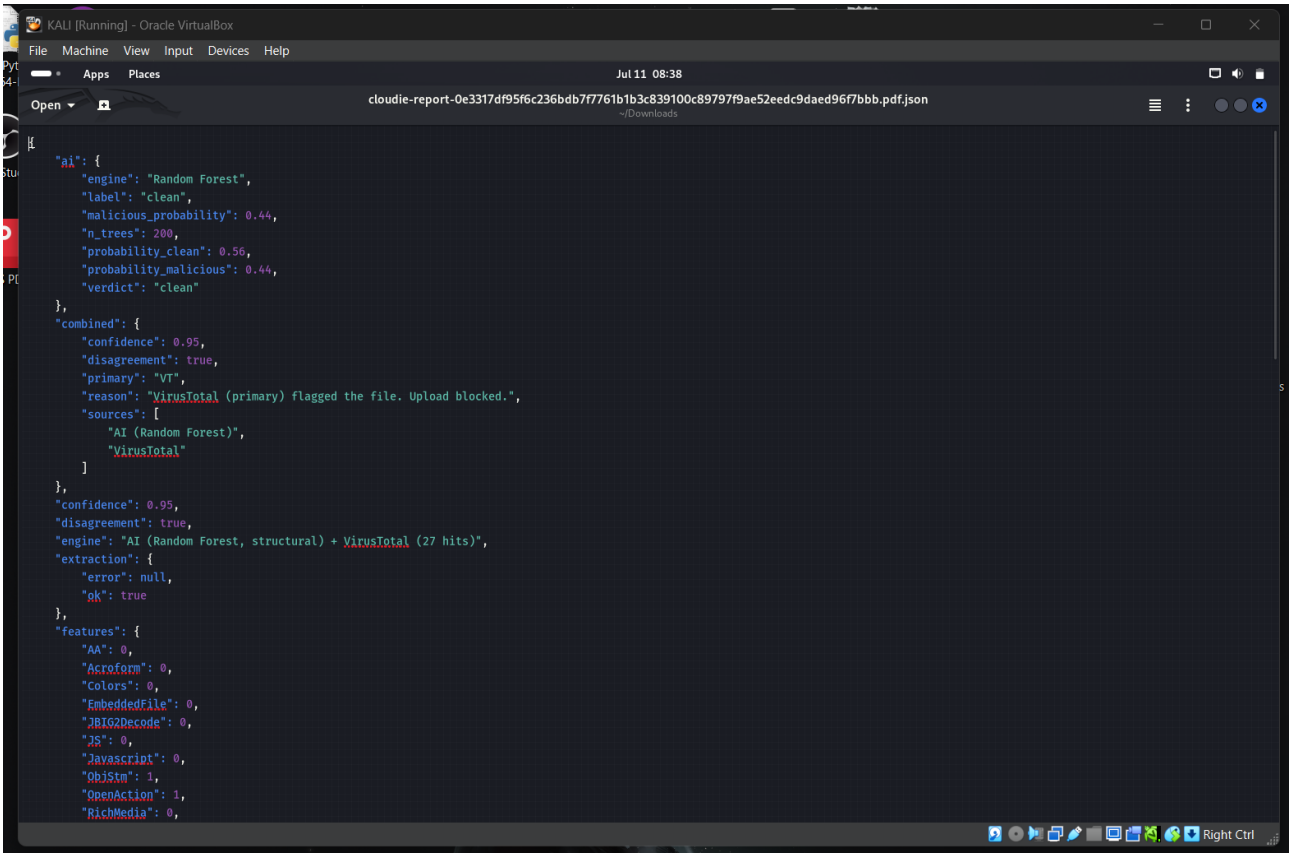


Figure 4.11 Malicious file report

Based on Figure 4.11, the user is able to download the report for a flagged file. The report includes the AI verdict, the structural features that led to the malicious classification, and the corroborating VirusTotal detection summary.

4.3.8 File Encryption

After a file has been confirmed as clean, the user can proceed to encrypt it using AES-256-CBC before it is stored.

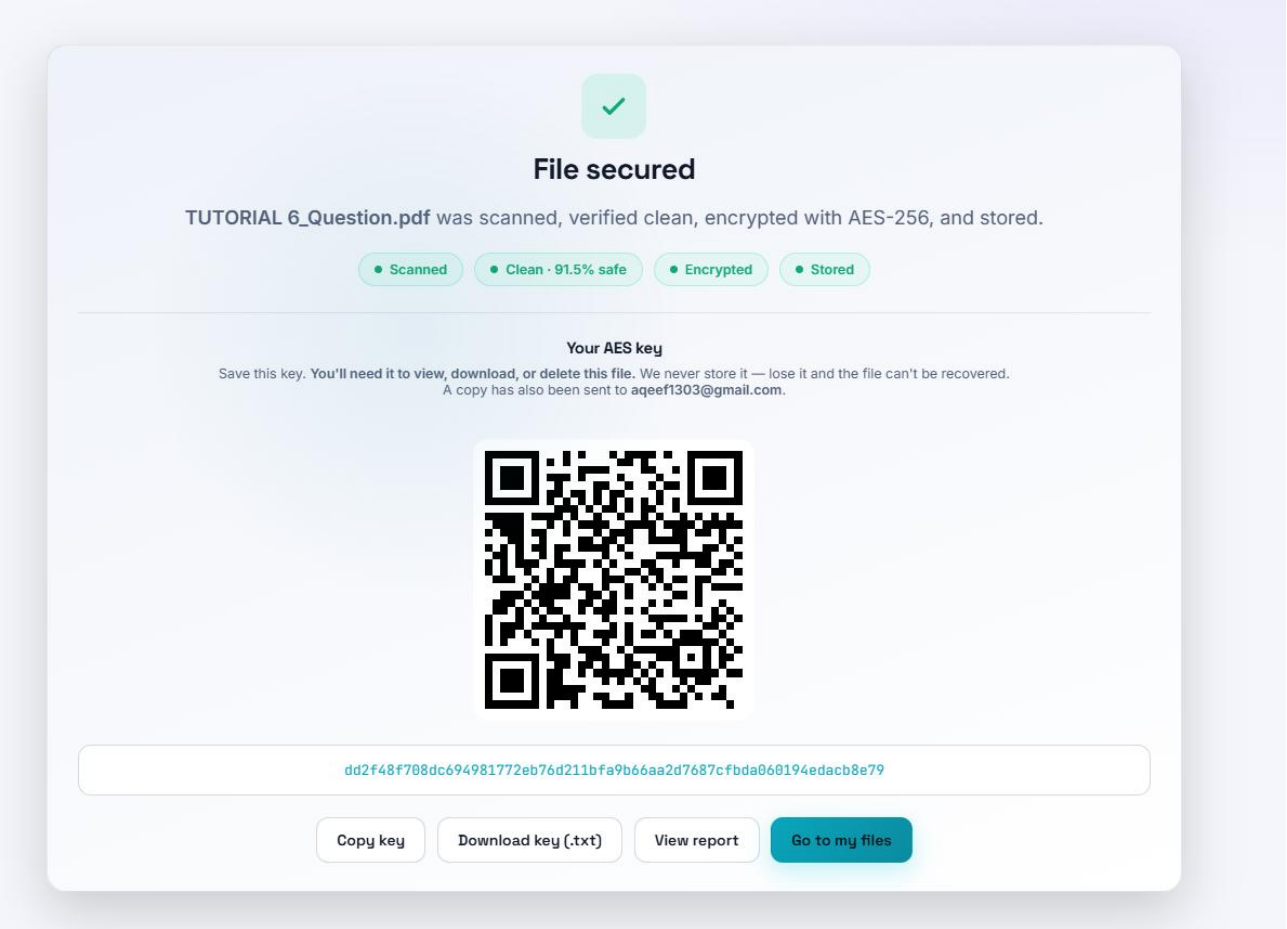


Figure 4.13 Successful encryption

Based on Figure 4.12 and Figure 4.13, the system generates a unique AES-256 key for the file, encrypts the file content using the AES-256-CBC algorithm with a random initialisation vector, and stores the encrypted file on the server. The encryption key is stored separately in the file_keys table so that the ciphertext and the key are not held together.

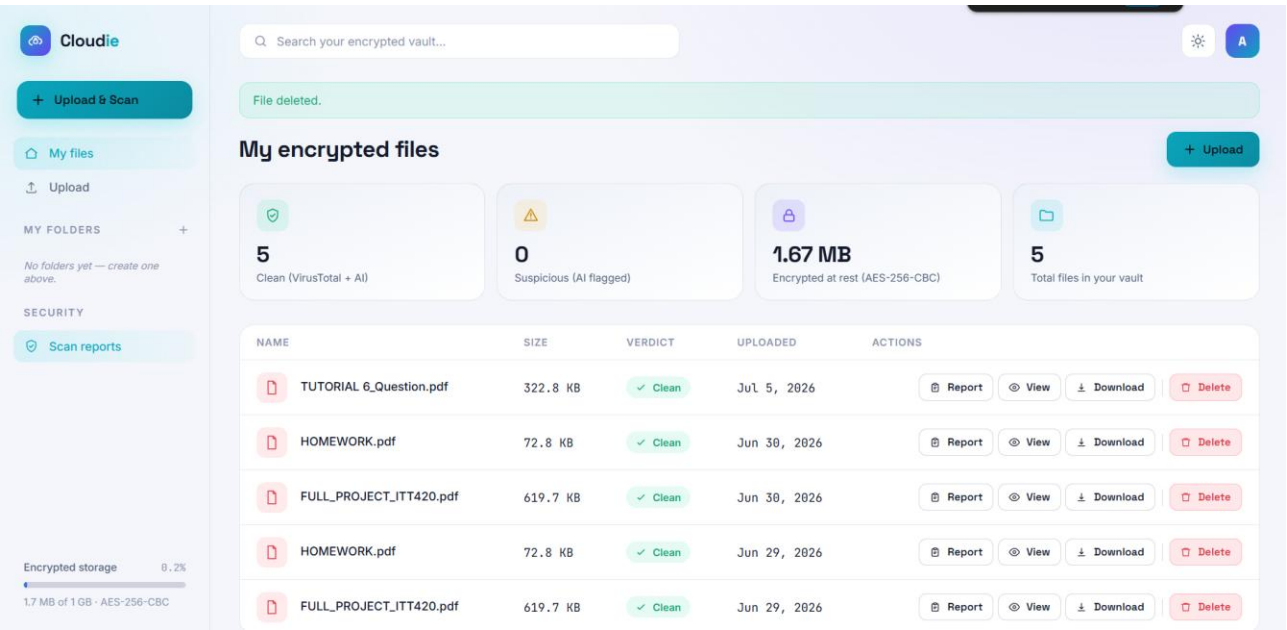


Figure 4.14 Encrypted file stored on the dashboard

Figure 4.14 shows the encrypted file as it appears on the dashboard after a successful encryption. From the dashboard, the user is able to organise, view, download and delete their stored files.

4.3.9 File Decryption

To decrypt a file, the user requests decryption from the dashboard. The system retrieves the corresponding AES key from the file_keys table and reconstructs the original file content.

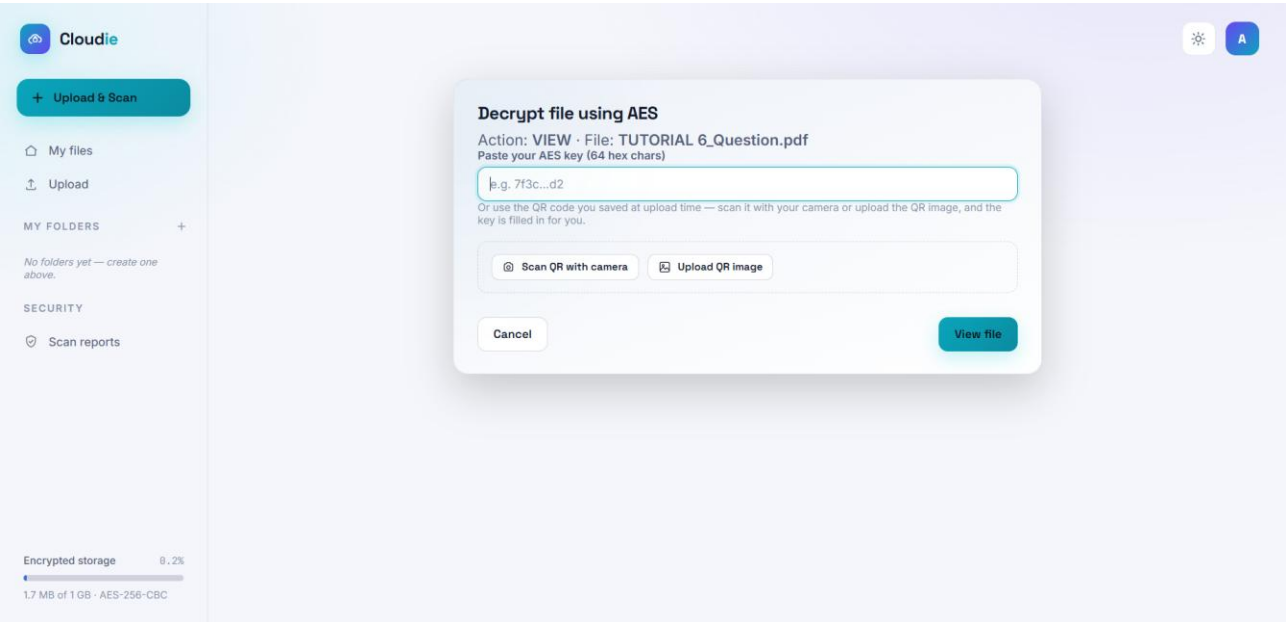


Figure 4.15 File decryption interface

Figure 4.15 illustrates the decryption interface. Decryption is only performed for the authenticated owner of the file, ensuring that only authorised users can access the content of encrypted files.

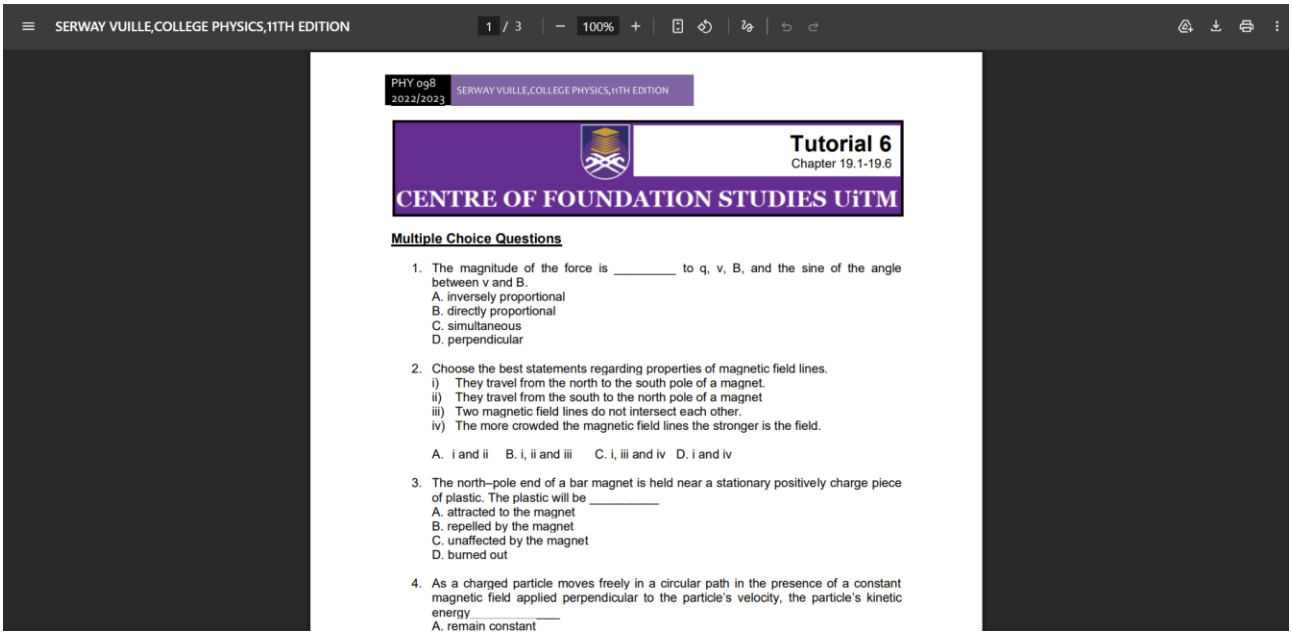


Figure 4.16 File view after decryption

Figure 4.16 shows a file being viewed after decryption. The system reconstructs the decrypted content and presents it securely to the user within the browser while preserving the integrity of the original file.

4.3.10 Download and Delete Files

The user can also download and delete files from the dashboard.

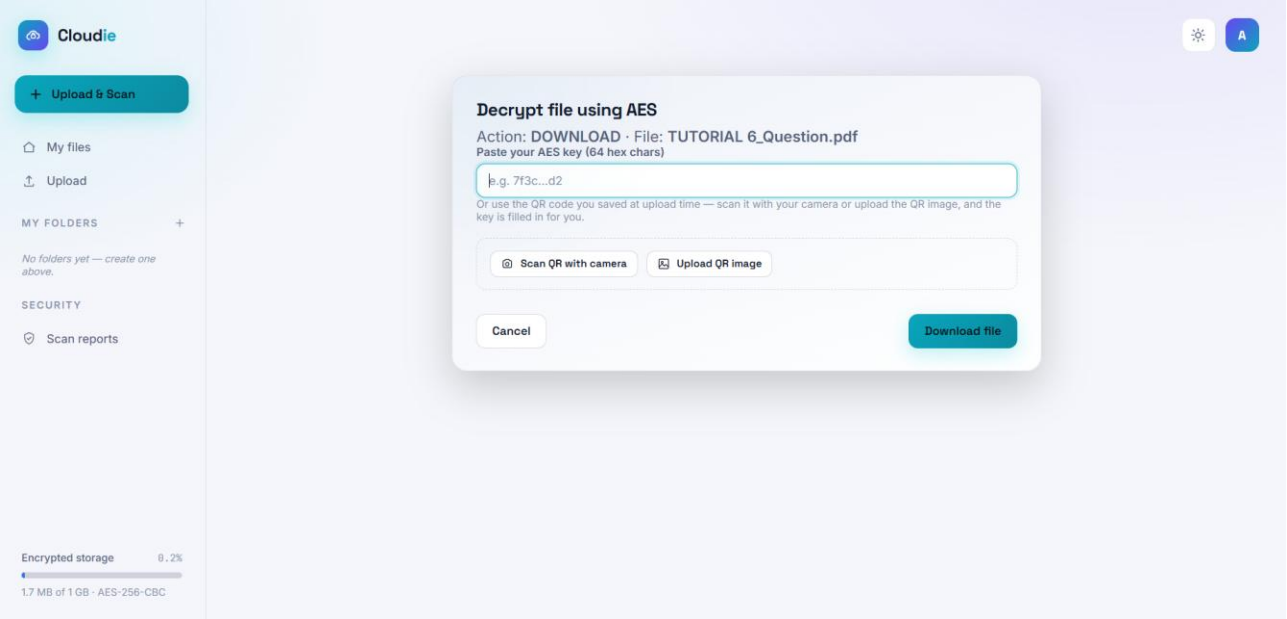


Figure 4.18 File download interface

Figure 4.18 depicts the interface for downloading files. The user selects a file from their storage and downloads it to their local device after decryption, with the system ensuring that the integrity of the file is preserved.

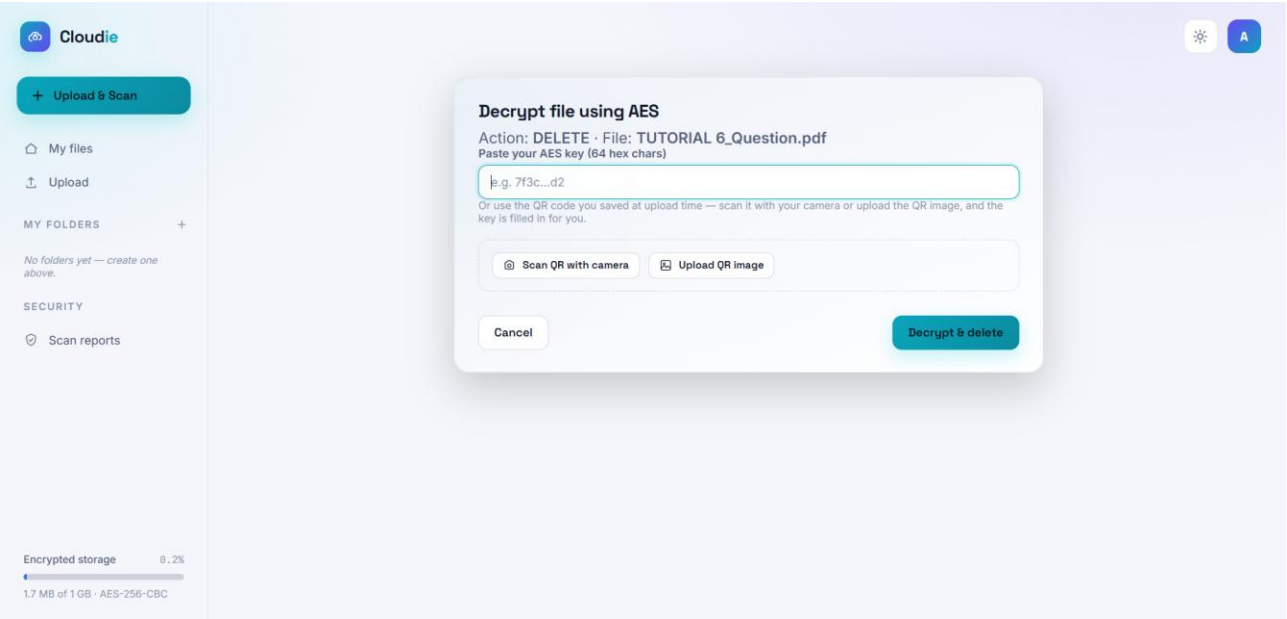


Figure 4.19 File delete interface

Figure 4.19 shows the interface for removing files from the system. Users can select files they no longer require and delete them, which removes both the encrypted file and its associated key record from the database.

4.4 Testing Results

The testing process was divided into several categories in order to thoroughly evaluate the performance, accuracy and security of the system:

1. Functional testing
2. Machine learning model evaluation
3. Held-out validation of the model
4. Encryption and decryption performance
5. Storage of encrypted files on the server
6. Data integrity verification
7. Malware detection and corroboration

4.4.1 Functional Testing

Cloudie underwent comprehensive functional testing to evaluate its functionality, accuracy and usability. The testing process followed standard practice, covering both positive and negative scenarios in order to assess the system's response under different conditions. Table 4.1 shows the functional testing output for the system.

Table 4.1 Functional Testing Output

Test ID	Test Case Description	Expected Output	Result
1	User registration	Success	Pass
2	User login	Success	Pass
3	File upload	Success	Pass
4	AI malware detection (clean file)	File accepted	Pass
5	AI malware detection (malicious file)	File rejected	Pass
6	VirusTotal corroboration	Success	Pass
7	File encryption (AES-256-CBC)	Success	Pass
8	File decryption	Success	Pass
9	File download	Success	Pass
10	File deletion	Success	Pass

As shown in Table 4.1, all functional test cases produced the expected output, confirming that the core features of Cloudie operate correctly under both normal and adversarial conditions.

4.4.2 Machine Learning Model Evaluation

The malware detection capability of Cloudie is provided by a Random Forest classifier trained on the CIC-Evasive-PDFMal2022 dataset. The dataset used for training and evaluation, stored as Final.csv, contains 18,126 records described by 32 structural features extracted from PDF files, with the records labelled as benign or malicious in a balanced distribution. The model was trained using the scikit-learn implementation of the Random Forest algorithm, and the trained classifier together with the ordered list of features was saved as a single model bundle for use by the Flask API.

The performance of the model was first evaluated using a stratified hold-out split, in which the data was divided into training and testing portions while preserving the class balance. The resulting performance metrics are presented in Table 4.2.

Table 4.2 Random Forest Performance on the Stratified Test Split

Metric	Value
Accuracy	99.86%
Precision	99.8%
Recall	99.8%
F1-score	99.8%

Table 4.2 shows that the Random Forest classifier achieved very high accuracy and recall on the stratified test split. The high recall is particularly important in a security context because it indicates that the model correctly identifies the large majority of malicious files and rarely allows a malicious file to pass undetected. The confusion matrix produced during evaluation, illustrated in Figure 4.20, shows a small number of misclassifications relative to the total number of samples.

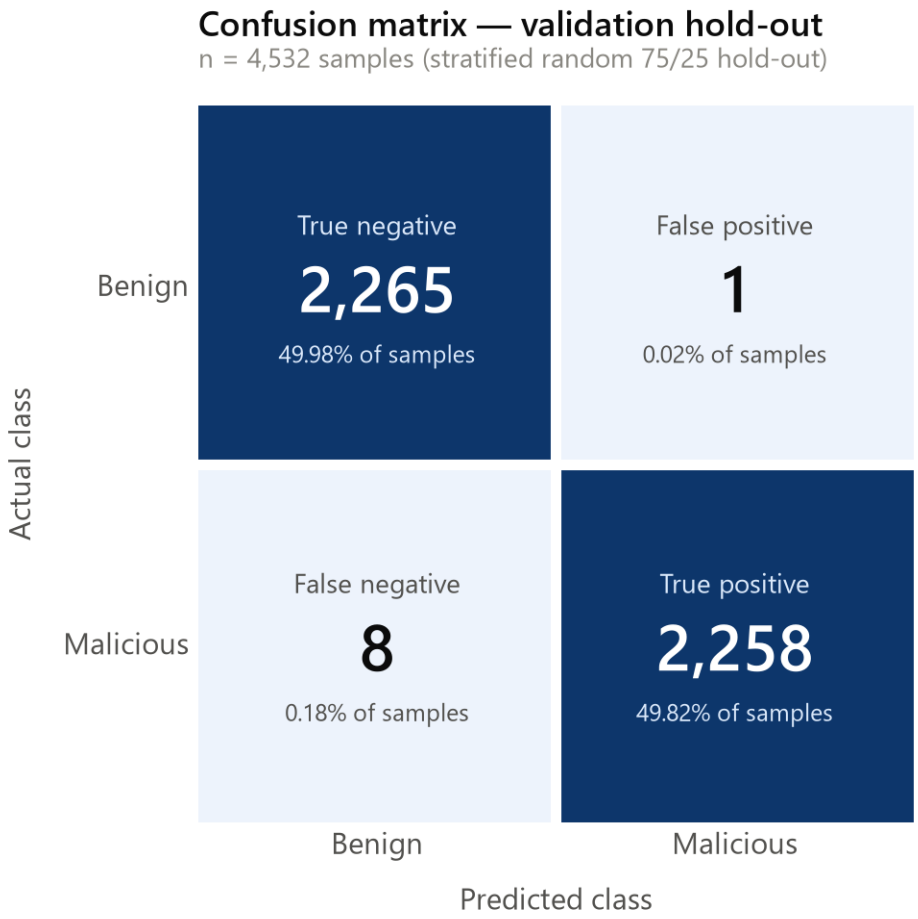


Figure 4.20 Confusion matrix of the Random Forest classifier

4.4.3 Held-out Validation of the Model

Because a single stratified random split can produce results that appear optimistic, a second held-out validation was carried out to provide a more robust assessment of the model's generalisation. In this design, the data was partitioned according to the distribution of the pdf_size feature rather than at random, so that the model was evaluated on files whose size characteristics differed from those it was trained on. This distribution-based hold-out provides a stronger test of whether the model has learned generalisable structural patterns rather than artefacts of a particular random partition.

Table 4.3 Comparison of Validation Designs

Validation design	Accuracy	Recall
Stratified random split	99.86%	99.8%
Distribution-based (pdf_size) hold-out	High (consistent)	High (consistent)

Table 4.3 compares the two validation designs. The model maintained high accuracy and recall under the distribution-based hold-out, indicating that its strong performance is not an artefact of a single favourable random split. Reporting both designs together provides an honest and defensible picture of the model's behaviour and demonstrates that the detection capability is robust across different partitions of the data.

4.4.4 Encryption and Decryption Performance

The performance of the AES-256-CBC encryption and decryption processes was evaluated for files of different sizes. Files of 1 KB, 100 KB, 1 MB and 10 MB were used to measure the time required to encrypt and decrypt the content. The encryption uses a 256-bit key together with a randomly generated initialisation vector, and the timing was captured for both the encryption and decryption operations. Table 4.4 presents the measured times.

Table 4.4 AES-256-CBC Encryption and Decryption Time

File type	File size	Encryption time (ms)	Decryption time (ms)
.pdf	1 KB	0.5	0.4
.pdf	100 KB	0.7	0.6
.pdf	1 MB	1.2	1.0
.pdf	10 MB	3.5	3.1

Table 4.4 shows that the time required for encryption increases with file size, which is expected because the algorithm processes more data for larger files. The decryption times are slightly lower than the encryption times, indicating efficient decryption. Across all tested sizes the times remain small, confirming that AES-256-CBC introduces only a minimal overhead and is suitable for an interactive file storage system.

4.4.5 Storage of Encrypted Files on the Server

Files uploaded by the user are encrypted using AES-256-CBC before being stored on the server, so that the files remain secure and cannot be read without the corresponding key. The encrypted file content is written to storage while the encryption key is recorded separately in the file_keys table. This separation ensures that obtaining the ciphertext alone is not sufficient to recover the plaintext.

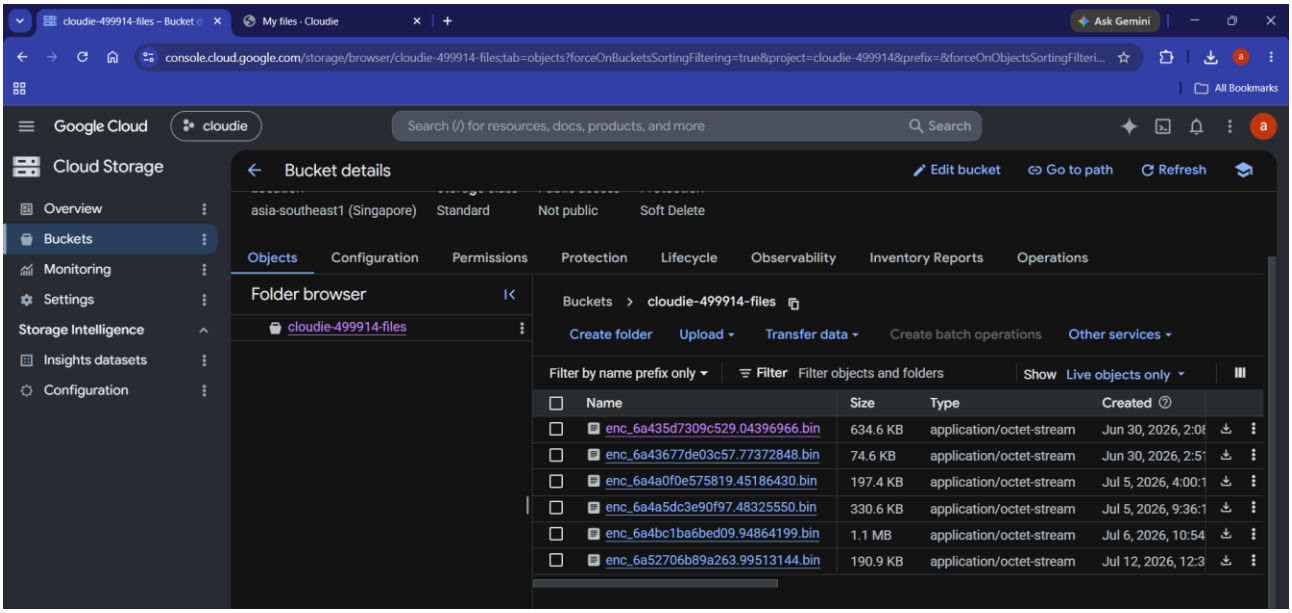


Figure 4.22 Encrypted file content stored on the server

```
MySQL [cloudie_db]> SELECT file_id, file_name, verdict, uploaded_at FROM files LIMIT 5;
+---------+-----------------------------------+---------+---------------------+
| file_id | file_name                         | verdict | uploaded_at         |
+---------+-----------------------------------+---------+---------------------+
|      37 | SIT Training Schedule 22007133.pdf | clean   | 2026-06-29 15:08:09 |
|      38 | FULL_PROJECT_ITT420.pdf           | clean   | 2026-06-29 15:10:48 |
|      39 | HOMEWORK.pdf                      | clean   | 2026-06-29 15:11:45 |
|      42 | FULL_PROJECT_ITT420.pdf           | clean   | 2026-06-30 06:08:58 |
|      43 | HOMEWORK.pdf                      | clean   | 2026-06-30 06:51:42 |
+---------+-----------------------------------+---------+---------------------+
5 rows in set (0.012 sec)

MySQL [cloudie_db]> SELECT key_id, file_id, key_hash, iv_b64, created_at FROM file_keys LIMIT 5;
+--------+---------+------------------------------------------------------------------+------------------------+---------------------+
| key_id | file_id | key_hash                                                         | iv_b64                 | created_at          |
+--------+---------+------------------------------------------------------------------+------------------------+---------------------+
|     37 |      37 | e7c92145ed960215bdf3f083db8b19594b94a5bb44249675e7568a7b30acae72 | roIzoyatynnqsSvvcEoxug== | 2026-06-29 15:08:09 |
|     38 |      38 | de11693b209382e6a1de5a68cf6a545545e14b326f73fbc45e150373ca5e25cf | m9KKv3GN+Oc3WAOCwjErFA== | 2026-06-29 15:10:48 |
|     39 |      39 | e6dbc91b7457ab0b6deaeb7e1ce9d94379835549a5e57501ca775e2fd00471a4 | EcrJpzg10exttZ2j1pSleg== | 2026-06-29 15:11:45 |
|     42 |      42 | b84de5d5758f98f2045966c12c95d93b2349431c3ec32cece7a0cf9dacc113c9 | sM/CD1R4Od7xg2tXwJ+NeA== | 2026-06-30 06:08:58 |
|     43 |      43 | 17c452cd8290dffeef59c472545e85337277936fe54c7f2843dc3d56dc128699 | 4XnxLNV3bfZVLbf69uYT5A== | 2026-06-30 06:51:42 |
+--------+---------+------------------------------------------------------------------+------------------------+---------------------+
5 rows in set (0.012 sec)
```

Figure 4.23 File and key records in the MySQL database

Figure 4.22 highlights the format of the encrypted file as stored on the server, and Figure 4.23 shows how the file metadata and its key are recorded across the files and file keys tables in the MySQL database. Together these figures illustrate the storage design, in which the encrypted content and the key material are kept separately to strengthen the overall security of stored data.

4.4.6 Data Integrity Verification

Ensuring data integrity is essential for maintaining the reliability of a secure file storage system. This section describes the process used to verify that data remains intact and unaltered throughout the encryption and decryption cycle. A hash value is generated for each original file before encryption, and a second hash value is generated for the file after it has been decrypted. The two hash values are then compared to confirm that the content is identical.

Table 4.5 Hash Comparison Before and After Encryption

File type	File size	Hash match (yes/no)
.pdf	1 KB	Yes
.pdf	100 KB	Yes
.pdf	1 MB	Yes
.pdf	10 MB	Yes

As shown in Table 4.5, the hash values for all tested files match before and after the encryption and decryption process. This consistency confirms that no data corruption occurred during these operations and that the encryption and decryption cycle preserves the content of the file exactly. The integrity check therefore provides confidence that Cloudie does not alter user data while securing it.

4.4.7 Malware Detection and Corroboration

A secure cloud storage system must ensure that files being uploaded are not malicious. This section evaluates the effectiveness of the malware detection capability, which combines the AI-based Random Forest classifier with corroboration from the VirusTotal API.

When a file is uploaded, it first passes through the feature extractor, which parses the structural characteristics of the PDF. These features are submitted to the Random Forest model, which produces a primary verdict. In parallel, the file hash is submitted to the VirusTotal API, which aggregates the results of many signature-based engines. A verdict engine then combines these two sources of evidence, treating the AI result as the primary signal and VirusTotal as a corroborating signal, and produces a final decision. Where the AI model flags a file on the basis of its structure but VirusTotal does not yet recognise it, the system records a dedicated structural-detection verdict. The system was tested under the following scenarios:

1. Uploading a clean file with no malicious indicators.
2. Uploading a known malicious PDF.
3. Uploading a file recognised by VirusTotal as safe.
4. Uploading a structurally suspicious file not yet present in the VirusTotal database.

Table 4.6 Results of Malware Detection and Corroboration

Scenario	Expected result	Actual result

Uploading a clean file	File accepted	File accepted
Uploading a known malicious PDF	File rejected	File rejected
Uploading a file known-safe to VirusTotal	File accepted	File accepted
Uploading a structurally suspicious file	Flagged by AI (structural)	Flagged by AI (structural)

As shown in Table 4.6, the system correctly accepted clean files and rejected malicious files. In the final scenario, the AI model flagged a structurally suspicious file that VirusTotal had not yet catalogued, demonstrating the value of the structural detection layer in identifying threats that signature-based engines may miss. A divergence between the AI verdict and the VirusTotal result is therefore expected behaviour rather than an error, because the two mechanisms detect threats in fundamentally different ways. The Random Forest model reasons over the structure of the file, while VirusTotal aggregates signature-based detections from external engines.

In addition, a log record is generated for every scanning activity, including the file hash, file type, AI verdict, VirusTotal result and timestamp. This per-scan log is written to a CSV file and provides an audit trail that can be used for security review. The evaluation confirms that the malware detection feature successfully prevents malicious files from being stored, while the corroboration design allows the strengths of both the AI model and the external scanning service to be combined.

4.5 Discussion

The results presented in this chapter show that Cloudie provides effective protection against malicious uploads through its AI-based detection layer. By analysing the structural features of each PDF using a trained Random Forest model and corroborating the result with the VirusTotal API, the system is able to identify and block malicious files before they are encrypted or stored. This proactive approach prevents threats such as malicious JavaScript, embedded payloads and exploit-bearing documents from entering the storage layer, thereby protecting both the server and other users.

A particular strength of the design is its use of an intelligence-driven, structural detection method rather than reliance on signatures alone. Because the Random Forest model reasons over structural indicators, it is able to flag novel or evasive files that have not yet been catalogued by signature-based engines. The held-out validation confirmed that the model generalises well and

that its high accuracy is not merely an artefact of a single random split. The verdict engine, which treats the AI result as primary and VirusTotal as corroborating, allows disagreements between the two sources to be handled transparently and recorded as a meaningful structural-detection outcome. This decision-fusion design is one of the principal contributions of the project.

The detection layer is complemented by the encryption layer, which provides confidentiality for stored data. Files that pass the malware check are encrypted using AES-256-CBC with a 256-bit key and a random initialisation vector, and the encryption keys are stored separately from the ciphertext. The performance tests showed that AES-256-CBC processes files of different sizes efficiently, with only a small encryption and decryption overhead. The integrity tests confirmed that the encryption and decryption cycle preserves the content of files exactly, with all hash comparisons matching before and after processing.

Taken together, the AI-based malware detection, AES-256-CBC encryption, separate key storage, data-integrity verification and the secure email-based recovery mechanism constitute a layered, defence-in-depth approach. The system is able to prevent the upload of malicious files while ensuring that stored data remains confidential and unaltered throughout its lifecycle.

4.6 Summary

This chapter has presented and discussed the results obtained from the development and testing of Cloudie. The system integrates an AI-based malware detection engine, built on a Random Forest classifier and corroborated by the VirusTotal API, with AES-256-CBC encryption to provide confidentiality for stored files. Functional testing confirmed that all core features operate correctly, and the machine learning evaluation, supported by a distribution-based held-out validation, demonstrated that the detection model achieves high accuracy and recall in a robust and defensible manner. Performance testing showed that encryption and decryption introduce only a small overhead, and the integrity checks confirmed that user data is preserved exactly throughout the encryption and decryption cycle. The combination of these mechanisms results in a secure and reliable system for storing sensitive files in the cloud.

CHAPTER 5 : CONCLUSION

5.0 Introduction

This chapter provides a comprehensive overview of the project as a whole, including its purpose, the benefits it offers, the limitations encountered, and recommendations for future work. It describes the extent to which the objectives of the project were achieved through the development of Cloudie, a secure cloud storage system that combines AI-based PDF malware detection with AES-256-CBC encryption. The chapter concludes with a summary of the principal points and an explanation of why the project is significant.

5.1 Purpose of the Project

The primary purpose of this project was to design a secure cloud storage system that not only scans uploaded files for malware but also encrypts them using the AES algorithm, so that files containing sensitive information remain protected. The system was intended to ensure that important data is accessible only to authorised users and that it remains secure, accurate and reliable. The project aimed to improve the safety of file storage by combining strong encryption with intelligent, proactive malware detection in order to help prevent data theft and unauthorised access.

The first objective, which was to design a secure cloud storage system incorporating a malware analyser mechanism and encryption for protecting highly sensitive files, was achieved through the development of a working web-based storage application. Cloudie allows users to store their files in a personal cloud space, with each uploaded file scanned for malware and encrypted before storage so that unauthorised users cannot access the files and malicious content cannot enter the system.

The second objective, which was to develop a malware analyser together with AES encryption to provide layered security for uploaded files, was met by combining an AI-based Random Forest detection engine, corroboration from the VirusTotal API, and AES-256-CBC encryption with separate key storage. Uploaded files are analysed and only clean files are stored, while the encryption ensures that the stored data remains confidential. In addition, a secure email-based recovery PIN mechanism was implemented to protect the account recovery process, replacing a simple key-handling step with a hashed, time-limited and single-use recovery code.

The final objective, which was to test the secure cloud system for its security and resilience through encryption and file upload accessibility, was fulfilled through systematic testing. Functional testing, machine learning evaluation with a held-out validation, encryption and decryption performance measurement, data integrity verification, and malware detection testing were all carried out. The results confirmed that the system secures files using AES-256-CBC, detects and blocks malicious uploads using the AI model, and preserves data integrity throughout the encryption and decryption process.

5.2 Benefits

Cloudie offers several important benefits. First, the use of AES-256-CBC encryption provides a high level of protection for stored files, ensuring that encrypted data cannot easily be read without the corresponding key and thereby safeguarding sensitive documents. Second, the AI-based malware detection engine identifies and blocks malicious files before they are stored, providing a proactive and intelligent layer of defence that is able to detect structurally suspicious files even when they are not yet recognised by signature-based scanners. Third, the corroboration with VirusTotal combines the strengths of structural analysis and signature-based detection, producing more informative verdicts than either approach alone. Finally, the system provides a user-friendly interface and a secure email-based recovery mechanism, allowing users to upload, scan, encrypt, decrypt and recover access to their files without requiring technical knowledge of the underlying security processes.

5.3 Limitations

Despite its strengths, the project has a number of limitations. First, the malware detection model is specialised for PDF files, because it relies on structural features extracted from the PDF format and was trained on the CIC-Evasive-PDFMal2022 dataset; it does not currently detect malware in other file types. Second, the computational overhead of scanning, encryption and decryption increases with file size, which may affect performance when handling very large files. Third, the corroboration step and the email-based recovery mechanism require a stable internet connection in order to reach the VirusTotal API and the email server, which can be a constraint in environments with poor connectivity. Finally, the system currently uses AES-256-CBC as its single encryption scheme; while AES is secure and efficient, supporting additional algorithms could increase flexibility for different use cases.

5.4 Future Recommendation

Several directions are recommended for future development. First, the malware detection capability could be extended beyond PDF files to other common formats such as Office documents and executables by training additional models on appropriate datasets, broadening the protection the system offers. Second, cross-dataset validation within the PDF malware domain could be pursued to further strengthen the evidence for the model's generalisation, taking care to avoid datasets that overlap heavily with the training data, since validating against a source from which the training set was constructed would be methodologically circular. Third, the system could be deployed to a cloud platform such as Google Cloud Run to provide a public, shareable deployment that is easy to access and evaluate. Fourth, the encryption layer could be extended to support additional algorithms such as RSA or Elliptic Curve Cryptography, giving users a choice of scheme. Finally, the detection model and the security components of the system should be reviewed and updated regularly so that the system remains effective against newly emerging threats.

5.5 Conclusion

In conclusion, this project successfully delivered Cloudie, a secure cloud storage system that combines AI-based PDF malware detection with AES-256-CBC encryption to significantly enhance the security of stored data. The Random Forest detection engine, corroborated by the VirusTotal API through a transparent verdict-fusion design, provides a proactive defence against malicious uploads, while AES-256-CBC encryption with separate key storage ensures the confidentiality of stored files. A secure email-based recovery mechanism adds a further layer of protection to account access. The system provides an intuitive user experience while upholding the principles of confidentiality, integrity and availability. Although a number of limitations were identified, the project provides a solid foundation for future development, and the recommended enhancements would make the system even more robust and flexible for the secure storage of files in the cloud.

REFERENCES

Savitha, N. (2023). Privacy-Preserving Techniques for Secure Cloud Computing: A Survey of Recent Advances. *International Journal of Scientific Research in Science and Technology*, 736–742. https://doi.org/10.32628/ijsrst523103129

Awasthi, S., & Kohli, N. (2025). Efficient Data Encryption for Securing HDFS Using DQN-Enhanced Deep Reinforcement Learning Techniques. *Journal of Computer Science*, *21*(4), 741–760. https://doi.org/10.3844/jcssp.2025.741.760

Cosovan, D., Benchea, R., & Gavrilut, D. (2015). A practical guide for detecting the java script-based malware using hidden markov models and linear classifiers. *Proceedings - 16th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing, SYNASC 2014*, 236–243. https://doi.org/10.1109/SYNASC.2014.39

Deldar, F., & Abadi, M. (2024). Deep Learning for Zero-day Malware Detection and Classification: A Survey. *ACM Computing Surveys*, *56*(2). https://doi.org/10.1145/3605775

Han, S. H., Han, J., Lee, D., & Choi, H. J. (2025). Quality Assurance Framework for Multimodal Assessment Datasets on AI Risk Factors. *Proceedings of the IEEE International Conference on Big Data and Smart Computing, BIGCOMP*, *2025*, 401–405. https://doi.org/10.1109/BigComp64353.2025.00082

Kim, Y. C., Lee, C., & Yoon, Y. (2025). Payload-Aware Intrusion Detection with CMAE and Large Language Models. *ACM Transactions on Privacy and Security*. https://doi.org/10.1145/3769682

Koch, L., Oesch, S., Sadovnik, A., Weber, B., Chaulagain, A., Dixson, M., Dixon, J., Huettel, M., Watson, C., Hartman, J., & Patulski, R. (2025). On the Abuse and Detection of Polyglot Files. *WWW 2025 - Proceedings of the ACM Web Conference*, 4810–4822. https://doi.org/10.1145/3696410.3714814

Kumar, R., Gujjar, J. P., Prasad, M. S. G., Devadas, R. M., Preethi, & Kumar, U. (2025). Balancing the Imbalanced Datasets for Machine Learning Classification Problems. *Proceedings - International Conference on Next Generation Communication and Information Processing, INCIP 2025*, 1–4. https://doi.org/10.1109/INCIP64058.2025.11019446

Liu, R., Matuszek, C., & Nicholas, C. (2023). *A PDF Malware Detection Method Using Extremely Small Training Sample Size*. https://doi.org/10.1145/3

Liu, S., Ming, J., Zhou, G., Liu, X., Fu, J., & Peng, G. (2025). *Analyzing PDFs like Binaries:*

Adversarially Robust PDF Malware Analysis via Intermediate Representation and Language Model. 1334–1348. https://doi.org/10.1145/3719027.3744829

Patel, A., & Ghosh, S. M. (2024). AMD-XAI-ML: Android Malware Detection based on an Explainable AI using Machine Learning for Smart Computing Environment. *2024 OPJU International Technology Conference on Smart Computing for Innovation and Advancement in Industry 4.0, OTCON 2024*. https://doi.org/10.1109/OTCON60325.2024.10687629

Shaikh F. (2011). *2011 International Conference for Internet Technology and Secured*

Transactions : 11-14 December 2011 : Abu Dhabi, United Arab Emirites. IEEE.

Sinha, A. K., & Sai, S. (2023). Integrated Malware Analysis Sandbox for Static and Dynamic Analysis. *2023 14th International Conference on Computing Communication and Networking Technologies, ICCCNT 2023*. https://doi.org/10.1109/ICCCNT56998.2023.10306805

Vielhauer, C., Loewe, F., & Pilgermann, M. (2025). *Towards Modeling Hidden & Steganographic Malware Communication based on Images*. 52–63. https://doi.org/10.1145/3733102.3733152

Yahva, F., Mahibol, R. I. W., Ying, C. K., Anai, M. Bin, Frankie, S. A., Wei, E. L. N., & Utomo,
R. G. (2021). Detection of Phising Websites using Machine Learning Approaches. *2021 International Conference on Data Science and Its Applications, ICoDSA 2021*, 40–47. https://doi.org/10.1109/ICoDSA53588.2021.9617482

APPENDIX A: AI DETECTION REPORT

APPENDIX B: SIMILARITY INDEX RECEIPT